GPU ARCHITECTURE & PARALLEL PROGRAMMING

from First Principles



Arpan Pathak

IN CUDA C++ AND RUST

Preface

In this book we program NVIDIA GPUs in CUDA C++ and in Rust with NVIDIA's cuda-oxide, and we explain the speed of each program from a model of the hardware. We build that model in the first 2 chapters. In Chapter 1 we derive it from the latency and throughput of memory and from the limits on parallel speedup; in Chapter 2 we add the streaming multiprocessor, its warps and the GPU memory system. The later chapters apply the model to reduction, scan, histogram and matrix multiplication. Each kernel is analysed by hand with small numbers, implemented in both languages, and measured against the bandwidth and arithmetic peak of the GPU. Readers need some experience of C++ or Rust and no previous experience of GPU programming.

I bought the GPU hardware for these experiments with a large part of my severance pay. Such hardware is expensive, and many people who want to learn parallel programming cannot buy it. Without it, a learner can read that coalesced memory accesses are faster than scattered ones but cannot measure by how much. The experiments began as small programs from linear algebra and calculus, written to compare the CPU and the GPU on the same task, and their measurements often contradicted my expectations.

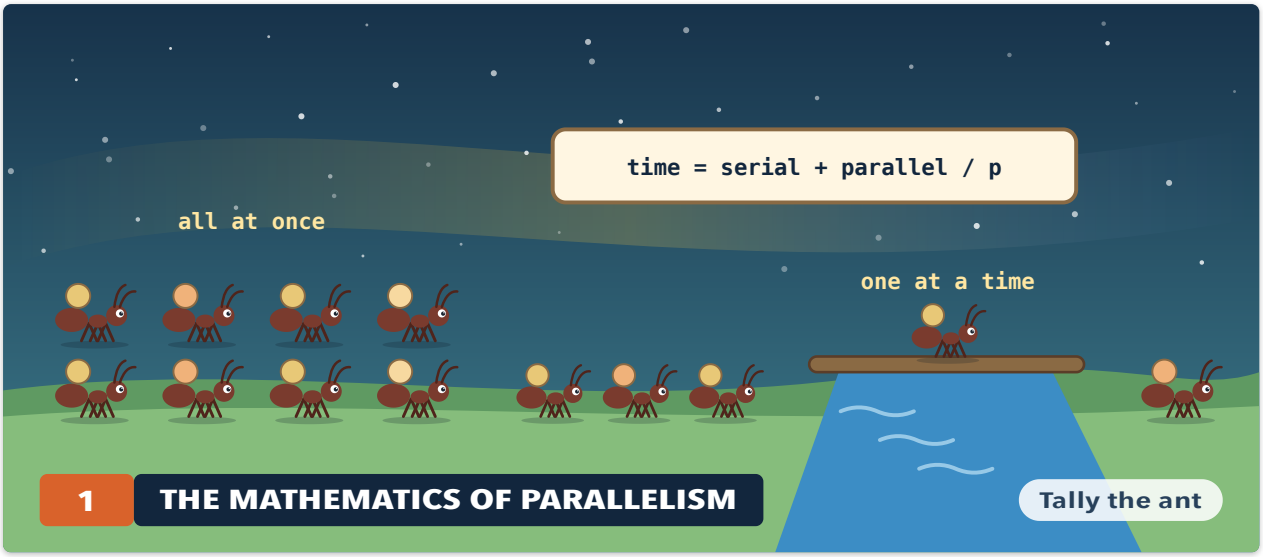
Access to GPU hardware

To give others the same opportunity, I run a private GPU cloud, Thor Thunder Tigress, which can be reached through a tunnel network. Anyone may use it free of charge to continue these experiments or for other educational work. To request access, open an issue at `github.com/arpanpathak/gpu-parallel-book`.

Review draft

The draft contains Chapters 1 and 2. I welcome corrections, from a wrong number to an unclear sentence; please report them as issues at the same address.

Arpan Pathak



Chapter 1: The Mathematics of Parallelism

A *parallel program* divides one computation among several processors that run at the same time. How much sooner such a program finishes than a sequential one can be estimated before any code is written. The estimate needs 3 things: how long each operation takes, how much of the computation can be divided, and how many bytes each operation reads from memory.

For several decades, programs became faster without any effort from the people who wrote them. Each generation of microprocessors ran at a higher clock rate than the one before, and the same sequential program finished sooner on the newer chip. That trend ended in the mid-2000s. A higher clock rate requires more power, and the power drawn by a chip becomes heat that must be removed from it; at clock rates of a few gigahertz, the heat reached the limit of what a chip package and its cooling could carry away. Since then, chip designers have spent the additional transistors of each generation on more processors per chip, and a program becomes faster on a newer chip only if it divides its work among those processors. A GPU contains thousands of simple arithmetic units and depends entirely on parallel programs to keep them busy.

In this chapter we develop the quantitative tools that we use throughout the book to reason about the performance of such programs. They do not depend on any

particular machine. A *processor* here is any unit that executes instructions: a core of a CPU, or one of the threads of a GPU, which we introduce in section 1.1.5. We begin with the time that individual operations take and the way in which overlapping them hides that time (section 1.1). We then ask how much faster a program can become on more processors, and find that the answer depends on the part of it that cannot be divided (sections 1.2 to 1.5). Sections 1.6 and 1.7 classify the ways in which work divides and the machines that execute it. In section 1.8 we introduce the model that decides whether memory or arithmetic limits a computation, and we use it in later chapters to judge every kernel. Sections 1.9 and 1.10 account for the costs that parallel execution adds: waiting at barriers, and performing more operations than a sequential program would.

THIS CHAPTER COVERS

- cycles, latency and throughput, and how they differ
- concurrency, and Little's law, which relates it to latency and throughput
- dependencies, and why independent operations can overlap
- speedup and efficiency
- Amdahl's law for a fixed computation, and Gustafson's law for a growing one
- Amdahl's law when only part of a program moves to a GPU, and the copies it needs
- strong and weak scaling
- task, data and pipeline parallelism, and Flynn's classes of machines
- arithmetic intensity and the roofline model
- the cost of a barrier
- work, steps, and why a parallel algorithm that does extra work can be slower

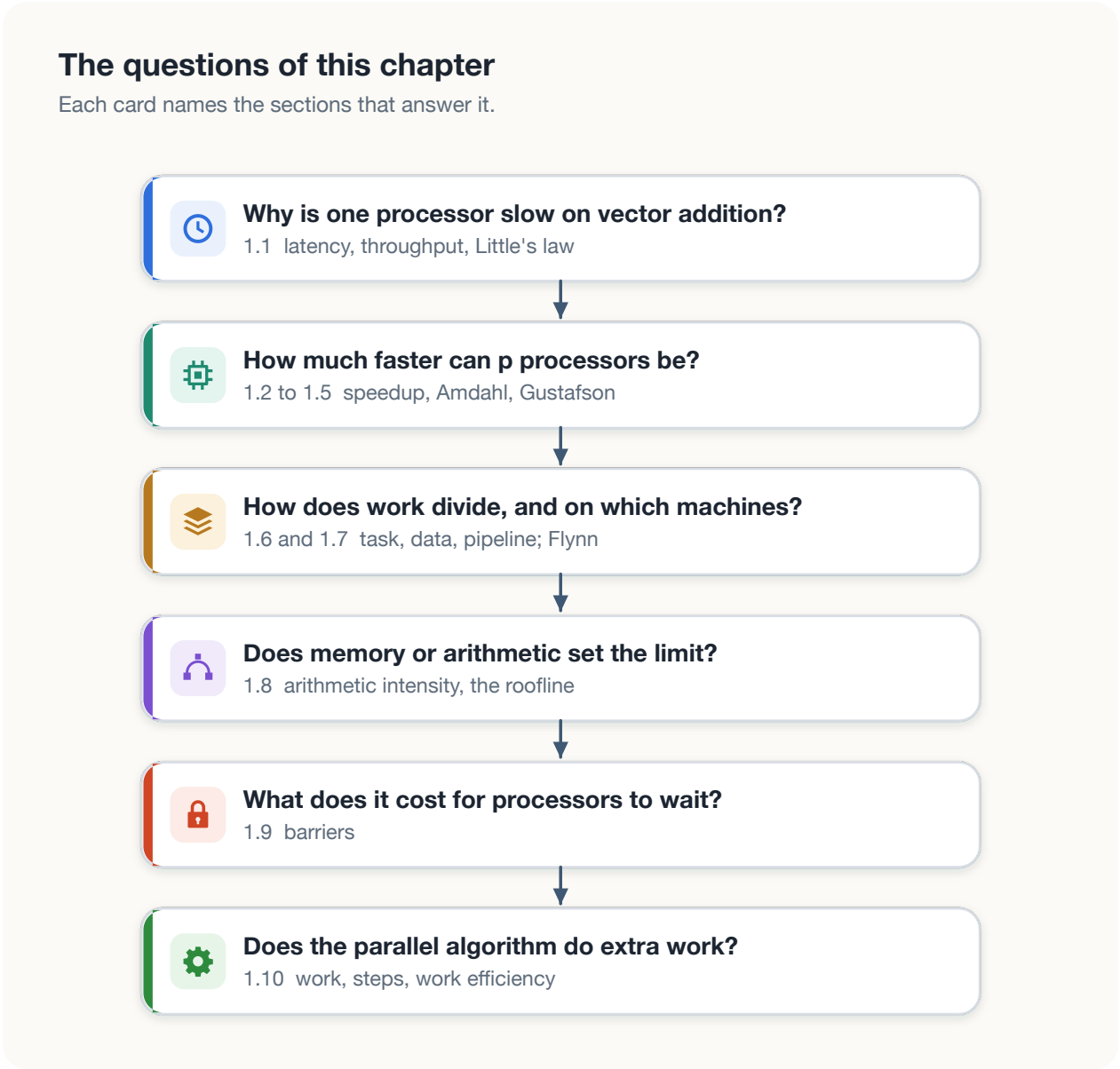


Figure 1.1 The questions of this chapter, and the sections that answer them.

The running example: vector addition

Most examples in this chapter use one computation, chosen because it is the simplest computation that a parallel machine can speed up. 2 arrays of numbers, `a` and `b`, each hold `N` entries, and a third array `c` receives their sum, entry by entry:

```
c[i] = a[i] + b[i]      for every i from 0 to N - 1
```

For `N = 4`:

i	a[i]	b[i]	c[i]
0	0.0	0.0	0.0
1	1.0	2.0	3.0
2	2.0	4.0	6.0
3	3.0	6.0	9.0

Each entry `c[i]` is computed from `a[i]` and `b[i]` alone; no entry of `c` needs another entry of `c`. Computations with this property are called **independent**. They give the same result in any order, and they can therefore run at the same time on different processors. Almost everything a GPU does well rests on this property, and much of the difficulty of parallel programming, which later chapters address, arises in computations that lack it.

1.1 Latency, throughput, and concurrency

A sequential program on a modern processor spends most of its time waiting for data, and very little of it computing. In this section we measure that waiting and show how a machine can hide it by keeping many operations under way at once. The argument introduces 3 quantities, latency, throughput and concurrency, and one law that connects them. The same law gives the number of threads a GPU must hold, which we work out for an H100 in section 1.1.5.

1.1.1 Cycles

A processor performs its work in steps timed by a clock, and one tick of the clock is a **cycle**. At a clock rate of 1 GHz there are 10^9 cycles per second, so a cycle lasts 1 nanosecond; at 2 GHz it lasts half a nanosecond. We count time in cycles because the number of cycles an operation takes changes much less from one machine to another than its duration in nanoseconds. A processor whose clock runs twice as fast completes each cycle in half the time, but an addition still occupies roughly the same number of cycles on it.

1.1.2 Latency

The **latency** of an operation is the time from the moment it starts to the moment its result can be used. The latencies of the 2 operations of vector addition differ by a factor of more than 100.

Operation	Latency
add 2 numbers that are already in the processor	about 4 cycles
read one number from main memory	about 500 cycles

The difference of more than 100 times comes from distance and from the kind of storage. An addition works on values held in registers inside the processor, a short distance from the arithmetic unit, and passes them through a few layers of logic. A read from main memory leaves the processor, crosses the chip to a memory controller, waits in a queue behind other requests, and travels over wires to separate memory chips. There a row of storage cells is opened and read, and the value returns by the same route. Each of these steps is fast by itself, but there are many of them, and the slowest, reading the storage cells, has hardly become faster in decades.



Why is a read from main memory so slow? Main memory is built from DRAM, dynamic random-access memory. A DRAM chip stores each bit as a small electrical charge in a capacitor, and connects thousands of these capacitors to each long wire, called a bit line, that leads to a detector called a sense amplifier. To read a bit, the chip first raises a row line that connects every cell of one row to its bit line. The small charge of the cell then spreads over the much larger bit line and shifts its voltage by a tiny amount, and the sense amplifier must wait until that shift is large enough to tell a 1 from a 0. Both steps take tens of nanoseconds, which is tens to hundreds of cycles of a processor that runs at 1 to 2 GHz. The process could be made faster with larger capacitors that hold more charge. DRAM makers have done the opposite: they have shrunk the capacitors in each generation to fit more bits on a chip. As a result, the capacity and the bandwidth of DRAM have grown by large factors over the decades, while its latency has changed little. In Chapter 2 we describe how DRAM delivers many bytes from each opened row, which is the property that GPUs exploit.

NOTE: The latencies in this chapter, and the 40 TFLOP/s machine of section 1.8, are round figures chosen to keep the arithmetic easy. They have the right order of magnitude, and in Chapter 2 we replace them with measured values for real GPUs.



Suppose one processor computes the 4 rows of the example, one row after another, and issues the reads of a row only after it has finished the previous row.

WORKED EXAMPLE: ONE PROCESSOR, 4 ROWS

1. The processor requests `a[0]` and `b[0]`. The 2 reads travel together, so both values arrive 500 cycles later.
2. It adds them: 4 cycles.
3. It sends `c[0]` towards memory. A processor does not wait for a write to complete, so it moves straight on to the next row.
4. Rows 1, 2 and 3 repeat the same steps.

Each row takes 504 cycles, the 4 rows take **2,016 cycles**, and the adder is busy for **16** of them.

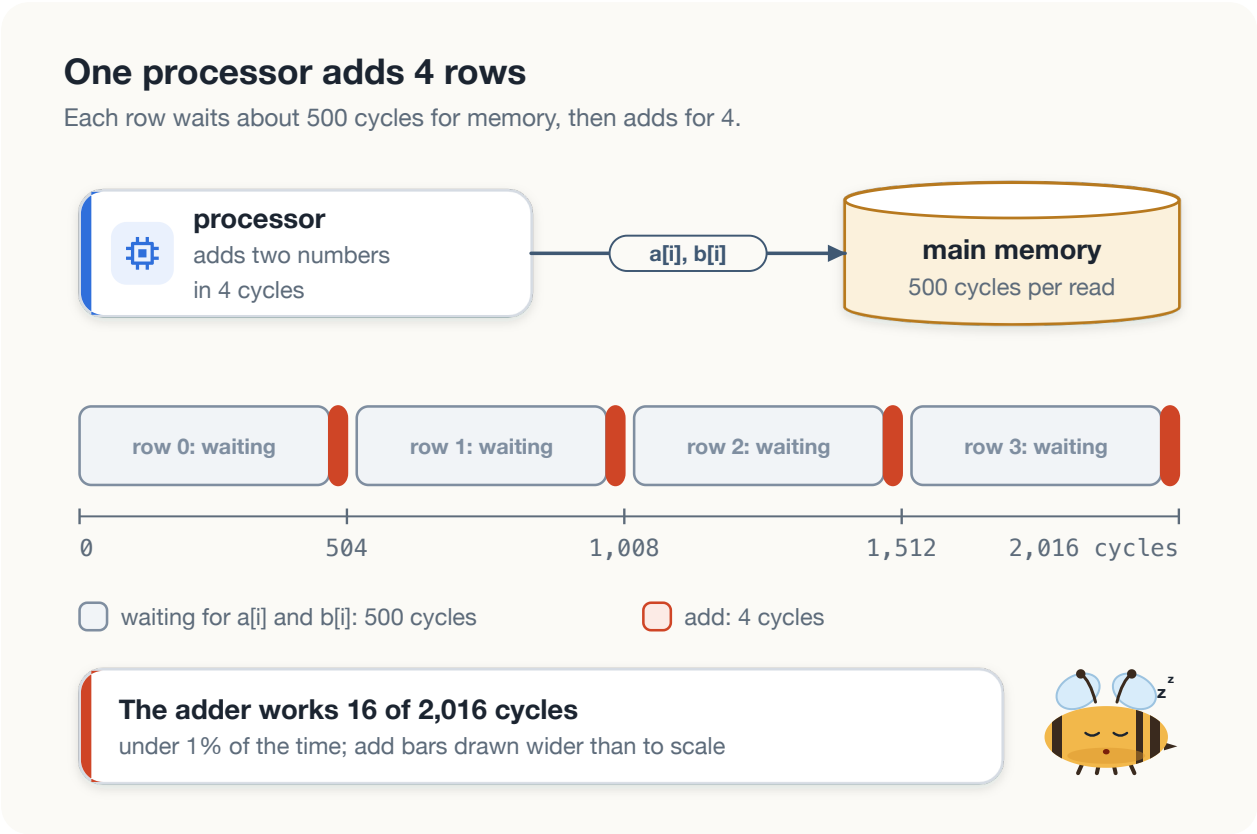


Figure 1.2 One processor computes 4 rows. Each row waits 500 cycles for its 2 reads, then adds for 4.

It may seem that a faster adder would shorten this run. It would not: the adder accounts for 16 of the 2,016 cycles, and an infinitely fast adder could save at most those 16. The run is long because the processor waits for each read before it asks for the next one, so the reads take place one after another and their latencies add up. The latency of a memory read is a property of the hardware, and the program cannot shorten it. What the program can change is how many reads it has under way at the same time. We treat that number in sections 1.1.3 to 1.1.5.



The processor in figure 1.2 spends more than 99% of the run waiting for memory.

1.1.3 Throughput

The **throughput** of a system is the rate at which it completes operations: the number of operations it completes during an interval of time, divided by the length

of the interval. Latency is a property of one operation, and throughput is a property of a stream of operations, so the 2 answer different questions. Latency tells us how long one result takes to arrive. Throughput tells us how many results arrive per unit of time once a steady stream of operations is under way.

A related quantity is **bandwidth**: the maximum amount of data that a connection or an electronic system, such as a memory system, can transfer in a given amount of time. Bandwidth is a capacity of the hardware. Throughput is the rate that a stream of operations actually achieves, and it can be anything up to that capacity. A memory system with a bandwidth of 1 TB/s that receives one read at a time achieves a small fraction of it, as the processor of figure 1.2 does. We use the bandwidth of a GPU's memory in section 1.8.

If a system worked on only one operation at a time, its throughput would follow from its latency: with reads that take 10 cycles, it would complete one read every 10 cycles. Many systems instead accept a new operation before the earlier ones have finished, and their throughput can then be much higher than one operation per latency. Memory systems work this way. A memory is divided into many independent parts, called channels and banks, and each part can work on a different read at the same time; we describe these parts of a GPU in Chapter 2.

Consider a simplified memory system in which every read takes 10 cycles, and which accepts a new read every 2 cycles. Figure 1.3 shows 8 reads sent to it, one every 2 cycles.



Figure 1.3 Reads that take 10 cycles each, started 2 cycles apart.

Read	Starts at cycle	Completes at cycle
0	0	10
1	2	12
2	4	14
3	6	16

From cycle 10 onwards a read completes every 2 cycles, so the throughput is 0.5 reads per cycle. Each individual read still takes 10 cycles. Starting reads in quick succession did not make any of them faster; it made them overlap. Most optimisations in this book work the same way: they raise throughput by overlapping operations, and leave the latency of each operation as it was.

1.1.4 Concurrency and Little's law

The reads that have started and not yet completed are said to be **in flight**, and their number is the **concurrency** of the system. The dashed line in figure 1.3, at cycle 10.5, crosses reads 1 to 5, so 5 reads are in flight at that moment. A line drawn at any later cycle, until the stream of reads ends, also crosses 5.

The count of 5 follows from the 2 rates. Each read stays in flight for 10 cycles, and while it does, a new read starts every 2 cycles, so $10 / 2 = 5$ reads start during its lifetime. When it completes, those 5 are the reads still in flight.

We write L for the latency (10 cycles here), X for the throughput (0.5 reads per cycle) and C for the concurrency (5 reads). The same argument holds for any system that runs at a steady rate:

$$C = X \times L \qquad \text{or equivalently} \qquad X = \frac{C}{L}$$

This relation is **Little's law**. John Little proved it in 1961 for queues of every kind: the average number of customers in a shop equals the rate at which they arrive multiplied by the average time each one stays. For the reads of figure 1.3, $0.5 \times 10 = 5$, the number we counted.

The second form, $X = C/L$, is the one that matters for performance. When the latency is fixed by the hardware, the only way to raise the throughput is to raise the concurrency: twice as many reads in flight complete twice as many reads per cycle. The law also works in the other direction. A memory system that can deliver a certain throughput only delivers it if the program keeps $X \times L$ reads in flight; with fewer, the throughput falls in proportion, however high the bandwidth printed on the data sheet.



A latency that cannot be shortened can still be hidden, by keeping enough operations in flight.

▶ CHECK YOUR UNDERSTANDING: **A memory read takes 400 cycles, and the memory accepts a new read every 4 cycles. How many reads must be in flight to keep it busy?**

1.1.5 Where the concurrency comes from

We now return to main memory, with its 500-cycle reads, and suppose that it accepts a new read every 2 cycles. To keep it busy, Little's law asks for

$$C = X \times L = 0.5 \times 500 = 250$$

reads in flight at all times. The processor of figure 1.2 has at most 2 in flight. It receives 2 values every 500 cycles from a memory that could have delivered 250 in the same time (figure 1.4).

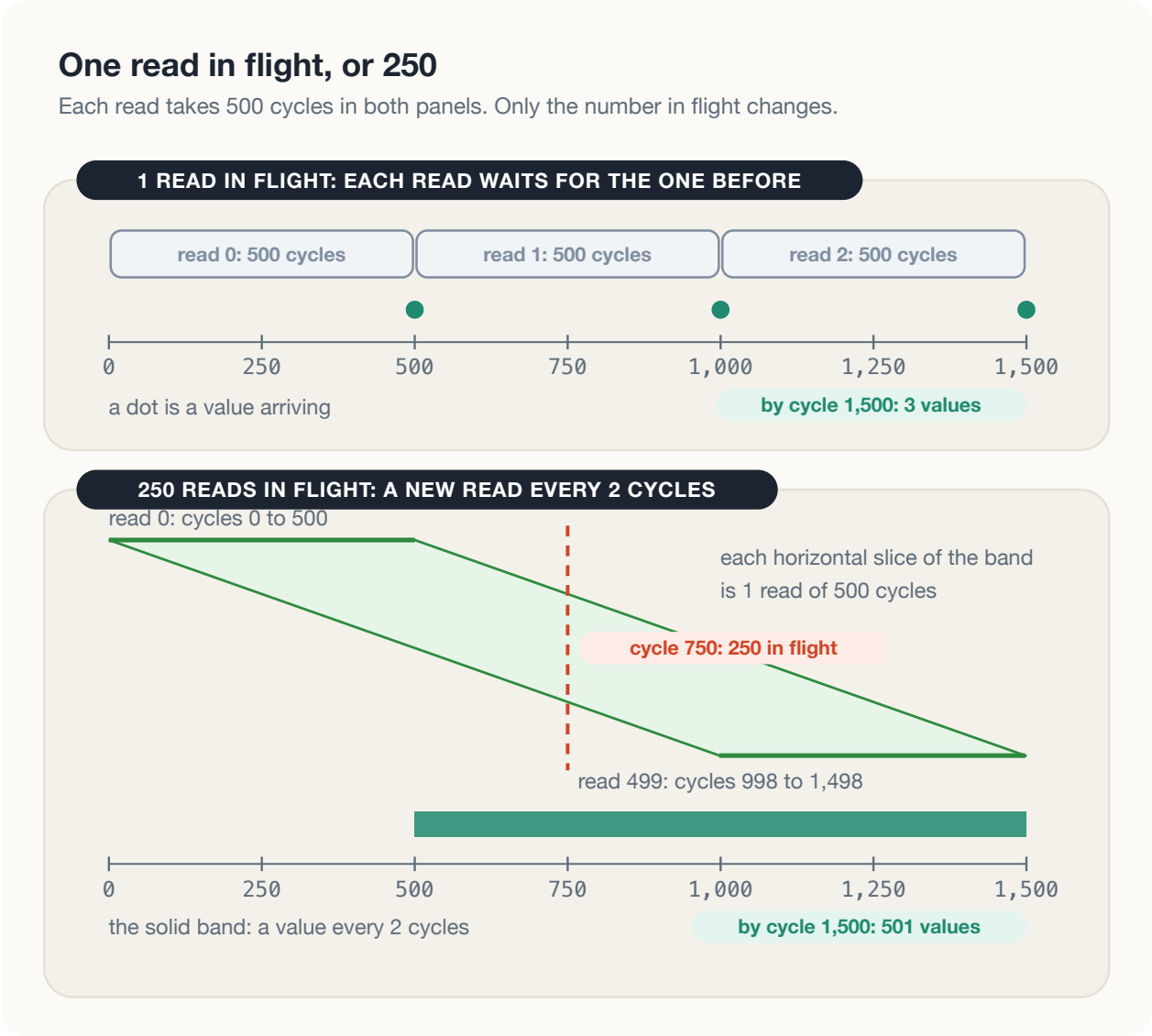
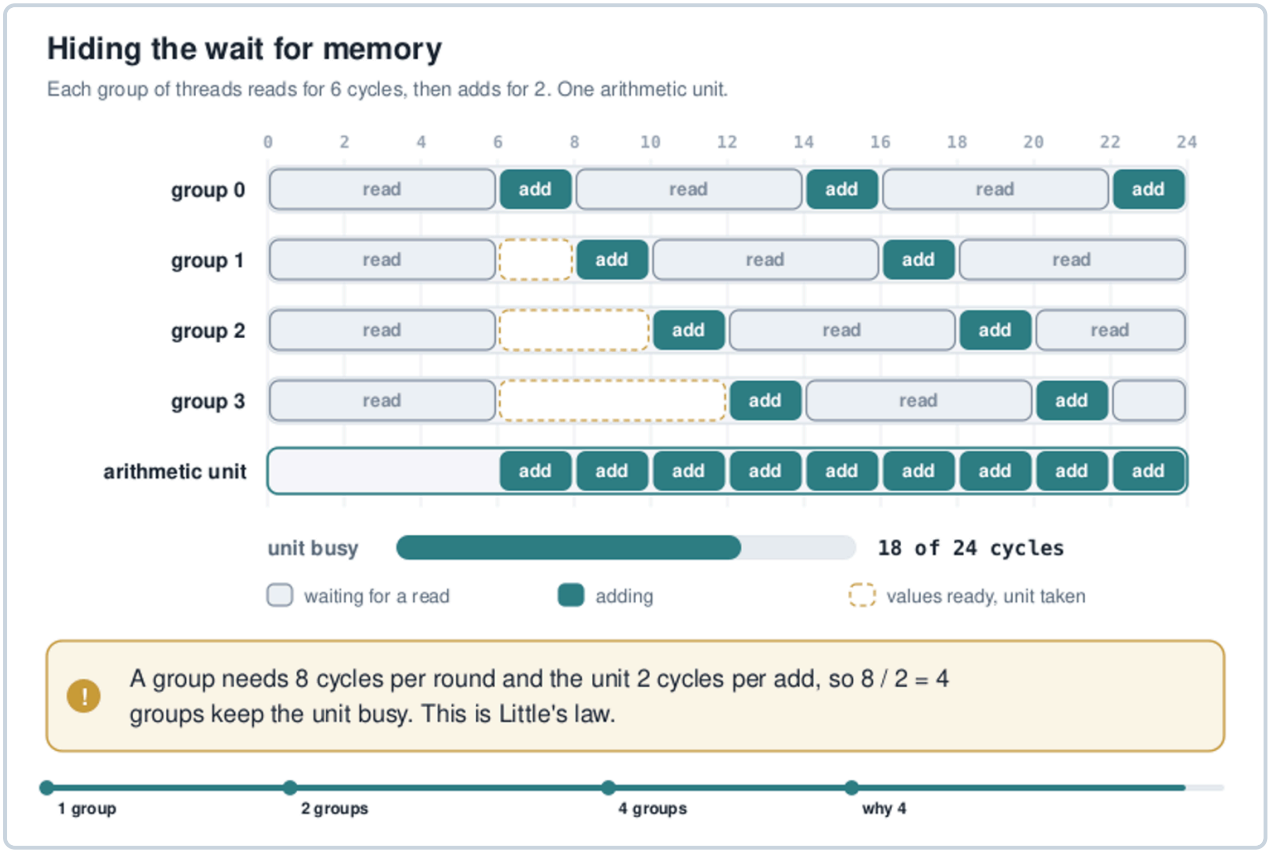


Figure 1.4 The same 500-cycle read latency with 1 read in flight and with 250.

The 250 reads must come from the computation, and its structure decides how many it can offer. An operation **depends** on another when it needs that operation's

result. In one row of vector addition, the add depends on both reads and the write depends on the add. A sequence of operations in which each depends on the one before is a **dependency chain**, and no 2 operations of the same chain can overlap. One row is a short chain, so it can offer at most its 2 reads at once.

The rows, however, are independent. The reads of row 7 do not need anything from row 6, so a program can issue the reads of 125 rows before the first of them returns. A GPU does this with threads. A **thread** is a sequence of instructions with its own **registers**, the small storage slots inside a processor that each hold one value. A GPU runs thousands of threads and gives each a few rows. While some threads wait for their values, others already have theirs and keep the arithmetic units busy (animation 1).



Animation 1 Groups of threads share one arithmetic unit. Each read takes 6 cycles and each add 2. With 4 groups the unit never waits.

The animation uses small numbers so that every cycle is visible. A group needs 8 cycles for one round, a read and an add, and the unit finishes an add every 2 cycles. Little's law gives the number of groups that keeps it busy: $C = 8 \times \frac{1}{2} = 4$. A

GPU applies the same arithmetic with latencies of hundreds of cycles, which is why it holds thousands of threads at once.

Applied to bytes instead of reads, the same law gives the number of threads that a real GPU needs. Little's law holds for any unit of work, so the number of bytes that must be in flight is the bandwidth multiplied by the latency.

WORKED EXAMPLE: HOW MANY THREADS AN H100 NEEDS FOR VECTOR ADDITION

1. The H100's memory delivers $X = 3.35$ TB/s (section 1.8).
2. A read from its memory takes about 500 cycles (section 2.6). At its clock rate of 1.98 GHz (section 2.2), 500 cycles last $500 / (1.98 \times 10^9)$ s = 253 ns.
3. Bytes in flight: $C = X \times L = 3.35 \times 10^{12}$ B/s $\times$ 253×10^{-9} s = 848,000 bytes, about 0.85 MB.
4. One row of vector addition reads `a[i]` and `b[i]`, 8 bytes. A thread that works on one row at a time has 8 bytes in flight, so the memory stays busy only with 848,000 / 8 = **106,000 threads** waiting at every moment.

An H100 can hold 132 SMs $\times$ 2,048 threads = 270,336 threads at once (section 2.9). Vector addition needs about 40% of them in flight just to keep the memory busy. A kernel that holds fewer threads, or gives each thread a long dependency chain, runs below the memory's bandwidth however simple its arithmetic. A CPU core keeps only a few reads outstanding and relies on its caches to make most reads short. The number of threads a GPU holds follows from Little's law and the latency of DRAM.

Why not reduce the latency instead? A CPU and a GPU read from the same kind of DRAM, with the same latency of hundreds of cycles. They differ in what they spend chip area on. A CPU core is designed to make one thread fast. It spends much of its area and power on large caches that turn many slow memory reads into fast ones, and on control logic that predicts branches and reorders instructions so that one thread rarely waits. This approach is called **latency-oriented design**. A GPU is designed to make many threads fast in total. It spends the same area on more arithmetic units and more memory channels, lets each operation take a long time, and relies on other threads to fill the wait. This approach is called **throughput-oriented design**. The choice follows from cost. Doubling the throughput of an arithmetic unit takes a second unit, about twice the area and power. Halving its latency requires faster circuits, which cost disproportionately more area and power. Per unit of area and power, throughput costs less than latency, provided the program supplies the threads. A GPU therefore performs poorly on a program with one thread or a few, and a CPU performs poorly on a program with many; most applications that use a GPU run their sequential parts on the CPU and their data-parallel parts on the GPU. We compare the 2 designs in hardware in section 2.1.1.



1.2 Speedup and efficiency

Section 1.1 made one processor faster by overlapping its reads. The other way to finish sooner is to share the work among several processors. In this section we define the 2 measures of the result. We write p for the number of processors, T_1 for the running time on one processor and T_p for the running time on p.

Consider a program with 3 phases that run in order: it reads its input for 20 microseconds (μs), computes for 100 μs and writes its result for 10 μs, so $T_1 = 130$ μs. Suppose that the compute phase can be shared evenly among any number of processors, but that the read and the write pass through a single channel and cannot be shared.

WORKED EXAMPLE: THE PROGRAM ON 1, 2, 4 AND 8 PROCESSORS

p	Compute (μs)	Read + write (μs)	T_p (μs)	Saved by the last doubling (μs)
1	100	30	130	
2	50	30	80	50
4	25	30	55	25
8	12.5	30	42.5	12.5

Every doubling of the processors saves half as much time as the doubling before it, because only the compute phase shrinks. 2 ratios describe the result. The **speedup** compares the running times:

$$S(p) = \frac{T_1}{T_p}$$

On 2 processors $S(2) = 130/80 = 1.63$. The **efficiency** divides the speedup by the number of processors, and so measures how much of the added capacity turned into speed:

$$E(p) = \frac{S(p)}{p}$$

p	T_p (μs)	Speedup S	Efficiency E
1	130	1.00	1.00
2	80	1.63	0.81
4	55	2.36	0.59
8	42.5	3.06	0.38

At 8 processors the efficiency has fallen to 0.38. During the 30 μs of reading and writing, 7 of the 8 processors have nothing to do, and those 30 μs are a larger share of each shorter run. At 8 processors, 62% of the processor time paid for is idle.

The speedup is normally no greater than p, and the efficiency no greater than 1. The exception, **superlinear speedup**, appears when each processor's share of the data becomes small enough to fit in its cache, a small fast memory beside the

processor. Each processor then runs faster than the single processor did, and the combined speedup can exceed p. The extra speed comes from the cache, so the program loses it once its data no longer fits.

> **TIP:** Report the efficiency with the speedup. A speedup of 10 sounds good until it turns out to need 64 processors, an efficiency of 0.16.



A speedup is only as meaningful as its baseline, the run it divides by. T_1 should be the time of the best sequential program for the task. The parallel program run on 1 processor is a weaker baseline, because it still carries the work of dividing the data and combining the results, so it makes T_1 larger and the speedup look better. The same rule applies between machines. A GPU program compared with 1 CPU core running plain loops shows a larger speedup than the same program compared with every core of the CPU using its vector instructions (section 1.7). Only the second comparison tells the reader whether the GPU is worth using.

1.3 Amdahl's law

The table of section 1.2 showed the speedup levelling off as processors were added. In this section we explain why and give the limit a formula. In 1967 Gene Amdahl, the chief architect of IBM's System/360, used a short argument to show that machines with many processors would gain less than their designers hoped. Part of every program, he observed, can only be executed by one processor at a time, and that part limits what the rest of the machine can achieve. His argument fits in the table of section 1.2, extended to many more processors.

The **serial part** of a program is the part that only one processor can execute at a time; the **parallel part** is the part that can be shared. In our program the read and the write, 30 μs, are serial, and the compute phase, 100 μs, is parallel.

WORKED EXAMPLE: MORE AND MORE PROCESSORS

p	Parallel part (μs)	Serial part (μs)	T_p (μs)	Speedup
1	100	30	130	1.00
8	12.5	30	42.5	3.06
100	1	30	31	4.19
1,000	0.1	30	30.1	4.32
unlimited	approaches 0	30	approaches 30	approaches 4.33

However many processors share the parallel part, the 30 serial microseconds remain. The program can never run in less than 30 μs, and so never more than 130 / 30 = 4.33 times faster than on one processor.

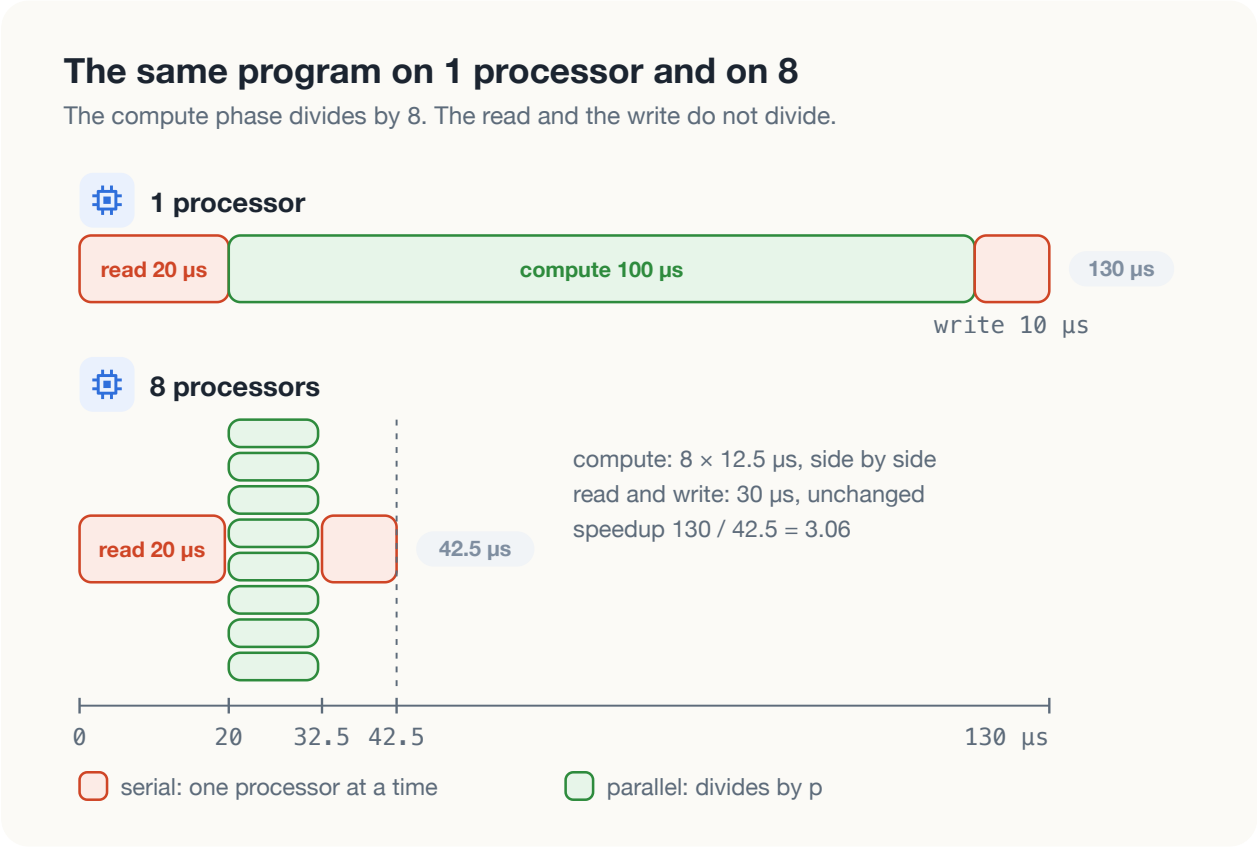


Figure 1.5 The program on 1 processor and on 8.

To state the result for any program, let f be the **serial fraction**: the share of the one-processor time that is serial. For our program $f = 30/130 = 0.23$, and the parallel share is $1 - f = 0.77$. On p processors the serial part keeps its length and the parallel part is divided by p:

$$T_p = f \cdot T_1 + \frac{(1 - f) \cdot T_1}{p}$$

For $p = 8$ this is $30 + 100/8 = 42.5\ \mu s$, as in the table. As p grows, the second term vanishes and T_p approaches $f \cdot T_1$. The speedup therefore approaches

$$S_{max} = \frac{T_1}{f \cdot T_1} = \frac{1}{f}$$

which is **Amdahl's law**. For our program, $1/0.23 = 4.3$. Dividing T_1 by T_p gives the speedup on a particular number of processors:

$$S(p) = \frac{1}{f + \dfrac{1 - f}{p}}$$



A program with serial fraction f can never run more than $1/f$ times faster, however many processors it is given.

1.3.1 How small the serial fraction must be

The limit rises steeply as the serial fraction falls. The table also gives the number of processors at which a program reaches half of its limit.

Serial fraction f	Limit $1/f$	Processors for half the limit
0.25	4	3
0.10	10	9
0.05	20	19
0.01	100	99
0.001	1,000	999

The last column follows from the formula for T_p. The speedup is half of its limit when the shared parallel part has shrunk to the size of the serial part, $(1 - f)/p = f$, that is at $p = 1/f - 1$. Beyond that point each additional processor divides an ever smaller remainder. A GPU runs thousands of threads, and a program can only make use of them if its serial fraction is a small fraction of a percent (figure 1.6). A program that spends 1% of its time in serial code cannot run more than 100 times faster, on a GPU with 100 threads or with 100,000.

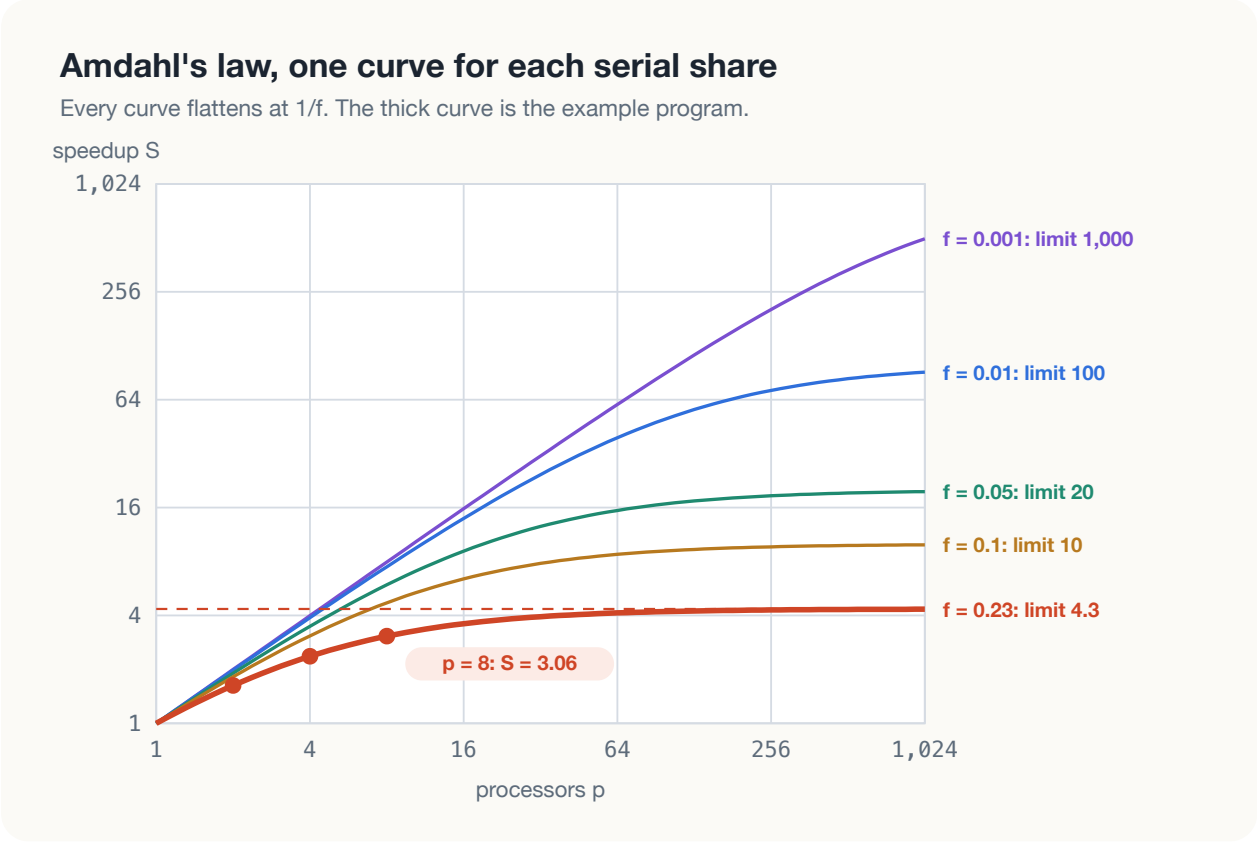
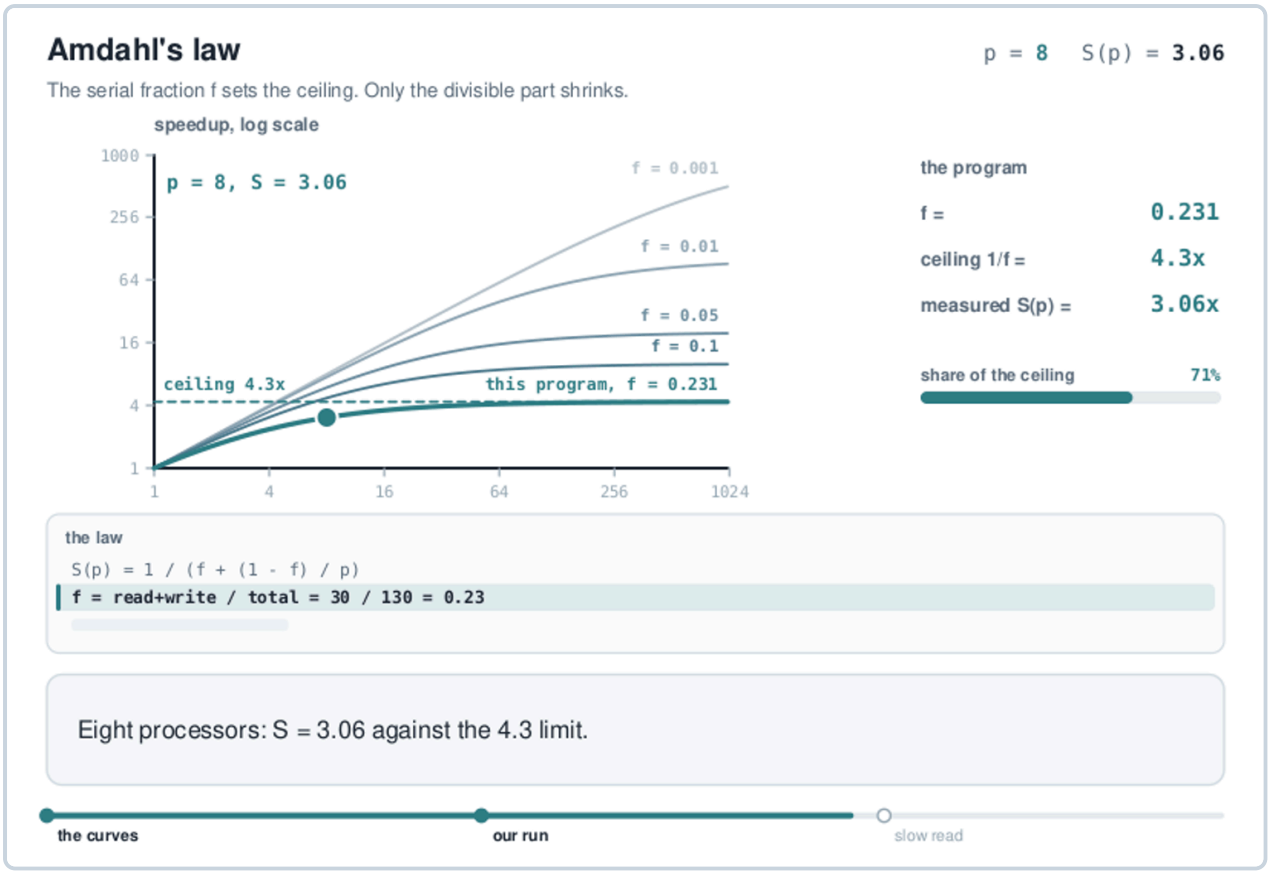


Figure 1.6 Amdahl's law for 5 serial fractions. Each curve flattens at its limit, 1/f.

Animation 2 follows the example along its curve, then lengthens the read, which raises f and lowers the limit.



Animation 2 The example on the f = 0.23 curve, up to 8 processors. In the last step a slower read raises f to 0.375 and lowers the limit to 2.7.

▶ CHECK YOUR UNDERSTANDING: **A program spends 5% of its one-processor time in a serial part. What is the highest speedup any number of processors can give it?**

1.3.2 Where the serial part comes from

The serial part of a real program is rarely a single phase that is easy to point to. It includes starting the program and loading its input, combining the processors' partial results at the end, and every step that has to wait for the one before it. It is often larger than its author expects, and measuring it is the first step in deciding how many processors a program can use. The serial parts of an application frequently make up most of its source code while taking a small share of its running time. Such code usually stays on the CPU, which runs one thread quickly, and the parallel programming effort goes into the small amount of code where the time is spent.

Amdahl's formula is also optimistic in one respect: it assumes that the parallel part divides for free. In practice, handing out work and collecting results take time, and that time grows with the number of processors, so measured speedup curves bend over earlier than the formula predicts. We calculate one such cost in section 1.9.

1.3.3 When the parallel part has a speedup of its own

The formula for T_p assumes that p processors divide the parallel part exactly p ways. A GPU program is measured differently. The serial part stays on the CPU, the parallel part moves to the GPU, and the GPU runs that part some number of times faster. We call that number s. It depends on the GPU and on the part, and it is not a count of processors. Putting s in place of p gives the speedup of the whole program:

$$S = \frac{1}{f + \frac{1 - f}{s}}$$

WORKED EXAMPLE: 1 SERIAL SECOND AND 99 PARALLEL SECONDS

A program runs for 100 s on 1 CPU core: 1 s of serial work and 99 s of parallel work, so $f = 0.01$.

1. The GPU runs the parallel part 100 times faster, in 99 / 100 = 0.99 s. The program takes 1 + 0.99 = 1.99 s, a speedup of 100 / 1.99 = **50**.
2. The parallel part is memory-bound, and the GPU runs it only 10 times faster, in 9.9 s. The program takes 1 + 9.9 = 10.9 s, a speedup of **9.2**.
3. The parallel part again runs 100 times faster, but its input must first be copied into the GPU's memory and its output copied back, which takes 2 s. The program takes 1 + 0.99 + 2 = 3.99 s, a speedup of **25**.

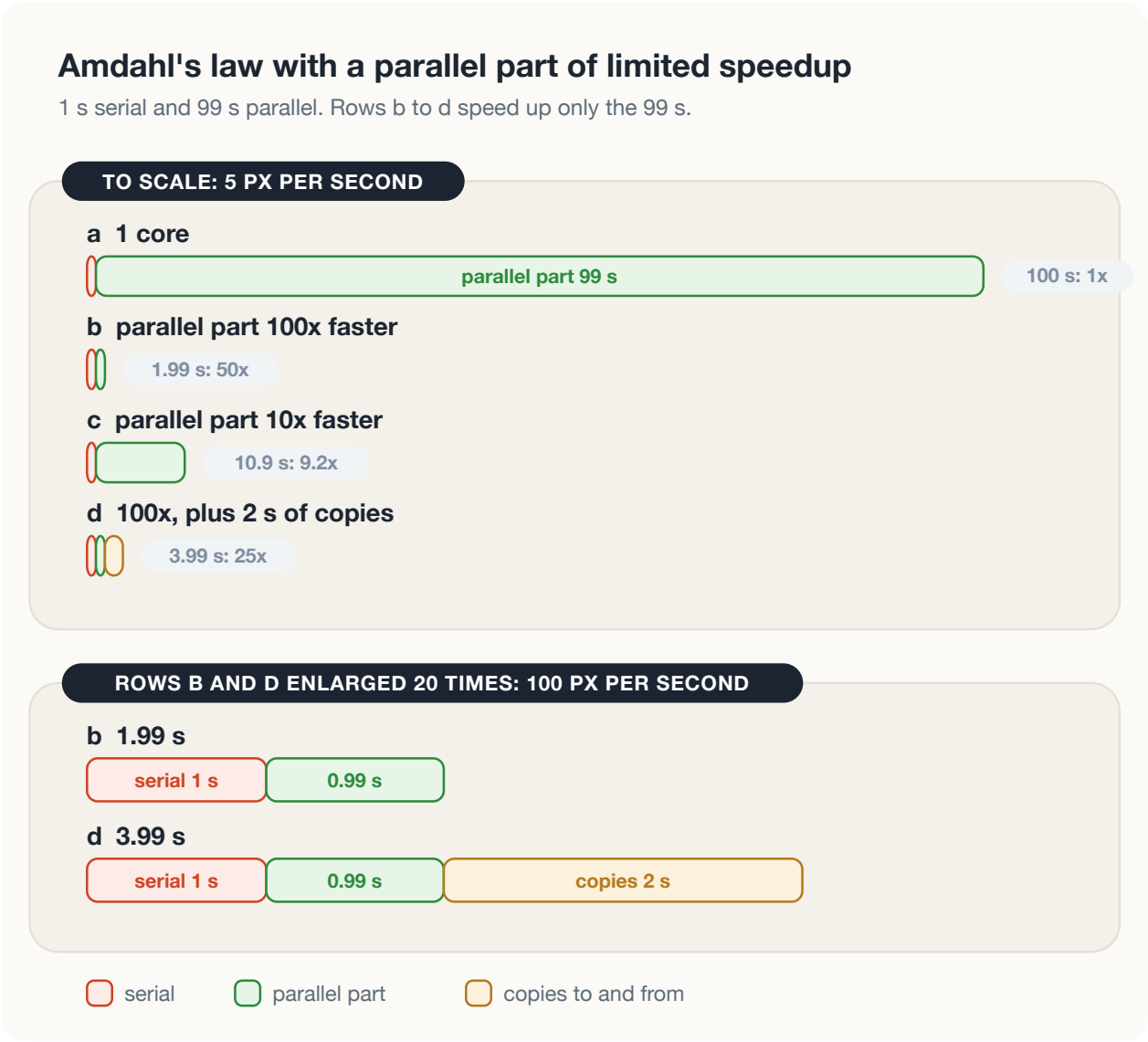


Figure 1.7 The 3 cases of the worked example, to scale and enlarged.

In case 1 the parallel part became 100 times faster and the program only 50 times. The serial second was 1% of the original run and is half of the new one (row b of figure 1.7). The table of section 1.3.1 predicts this: with $f = 0.01$, the speedup reaches half of its limit of 100 when the parallel part shrinks to the size of the serial part, at $s = 99$.

The factor of 10 in case 2 has a specific cause. A memory-bound part runs at the rate at which its memory delivers bytes (section 1.8), so moving it to a GPU makes it faster by about the ratio of the 2 memory bandwidths, whatever the GPU's arithmetic units could do. A desktop CPU with 2 channels of DDR5-5600 memory moves 2×8 bytes $\times\ 5.6 \times 10^9$ per second $= 89.6$ GB/s. A GeForce RTX 4090 moves 1,008 GB/s, about 11 times as much. A straightforward port of a memory-bound program to a GPU therefore often stops near a speedup of 10. Chapters 7 to

9 go beyond it by reusing each byte from on-chip memory once it has arrived, which reduces the number of bytes the part must fetch from DRAM.

The copies of case 3 exist only because the work moved, and they behave like serial time: the GPU's speed does not shorten them. With t for the copy time as a fraction of the original run, here 2 / 100 = 0.02:

$$S = \frac{1}{f + \frac{1 - f}{s} + t} = \frac{1}{0.01 + 0.0099 + 0.02} = 25$$

Symbol	Meaning	In the example
f	serial fraction of the 1-core run	0.01
s	speedup of the parallel part alone	100 in cases 1 and 3, 10 in case 2
t	copy time as a fraction of the 1-core run	0.02 in case 3
S	speedup of the whole program	50, 9.2 and 25

In Chapter 4 we measure what copies cost and show that a GPU sharing the CPU's memory, as on a Jetson board, can skip them. In Chapter 6 we run copies at the same time as computation, which removes most of t from the formula.



A GPU speeds up only the part it runs. The serial part and the copies keep their length, and they set the limit.

▶ CHECK YOUR UNDERSTANDING: **The parallel part of a program is 90% of its 1-core time, and a GPU runs it 20 times faster. What is the speedup of the program, and what is its limit for a GPU of unlimited speed?**

1.4 Gustafson's law

Amdahl's law was derived for a computation of fixed size. In this section we show that the conclusion changes when the size of the computation grows with the

machine, as it usually does in practice. For 2 decades Amdahl's law was read as a verdict against large parallel machines. In 1988 John Gustafson and his colleagues at Sandia National Laboratories ran 3 scientific programs on a machine with 1,024 processors. They measured speedups of 1,021, 1,020 and 1,016, for programs whose serial fractions were 0.4 to 0.8 percent. By Amdahl's formula, a serial fraction of 0.4 percent allows a speedup of about 200 on 1,024 processors. The measured speedups were 5 times higher, and the arithmetic of both results was correct.

The 2 results answer different questions. Amdahl asked how much faster a fixed computation runs on more processors. Gustafson's group did what users of large machines usually do: they enlarged the problem to fill the machine, and asked how much more work could be done in the same time. A larger machine renders a larger image, trains on a larger batch or simulates a finer grid. In such programs the serial part, typically the setup and the input and output, grows slowly or not at all, while the parallel part grows with the data.

WORKED EXAMPLE: A COMPUTATION THAT GROWS WITH THE MACHINE

A program runs for 10 seconds on 4 processors: 1 second in a serial part, then 9 seconds in which all 4 processors compute.

1. On one processor, the serial second still takes 1 second.
2. The 9 parallel seconds kept 4 processors busy, which is $4 \times 9 = 36$ processor-seconds of work. One processor needs 36 seconds for it.
3. So $T_1 = 37$ s and $T_4 = 10$ s, a speedup of **3.7**.

On 8 processors, with twice the data, the run again takes $1 + 9 = 10$ seconds, and one processor would need $1 + 8 \times 9 = 73$. The speedup is **7.3**.

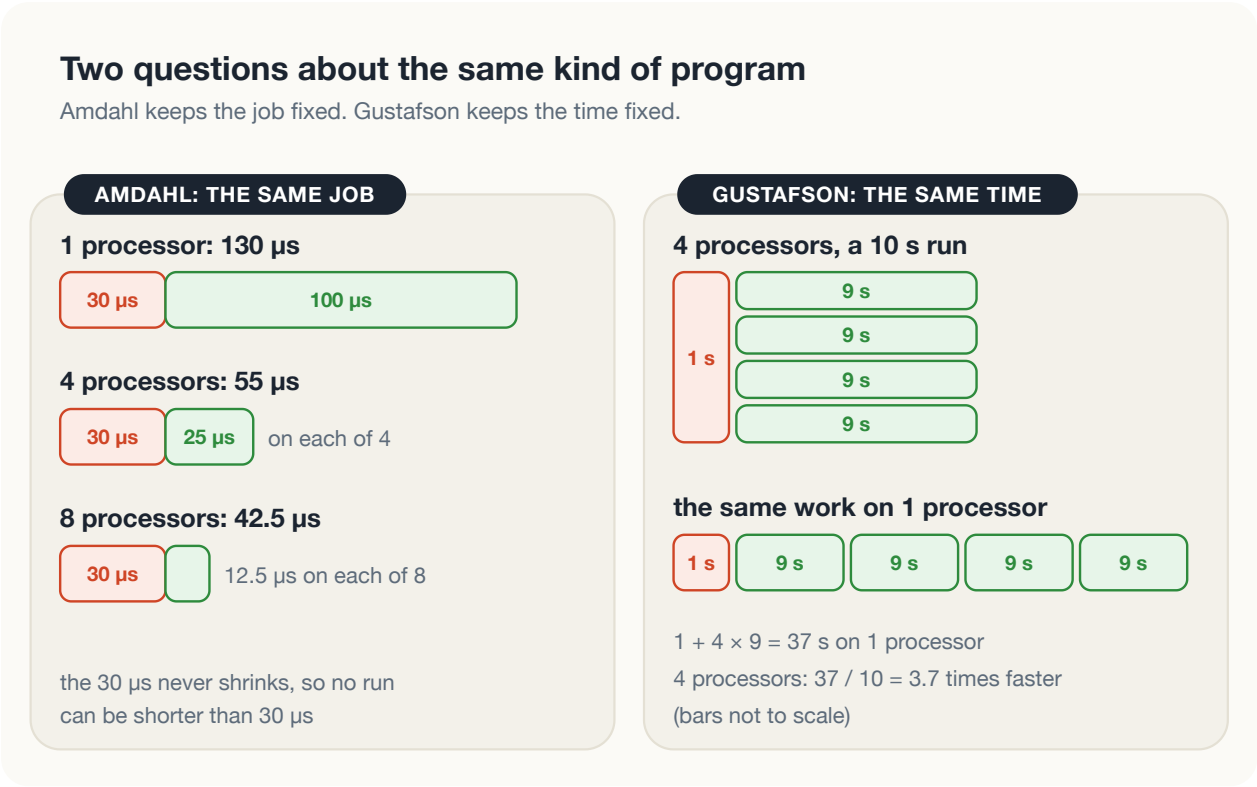


Figure 1.8 Left: a fixed computation on more processors. Right: a computation that grows with the number of processors.

Let s be the serial fraction measured on the parallel run, here 1 second out of 10, so $s = 0.1$. The worked example generalises to

$$S(p) = s + p \cdot (1 - s)$$

which is **Gustafson's law**. For the example, $0.1 + 4 \times 0.9 = 3.7$. For the Sandia programs, with $s = 0.004$ and $p = 1{,}024$, it gives 1,020, the speedup they measured.

The 2 laws are consistent with each other. They measure the serial fraction against different running times, and answer different questions.

	Amdahl	Gustafson
Question	how much faster does the same computation run?	how much more computation fits in the same time?
Serial fraction measured on	the one-processor run, f	the parallel run, s
As processors are added	the computation stays the same	the computation grows
Speedup	approaches 1/f	grows in proportion to p

In the worked example the serial second is a tenth of the parallel run but only a thirty-seventh of the one-processor run. Its share of the one-processor run keeps falling as the problem grows. A video frame of fixed size that must be ready in 16 milliseconds is governed by Amdahl's law; a training batch that grows with the machine is governed by Gustafson's.

1.5 Strong and weak scaling

The 2 laws correspond to the 2 standard ways of measuring a parallel program. **Strong scaling** keeps the total problem fixed and adds processors; the running time should fall, and Amdahl's law predicts where it stops falling. **Weak scaling** keeps the problem *per processor* fixed, so the total grows with the processors; the running time should stay the same, as Gustafson's law describes.

Measurement	Problem size	Good result
strong scaling	fixed	time falls as processors are added
weak scaling	grows with the processors	time stays the same

Weak scaling exposes overheads directly. Suppose that one processor handles 1,000 elements in 1 ms, and that 16 processors are given 16,000 elements, still 1,000 each. A run of 1 ms is perfect scaling. A run of 1.25 ms means that 0.25 ms went to distributing work, exchanging data and waiting, costs that grow with the number of processors.

> **TIP:** When you report a speedup, say whether it comes from strong or weak scaling. The 2 have different limits, so a speedup without that information cannot be checked.



1.6 How a computation divides

Both laws assume that the parallel part can be divided, but they say nothing about how. The way in which a computation divides decides which machines can run it well, and 3 patterns cover most programs.

Task parallelism runs different computations at the same time: one processor decodes the next video frame while another filters the current one. Programs rarely have more than a handful of separate tasks, so task parallelism occupies a few processors at most. In Chapter 6 we use it to overlap data transfers with computation.

Data parallelism runs the same computation on different parts of the data at the same time. Vector addition is data parallel: every row runs the same instructions, and each processor takes some of the rows. The number of parts grows with the data, which is what a machine with thousands of processors needs. Data parallelism is also easier to reason about than task parallelism, because every processor executes the same code, and a programmer can understand the whole computation by understanding one part of it. GPUs are built for data parallelism, and we use it in most of this book.

Pipeline parallelism splits the work on each item into stages, such as load, compute and store, and lets different items occupy different stages at the same time, like an assembly line. Figure 1.9 shows 4 items passing through 3 stages.

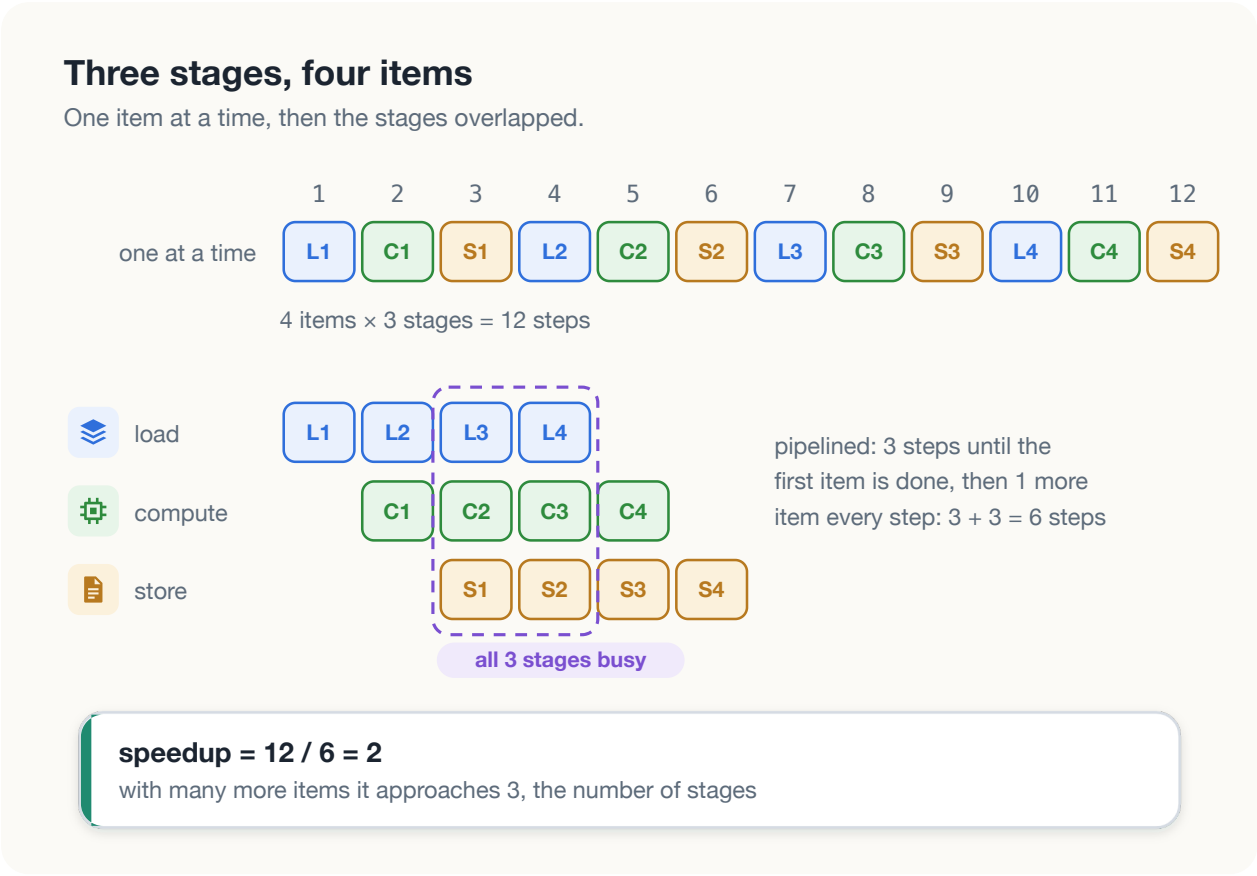


Figure 1.9 4 items through 3 stages, first one item at a time, then pipelined.

One item at a time, each item needs 3 steps and the 4 need 12. In the pipeline, the first item still needs 3 steps to pass through every stage. From then on one item leaves at every step, so the remaining 3 items need only 3 more: 6 steps in all, a speedup of 2. In general N items through S stages take $S \cdot N$ steps one at a time and $S + N - 1$ in a pipeline, a speedup of

$$\frac{S \cdot N}{S + N - 1}$$

For figure 1.9, $3 \times 4/6 = 2$. As N grows the speedup approaches S, the number of stages. A short batch spends most of its time filling and emptying the pipeline, which is why pipelines pay off on long streams. The same arithmetic reappears in Chapter 6, where the stages are a copy to the GPU, a kernel and a copy back.

1.7 Flynn's taxonomy

The patterns of section 1.6 describe programs. In this section we classify the machines that run them, by the way in which their processors receive instructions. In 1966 Michael Flynn sorted computers by 2 counts: the number of **instruction streams** a machine follows at once, and the number of **data streams** those instructions work on. With one or many of each there are 4 classes. Figure 1.10 shows one computation, adding 8 pairs of numbers, on each kind of machine.

- **SISD**, single instruction stream, single data stream: an ordinary core. Each add instruction adds one pair, so 8 pairs take 8 instructions in sequence.
- **SIMD**, single instruction stream, multiple data streams: a core with *vector registers* that hold several numbers side by side. One add instruction adds all 8 pairs. AVX on x86 processors and NEON on Arm processors are SIMD instruction sets.
- **MIMD**, multiple instruction streams, multiple data streams: several cores, each running its own program on its own data. Every multi-core CPU is a MIMD machine.
- **MISD**, multiple instruction streams, single data stream, is rare. Some fault-tolerant systems run the same data through different programs and compare the results.

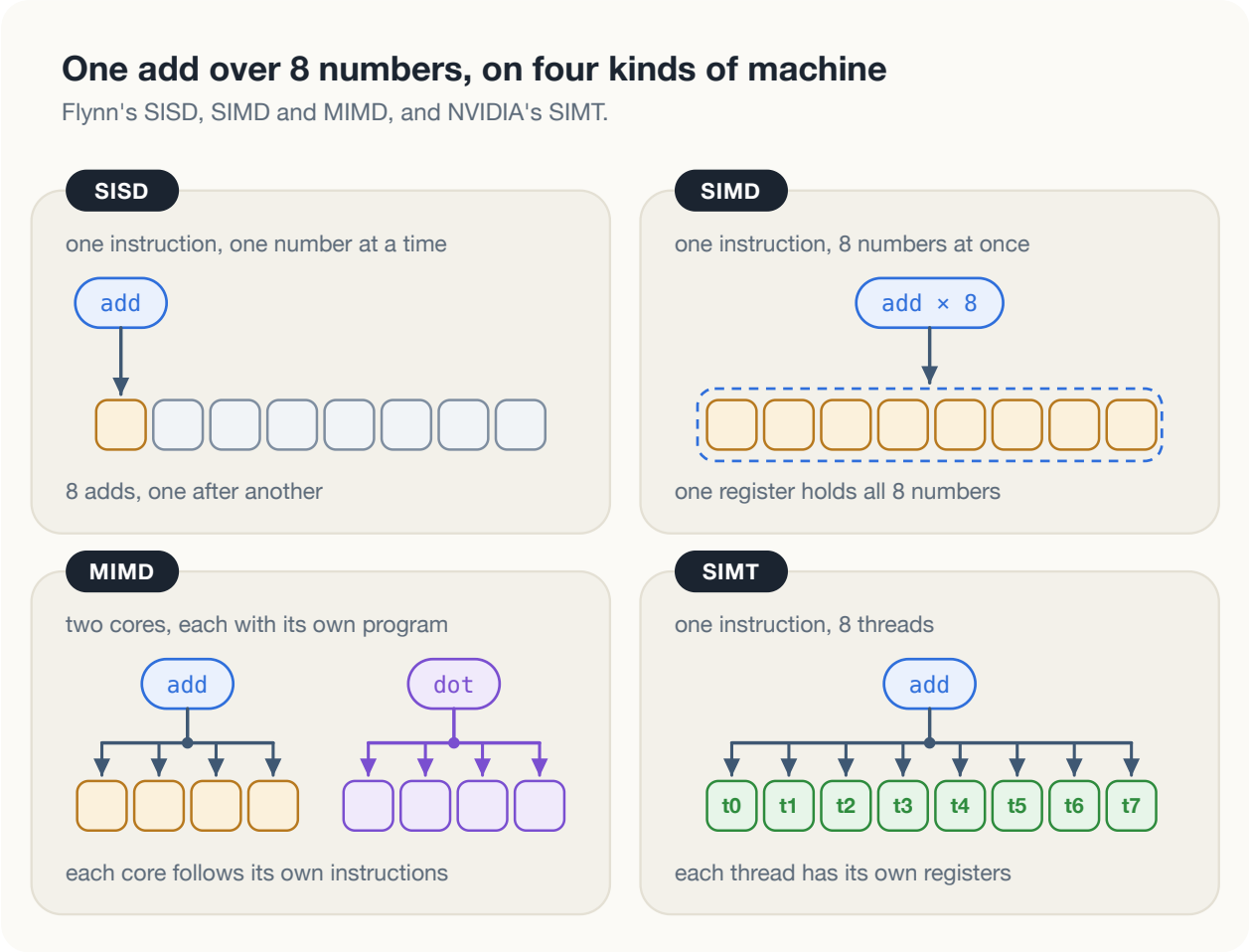


Figure 1.10 One add over 8 numbers on SISD, SIMD and MIMD machines, and on NVIDIA's SIMT design.

A GPU fits none of the 4 exactly. NVIDIA calls its design **SIMT**, for single instruction, multiple threads. As in SIMD, one instruction is issued to a whole group of threads, so the cost of fetching and decoding the instruction is shared among them; this sharing is what lets a GPU devote most of its area to arithmetic units. Unlike SIMD, every thread has its own registers, so a programmer writes an ordinary sequential program for one thread and the hardware runs it for many. A SIMD programmer must arrange the data into vector registers by hand; a SIMT programmer does not.

The difference becomes visible at a branch. When some threads of a group take the `if` side and others the `else` side, the group cannot do both at once. It runs one side while the threads of the other side wait, and then runs the other side. The program remains correct, but the group spends time on both sides. In Chapter 2 we show how the hardware forms these groups and what such a split costs.

1.8 Arithmetic intensity and the roofline

The previous sections asked how many processors a program can use. In this section we ask a different question: given a machine, which of its resources limits a computation. An NVIDIA H100 can perform 67 trillion additions of 32-bit numbers per second. Given 2 long arrays to add, element by element, it performs about 0.28 trillion, less than half of one percent of its peak. The GPU is working correctly: its arithmetic units spend almost all of their time waiting for numbers to arrive from memory, as the processor of section 1.1 did. Both observations follow from one ratio, which compares the arithmetic a computation performs with the bytes it moves.

1.8.1 Arithmetic intensity

A computation on arrays does 2 kinds of work: it moves bytes between main memory and the processor, and it performs arithmetic on them. One addition or multiplication of 2 floating-point numbers is one **FLOP**, for floating-point operation. The **arithmetic intensity** of a computation is the number of FLOPs it performs for each byte it moves:

$$I = \frac{\text{FLOPs performed}}{\text{bytes moved}}$$

WORKED EXAMPLE: 2 INTENSITIES COUNTED BY HAND

Vector addition, one row. The processor reads `a[i]` and `b[i]`, 4 bytes each, and writes `c[i]`, 4 bytes: 12 bytes moved for 1 addition. $I = 1/12 \approx 0.08$ FLOP per byte.

A 2 × 2 matrix product $C = A \times B$. Each entry of C is a row of A multiplied by a column of B:

```
C[0][0] = A[0][0] × B[0][0] + A[0][1] × B[1][0]
```

Each of the 4 entries takes 2 multiplications and 2 additions, counting the addition to the initial zero: 16 FLOPs. The 3 matrices hold 12 numbers of 4 bytes, 48 bytes. $I = 16/48 = 1/3$ FLOP per byte.

The matrix product already does 4 times as much arithmetic per byte, and the ratio grows with the size of the matrices, because each number takes part in many products. For 2 $N \times N$ matrices the same count gives $2N^3$ FLOPs on $3N^2$ numbers of 4 bytes:

$$I = \frac{2N^3}{3N^2 \times 4} = \frac{N}{6} \text{ FLOP per byte}$$

For $N = 2$ this is the 1/3 we counted. For $N = 4{,}096$ it is about 680 (figure 1.11). The number of times a byte is used after it arrives is its **reuse**. Vector addition uses every byte once, and a large matrix product uses each one thousands of times. The count above assumes that each number is fetched from memory only once, which a program achieves only if it keeps the numbers close to the processor between uses; we examine whether it does in Chapter 9.

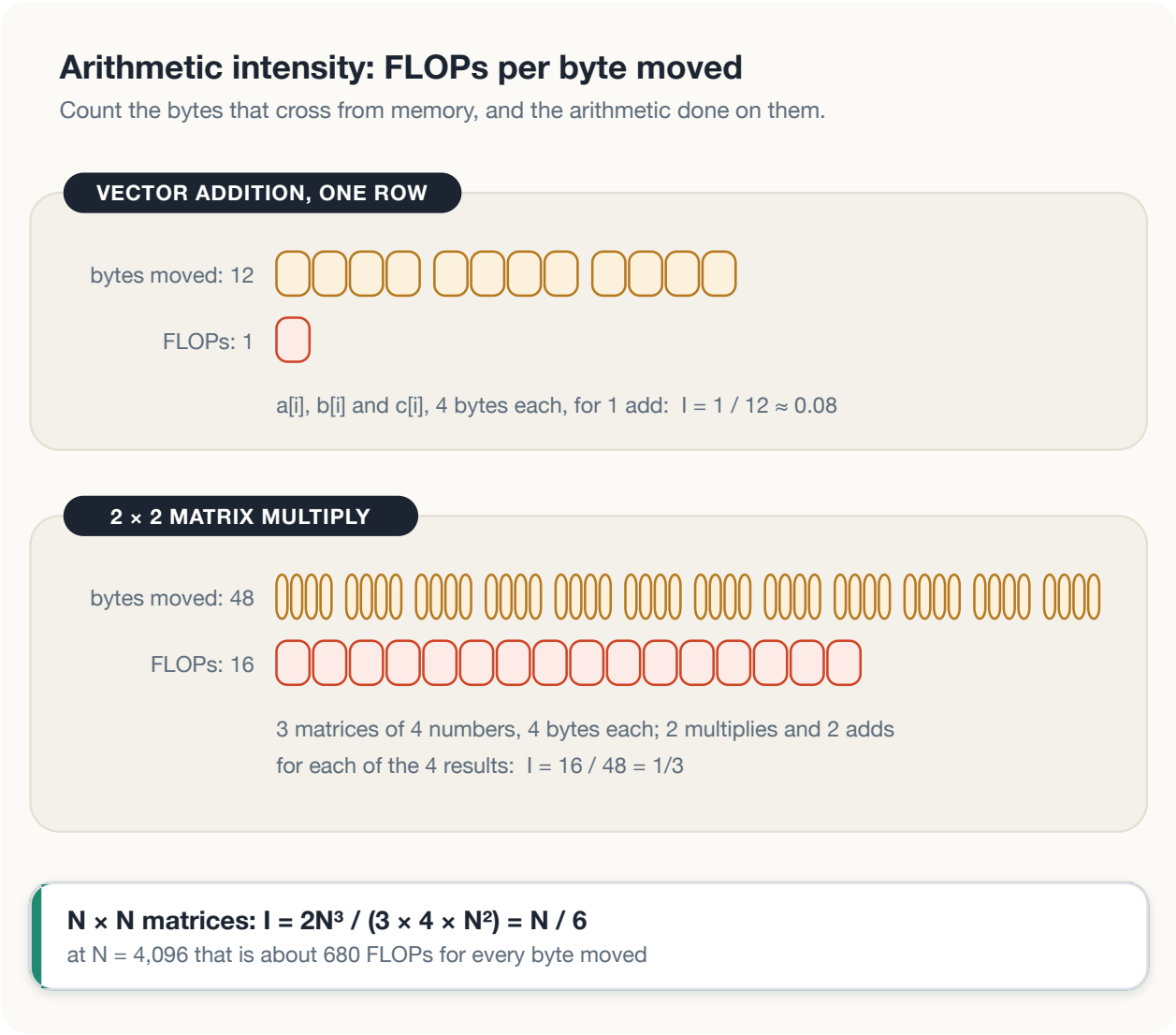


Figure 1.11 The bytes moved and the FLOPs performed for one row of vector addition and for a 2 × 2 matrix product.

1.8.2 The ridge point

A machine has 2 speed limits. Its arithmetic units perform at most P_{peak} FLOPs per second, and its memory delivers at most B bytes per second, a figure called the **memory bandwidth**. A computation with intensity I performs I FLOPs per byte, so memory alone lets it reach $I \times B$ FLOPs per second. It runs at whichever limit is lower:

$$P = \min(P_{peak}, I \times B)$$

WORKED EXAMPLE: 3 COMPUTATIONS ON A MACHINE WITH 40 TFLOP/S AND 1 TB/S

Computation	I (FLOP/byte)	Memory allows I × B	Arithmetic allows	Runs at
vector addition	0.08	0.08 TFLOP/s	40 TFLOP/s	**0.08 TFLOP/s**
matrix product, N = 64	10.7	10.7 TFLOP/s	40 TFLOP/s	**10.7 TFLOP/s**
matrix product, N = 4,096	683	683 TFLOP/s	40 TFLOP/s	**40 TFLOP/s**

For the first 2 computations memory sets the speed. Such a computation is **memory-bound**, and faster arithmetic units would not help it. For the third, arithmetic sets the speed; it is **compute-bound**, and more memory bandwidth would not help. The 2 limits are equal at the intensity

$$I_{ridge} = \frac{P_{peak}}{B} = \frac{40}{1} = 40 \text{ FLOP per byte}$$

called the **ridge point** of the machine. The ridge point has a direct reading: it is the number of FLOPs the machine can perform in the time its memory delivers 1 byte. A computation that performs fewer FLOPs than that per byte leaves the arithmetic units idle part of the time. Drawn on logarithmic axes, the limit P rises along a slope and then runs flat, like a roof. Samuel Williams, Andrew Waterman and David Patterson introduced the picture in 2009 and named it the **roofline** (figure 1.12).

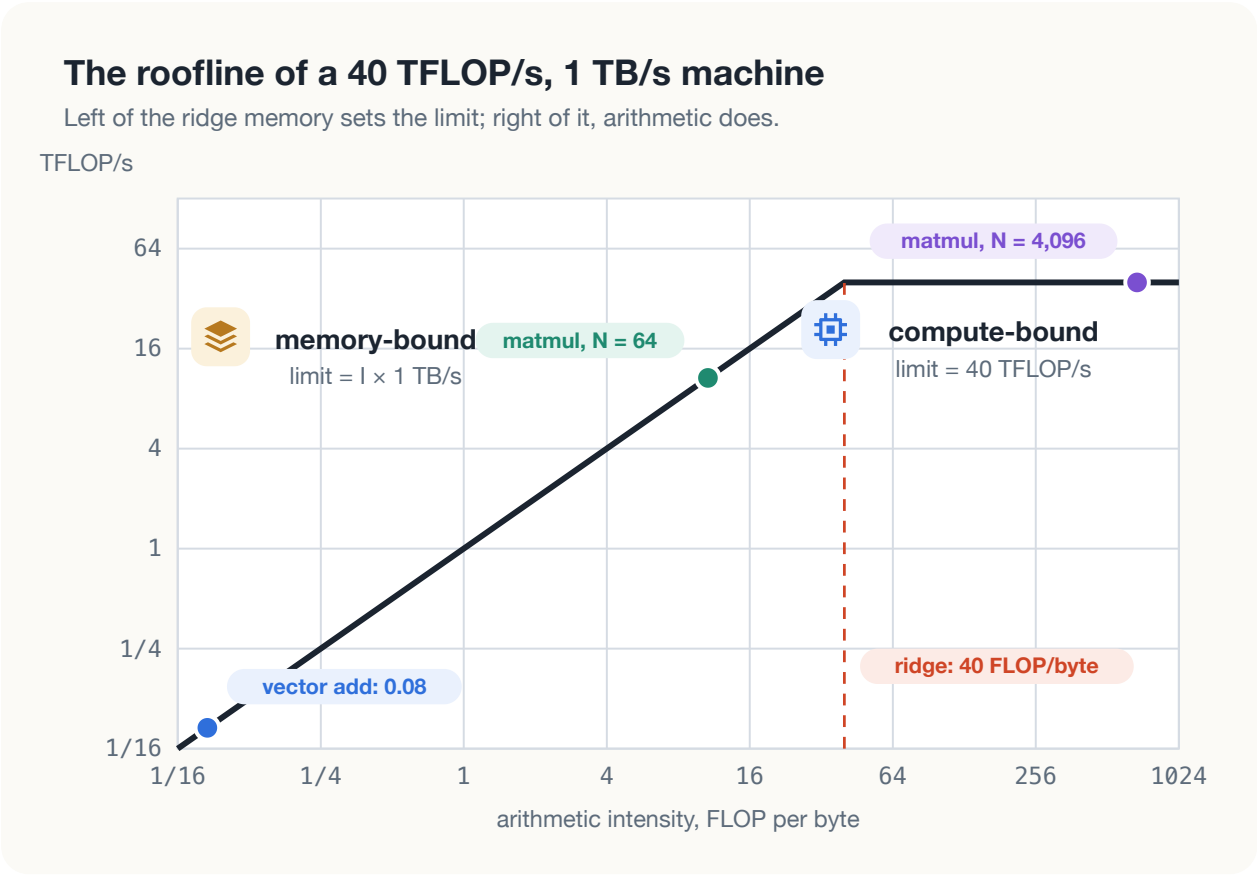


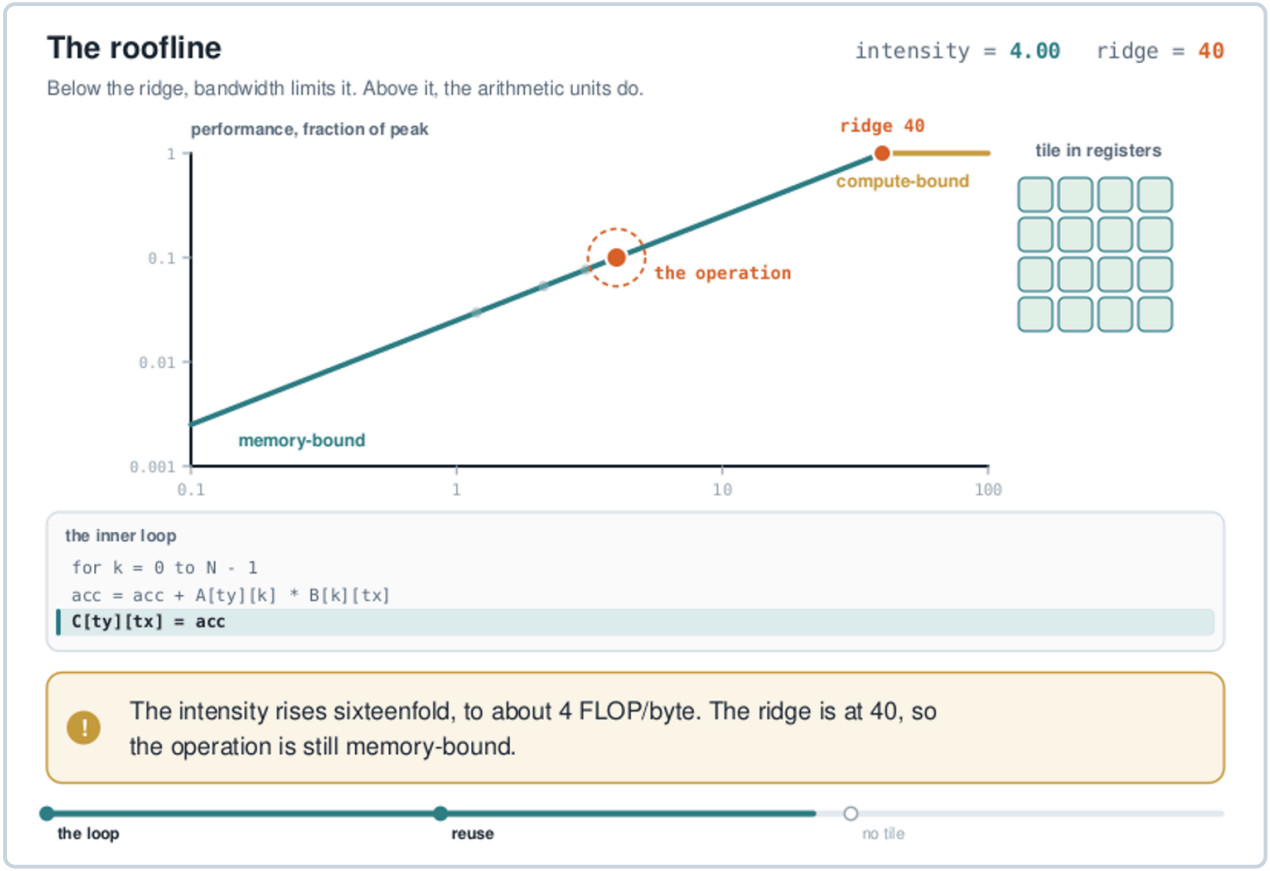
Figure 1.12 The roofline of a machine with 40 TFLOP/s and 1 TB/s, with the 3 computations of the worked example.

The H100 delivers 3.35 TB/s from its memory, so its ridge is at $67 / 3.35 = 20$ FLOP per byte. Vector addition, at 1/12, sits far down the slope, and $\frac{1}{12} \times 3.35 = 0.28$ TFLOP/s is the speed quoted at the start of this section. In section 2.1.5 we repeat the calculation for 7 current GPUs.

The roofline is an upper bound, and a program can sit well below it. The most common reason is the one of section 1.1. The memory delivers B bytes per second only if enough reads are in flight to satisfy Little's law; a program that offers too few independent reads reaches a fraction of B, and therefore a fraction of its roof. A measured point far below the roof means that the program has too few reads in flight, or moves more bytes than the count of section 1.8.1 assumed.

▶ CHECK YOUR UNDERSTANDING: **A machine performs 20 TFLOP/s and its memory delivers 0.5 TB/s. Where is its ridge point, and is a computation with an intensity of 10 FLOP per byte memory-bound on it?**

A computation moves to the right on the roofline when each byte it loads is used more often. In the inner loop of a straightforward matrix product, each step reads 2 numbers, 8 bytes, for 2 FLOPs, an intensity of 0.25. If a 16 × 16 block of the result is kept in registers, every number read is used 16 times and the intensity rises to about 4. The technique is called **tiling**, and we build it step by step in Chapter 9 (animation 3). Vector addition offers nothing to reuse, so it can only become faster by moving its bytes more efficiently, which we treat in section 2.7 and Chapter 7.



Animation 3 The inner loop of a matrix product. Reusing a tile of values raises its intensity; removing the tile lowers it again.

1.8.3 CPU-bound, memory-bound, and I/O-bound programs

The same words describe whole programs once a third resource is added: **input and output** (I/O), such as a disk or a network. Whichever resource stays busy while the others wait gives the program its name. A **CPU-bound** program, also called compute-bound, keeps the processor busy. A **memory-bound** program keeps the memory system busy while the processor waits for bytes. An **I/O-bound** program keeps a disk or network link busy while the processor waits for data.

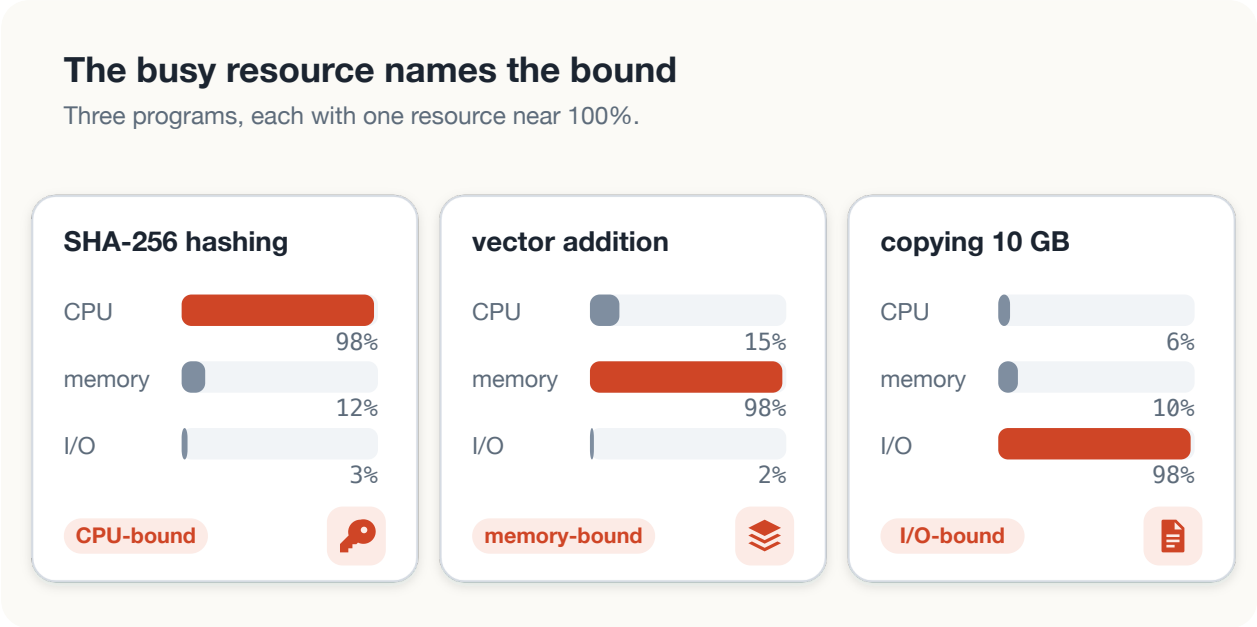


Figure 1.13 3 programs, each with one resource near full use.

An experiment confirms the bound: make one resource twice as fast and measure again.

Resource made twice as fast	Running time halves?	The program is
processor	yes	CPU-bound
memory bandwidth	yes	memory-bound
disk or network	yes	I/O-bound
number of processors	no	limited by its serial part (section 1.3)

A video encoder that spends 98% of its time computing and 2% reading frames is CPU-bound: a faster disk changes nothing, and a processor twice as fast nearly halves its running time. The bound of a program is the first thing to establish before optimising it, because an optimisation aimed at a resource that is not the bound cannot help. We treat memory-bound GPU programs in Chapter 7, compute-bound ones in Chapter 9 and programs that wait for data transfers in Chapter 6. In Chapter 16 we show how a profiler measures the bound instead of guessing it.

1.9 The cost of synchronization

The laws of sections 1.3 and 1.4 treat the parallel part as if each processor worked alone from start to finish. Real parallel computations must also coordinate. Many of them end by combining the results of their processors, for example by adding partial sums, and the results cannot be combined until every processor has finished. The usual mechanism is a **barrier**: a point in the program at which each processor stops until all the others have reached it. We show how barriers are built in Chapter 5; here we calculate what they cost.

If 4 processors each perform 1,000 cycles of work and then spend 100 cycles at a barrier, each works for 1,000 of every 1,100 cycles, an efficiency of 0.91. Work is rarely divided that evenly.

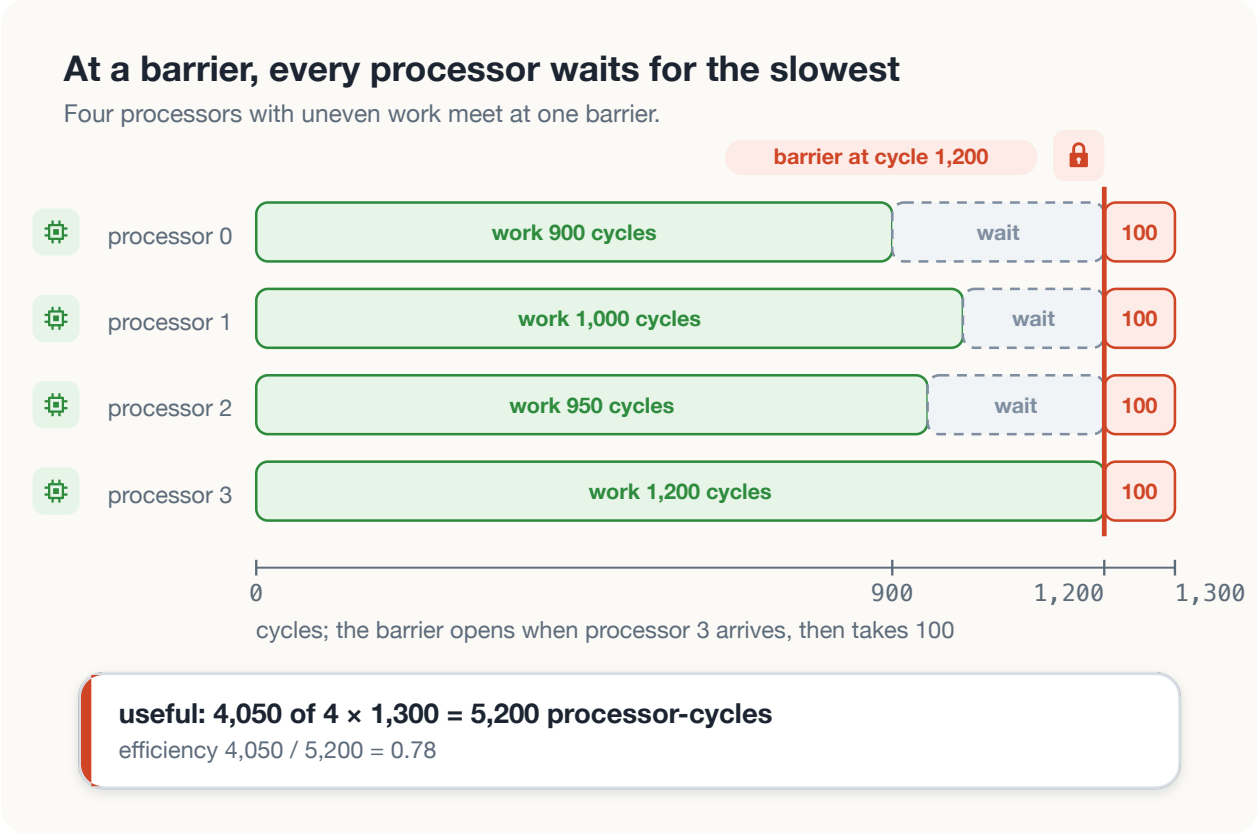


Figure 1.14 4 processors with uneven work meet at a barrier.

WORKED EXAMPLE: UNEVEN WORK AT A BARRIER

1. The processors have 900, 1,000, 950 and 1,200 cycles of work.
2. The barrier opens at cycle 1,200, when processor 3 arrives; the other 3 stand idle until then.
3. Useful work: 900 + 1,000 + 950 + 1,200 = 4,050 processor-cycles.
4. Time paid for: 4 processors × (1,200 + 100) cycles = 5,200 processor-cycles.
5. Efficiency: 4,050 / 5,200 = **0.78**.

A barrier makes every processor finish at the pace of the slowest. The amount of work each processor receives often depends on the input data, for example on the number of non-zero entries in its part of a sparse matrix, so the same program can scale well on one input and poorly on another. A barrier also has a cost that the figure does not show. A value written by one processor does not become visible to the others instantly. A barrier has to wait until all such writes are visible, which is part of why it takes 100 cycles and not 1.

The idle time and the barrier itself behave like serial time, since adding processors shortens neither. Under strong scaling each processor's share of the work shrinks while the barrier stays the same, so the barrier takes a growing share of each run and Amdahl's limit falls.



Fast parallel programs meet as rarely as they can: each processor writes its own partial result, and the results are combined once, at the end.

In Chapter 8 we apply this to the sum of N numbers and reduce the number of barriers from about $\log_2 N$ to a small constant.

1.10 Work, steps and work efficiency

The laws of this chapter compare a parallel program with a sequential one as if both performed the same operations. That assumption often fails. Many parallel

algorithms perform more operations than the best sequential algorithm, and for large inputs the extra operations can make them slower than the program they replace. In this section we measure a parallel algorithm by 2 counts that expose the problem, and derive bounds on its running time from them.

Section 1.1.5 showed that the operations of a dependency chain cannot overlap. The 2 counts turn that observation into numbers. The **work** W of an algorithm is the number of operations it performs. Its **steps** D, also called its depth or span, is the number of operations in its longest dependency chain. We use the same 2 counts for sums in Chapter 8. In this section we count time in units of 1 operation, and assume that every operation takes 1 unit.

WORKED EXAMPLE: 3 COMPUTATIONS ON THE ENTRIES OF C

The running example produced $c = 0, 3, 6, 9$.

1. Vector addition itself: 4 additions, and none needs the result of another. $W = 4$, $D = 1$.
2. The sum of c, from left to right: $0 + 3 = 3$, then $3 + 6 = 9$, then $9 + 9 = 18$. Each addition needs the one before. $W = 3$, $D = 3$.
3. The same sum in pairs: $0 + 3 = 3$ and $6 + 9 = 15$ are independent, then $3 + 15 = 18$. $W = 3$, $D = 2$.

On p processors, at most p operations complete in each unit of time, so the run takes at least W/p units. It also takes at least D units, because the operations of the longest chain run one after another however many processors are free. Both limits hold at once:

$$T_p \geq \max\left(\frac{W}{p},\ D\right)$$

In 1974 Richard Brent showed that the operations can always be scheduled so that the run is not much longer than this bound:

$$T_p \leq \frac{W}{p} + D$$

For the sum in pairs on 2 processors, the bounds give $\max(1.5,\ 2) = 2$ and $1.5 + 2 = 3.5$ units. The run takes 2: both first additions in the first unit, the last

addition in the second.

The ratio W/D is the **parallelism** of the algorithm: the average number of operations available per unit of time. Up to about W/D processors, the W/p term dominates and each added processor shortens the run. Beyond it, D dominates and added processors wait. Vector addition of N entries has parallelism $N/1 = N$. A chain has parallelism 1, which is why exercise 8 finds no gain from more processors. A sum in pairs of $2^{20} = 1{,}048{,}576$ values has $W = 1{,}048{,}575$ and $D = 20$, a parallelism of about 52,000.

Symbol	Meaning	In the example
W	work: operations performed	3 for the sum of c
D	steps: operations on the longest chain	3 left to right, 2 in pairs
p	processors	2
W/D	parallelism	1 left to right, 1.5 in pairs

A parallel algorithm is **work-efficient** when its work is within a constant factor of the work of the best sequential algorithm for the same task. Many parallel algorithms buy fewer steps with more work, and the bounds show what that costs.

A **scan**, or running sum, replaces each entry by the sum of all entries up to it. The scan of $c = 0, 3, 6, 9$ is 0, 3, 9, 18. A scan appears to be inherently sequential, since each output needs the one before it, yet it can be computed in parallel by algorithms that perform extra additions. A sequential scan of N values performs $N - 1$ additions, each needing the one before: $W = D = N - 1$. 2 parallel scans appear in Chapter 8.

- The **shifted scan** (section 8.7.2) runs rounds at offsets 1, 2, 4, up to $N/2$. In the round at offset d, every entry from position d on adds the entry d places to its left. On c, offset 1 gives 0, 3, 9, 15 with 3 additions, and offset 2 gives 0, 3, 9, 18 with 2 more: $W = 5$, $D = 2$. In general $W = N \log_2 N - (N - 1)$ and $D = \log_2 N$.
- The **tree scan**, due to Guy Blelloch, adds pairs up a tree and then passes the partial sums back down it. It has $W = 2(N - 1)$ and $D = 2 \log_2 N$.

WORKED EXAMPLE: 3 SCANS OF 1,024 VALUES

With $N = 1{,}024 = 2^{10}$, the shifted scan performs $1{,}024 \times 10 - 1{,}023 = 9{,}217$ additions, 9 times the sequential 1,023. The table gives the lower bound $\max(W/p, D)$ in units of 1 addition.

Scan	W	D	p = 4	p = 32	p = 1,024
sequential	1,023	1,023	1,023	1,023	1,023
tree	2,046	20	511.5	63.9	20
shifted	9,217	10	2,304	288	10

On 4 processors the shifted scan needs at least 2,304 units, more than twice the sequential scan. Its extra work needs more than 9,217 / 1,023 = 9.01 processors before it can even break even, and the ratio grows with N: for a scan of 2^{20} values it is about 19. The tree scan breaks even at 2 processors whatever the size. On 1,024 processors the order reverses: each scan has processors to spare for its work, the steps decide, and the shifted scan's 10 beat the tree scan's 20 (figure 1.15).

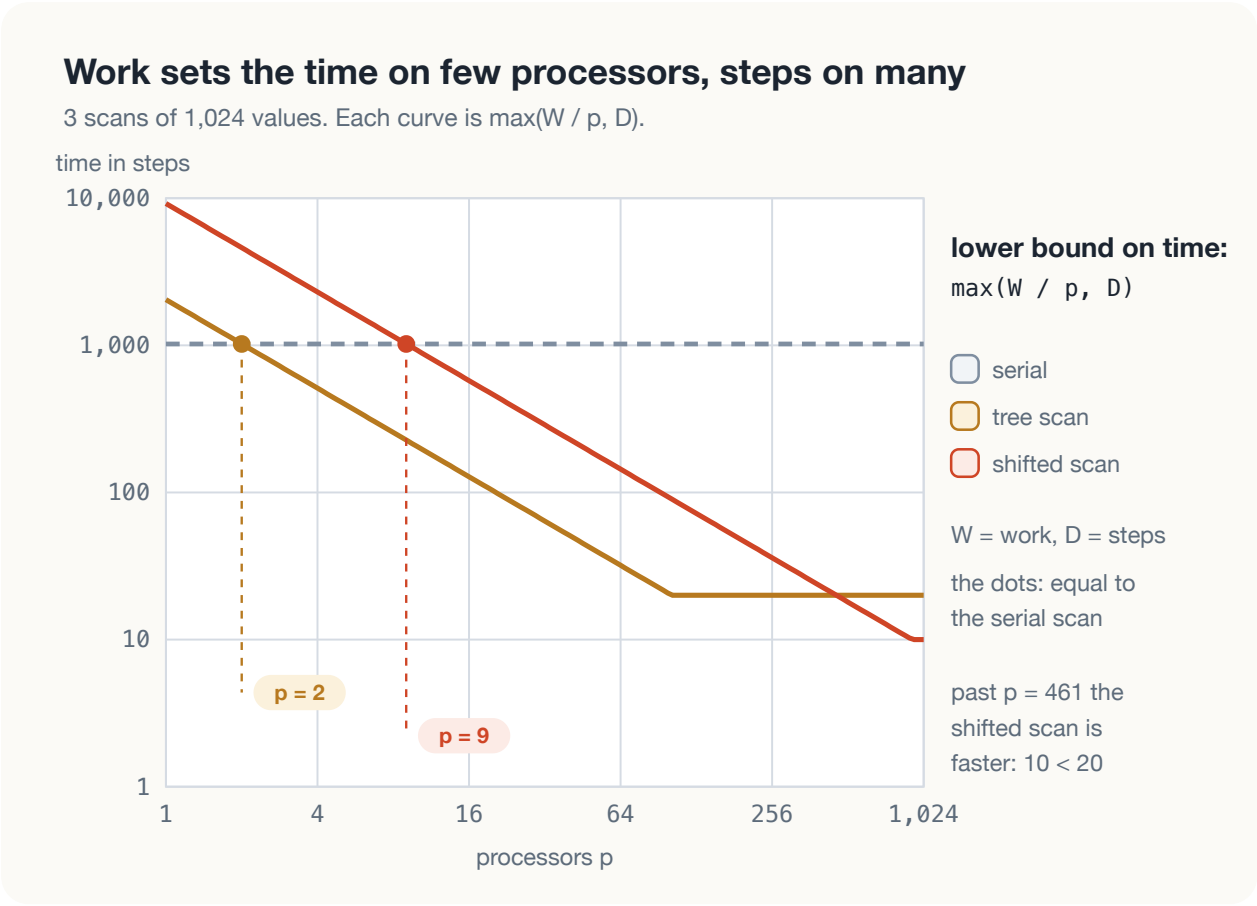


Figure 1.15 The lower bound $\max(W/p, D)$ for 3 scans of 1,024 values.

Neither scan is better in general; the choice depends on how many processors the scan actually has. A warp of 32 GPU threads executes every step of the shifted scan together, whether a thread adds or not, so its extra work costs no time there, and for that reason we use the shifted scan inside a warp in section 8.7.2. Across a whole block, each step usually ends at a barrier. D then counts barriers too, and the price of each is calculated in section 1.9.



Work decides the running time on few processors, and steps decide it on many. Count both before choosing a parallel algorithm.

▶ CHECK YOUR UNDERSTANDING: **An algorithm has $W = 10^6$ and $D = 10^3$. Between which times does it run on 100 processors, and on 10,000?**

Summary 

- **Latency** is the time one operation takes; **throughput** is the number of operations completed per unit of time. **Little's law**, $C = X \times L$, gives the concurrency needed to reach a throughput when the latency is fixed.
- Operations in a **dependency chain** cannot overlap. Independent operations can, and they supply the concurrency that hides memory latency. A CPU is designed to shorten latency, a GPU to hide it.
- **Speedup** is T_1/T_p and **efficiency** is S/p. Measure every speedup against the best sequential program.
- **Amdahl's law**: a fixed computation with serial fraction f runs at most $1/f$ times faster on any number of processors.
- A GPU runs the parallel part s times faster, and copies take a fraction t of the original run. The program's speedup is $1/(f + (1 - f)/s + t)$.
- **Gustafson's law**: a computation that grows with the machine has a speedup of $s + p(1 - s)$.
- **Strong scaling** fixes the total problem; **weak scaling** fixes the problem per processor.
- Computations divide by task, by data, or into pipeline stages. GPUs are built for data parallelism.
- **Flynn's taxonomy** classifies machines as SISD, SIMD, MISD or MIMD. A GPU uses **SIMT**: one instruction issued to a group of threads that each have their own registers.
- **Arithmetic intensity**, FLOPs per byte, compared with the machine's **ridge point**, shows whether memory or arithmetic limits a computation. The roofline is reached only with enough reads in flight.
- A **barrier** costs the wait for the slowest processor plus the barrier itself.
- **Work** W counts operations and **steps** D the longest chain; $\max(W/p, D) \le T_p \le W/p + D$. A **work-efficient** parallel algorithm does no more than a constant factor of the sequential work.

In Chapter 2 we describe the organisation of the GPU: how its processors are arranged, how it keeps hundreds of memory reads in flight, and where the ridge points of current GPUs lie.

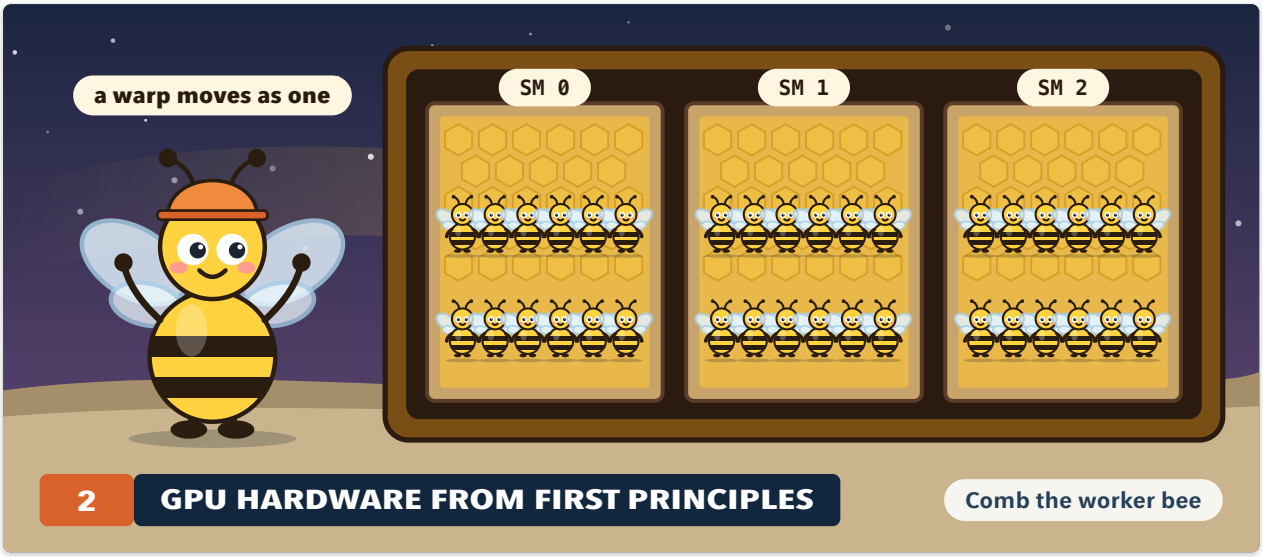
Exercises

1. A memory read has a latency of 600 cycles, and the memory accepts a new read every 3 cycles. How many reads must be in flight to reach its highest throughput?
2. A program takes 10.0 s on one processor, 5.5 s on 2 and 4.0 s on 4. Compute the speedup and efficiency for each number of processors.
3. A program spends 10 μs reading its input, 100 μs computing and 10 μs writing its output, and only the computing can be shared. Compute the serial fraction and Amdahl's limit.
4. The same program grows its problem with the number of processors, so that the serial fraction of the parallel run falls to 0.001. Estimate the speedup on 1,000 processors.
5. Using Amdahl's formula, what speedup would the Sandia programs of section 1.4, with a serial fraction of 0.8 percent, reach on 1,024 processors? What does Gustafson's law give?
6. An $N \times N$ matrix product has an intensity of $N/6$ FLOP per byte. For which N does it reach a ridge point of 40 FLOP per byte?
7. A program downloads 10 GB over a network at 2 GB/s and compresses it on the CPU at 20 GB/s. Estimate the fraction of the time the CPU is busy and name the bound. Which change shortens the run more: a CPU twice as fast, or a network twice as fast?
8. Explain why a program whose operations form one long dependency chain gains nothing from more processors.
9. Name the class of each machine: one ordinary core; one vector instruction that adds 8 pairs; 4 cores, each running its own program; one instruction issued to many threads. Which of these is not one of Flynn's classes?
10. In figure 1.14, the work of processor 3 falls from 1,200 to 1,000 cycles. Compute the efficiency.
11. A program spends 5% of its 1-core time in a serial part, and a GPU runs the rest 40 times faster. Compute the speedup with no copies, and with copies that take 3% of the 1-core time.
12. Count W and D for the shifted scan and the tree scan of 8 values. On how many processors does each first match the sequential scan's 7 units?
13. A report compares a GPU kernel with the same parallel code run on 1 CPU thread. Explain why the speedup it quotes is larger than the speedup over the best sequential program.

14. A memory system has a bandwidth of 1 TB/s and a read latency of 500 ns. Each read returns 32 bytes. How many reads must be in flight for a program to reach the bandwidth, and what fraction of it does a program reach with 1,000 reads in flight?

Sources and further reading

- Gene M. Amdahl, “Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities,” AFIPS Conference Proceedings 30, 1967.
- John L. Gustafson, “Reevaluating Amdahl’s Law,” Communications of the ACM 31(5), 1988.
- John L. Gustafson, Gary R. Montry and Robert E. Benner, “Development of Parallel Methods for a 1024-Processor Hypercube,” SIAM Journal on Scientific and Statistical Computing 9(4), 1988.
- John D. C. Little, “A Proof for the Queuing Formula: L = λW,” Operations Research 9(3), 1961.
- Michael J. Flynn, “Very High-Speed Computing Systems,” Proceedings of the IEEE 54(12), 1966.
- Samuel Williams, Andrew Waterman and David Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” Communications of the ACM 52(4), 2009.
- NVIDIA, *H100 Tensor Core GPU* datasheet, for the 67 TFLOP/s and 3.35 TB/s of the SXM part.
- NVIDIA, *GeForce RTX 4090* specifications, for the 1,008 GB/s of section 1.3.3.
- Richard P. Brent, “The Parallel Evaluation of General Arithmetic Expressions,” Journal of the ACM 21(2), 1974.
- Guy E. Blelloch, “Programming Parallel Algorithms,” Communications of the ACM 39(3), 1996, for work and depth as a way to compare parallel algorithms.



Chapter 2: GPU Hardware from First Principles

In Chapter 1 we saw that a processor that reads its operands from DRAM waits hundreds of cycles for each of them, and that the wait can be hidden only by keeping many independent operations in flight. A *graphics processing unit*, or GPU, is designed around this observation. It executes the same instructions on many data elements at once, and it holds enough threads that its arithmetic units remain busy while many of those threads wait for memory.

In this book we program NVIDIA GPUs with **CUDA**, the platform for general-purpose GPU computing that NVIDIA introduced in 2007. Before CUDA, a general computation could run on a GPU only if it was expressed through a graphics interface, for example as an operation that colours the pixels of an image. CUDA C++ extends C++ with a small amount of syntax for marking and launching functions that run on the GPU; its compiler, `nvcc`, generates GPU code from those functions; and its runtime library lets code on the CPU allocate GPU memory, transfer data and start work on the GPU. We present each program both in CUDA C++ and in Rust, using NVIDIA's cuda-oxide compiler and libraries. In Chapter 3 we write the first of these programs.

In this chapter we describe the GPU hardware that a CUDA programmer must understand in order to reason about the performance of a GPU program. We begin

with the terms in which work is handed to a GPU and with the published characteristics of 7 current GPUs. We then examine the streaming multiprocessor, the warp, and the assignment of thread blocks to streaming multiprocessors. The remainder of the chapter covers the memory hierarchy, the access patterns that determine whether memory delivers its full bandwidth, and occupancy, which determines how well a streaming multiprocessor hides the latency of memory. Because these limits differ between generations of GPUs, every number in the chapter is given for a named GPU.

THIS CHAPTER COVERS

- kernels, threads, blocks and grids: how work is handed to a GPU
- the parts of a GPU, from the whole chip down to a single thread
- the streaming multiprocessor (SM) and its schedulers
- the warp, a group of 32 threads that carry out each instruction together, and what happens when its threads take different sides of a branch or loop
- how blocks are assigned to SMs in waves, and how a block is divided into warps
- the memory hierarchy, from registers to the CPU's memory, and where each kind of variable lives
- coalescing: why neighbouring threads should read neighbouring addresses
- shared-memory banks and bank conflicts
- occupancy, the resources that limit it, and how switching between warps hides memory latency

2.1 The GPU at a glance

2.1.1 How a CPU and a GPU deal with memory latency

A CPU and a GPU read from the same kind of DRAM and wait the same hundreds of cycles for each read. They differ in what they do during that wait, and the difference determines how each divides its chip area (figure 2.1).

A CPU core runs 1 sequence of instructions as fast as possible, and spends much of its area on avoiding waits. Large **caches**, small fast memories near a core that keep copies of recently used data, answer most reads without a trip to main memory. Prediction logic starts the likely next instructions early, so that a branch rarely stalls the core. A desktop CPU has 4 to 32 such cores. This approach is called **latency-oriented design**: it shortens the time that each thread waits.

A GPU accepts that a read from main memory takes hundreds of cycles. It spends its area on thousands of simple arithmetic units and on room to hold tens of thousands of threads at once. A **thread**, as in Chapter 1, is 1 sequence of instructions with its own working values. While some threads wait for memory, others compute. This approach is called **throughput-oriented design**: each thread may wait a long time, but the GPU as a whole completes more operations per second (section 1.1.3).

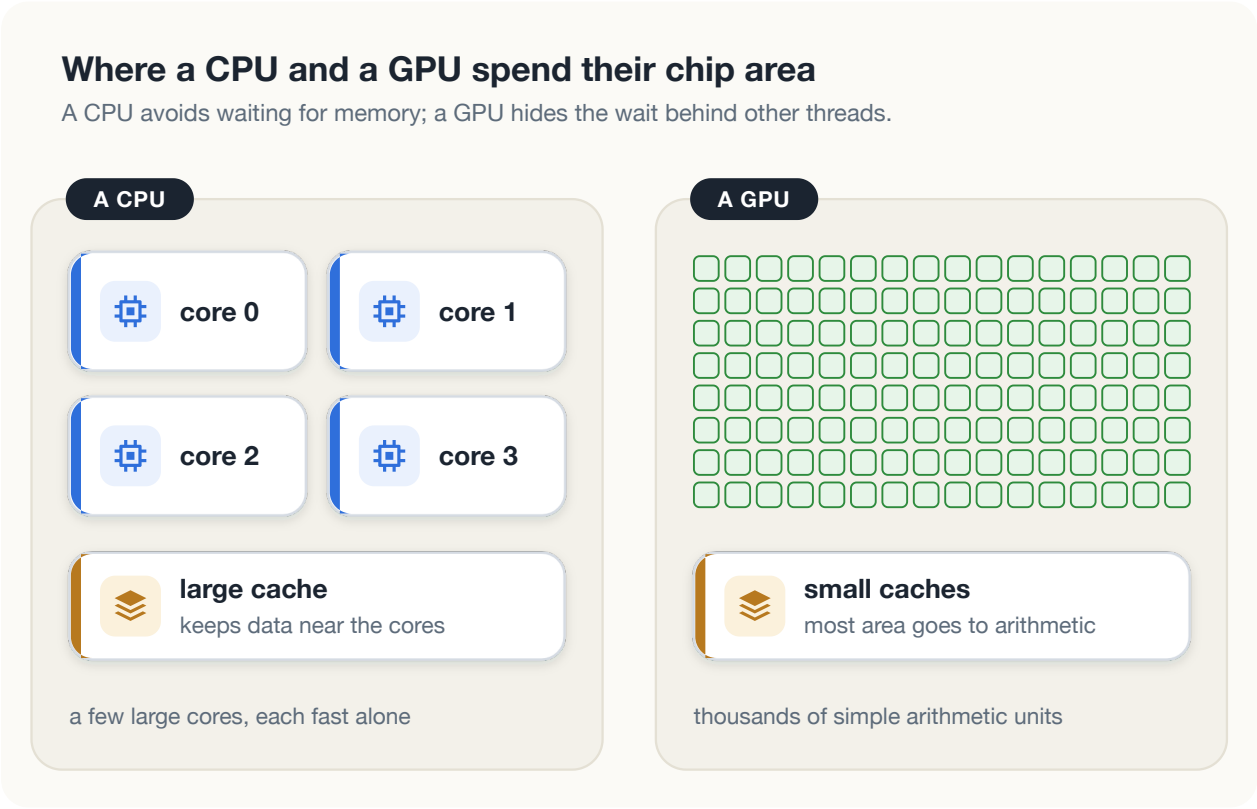


Figure 2.1 A CPU spends its chip on a few fast cores and large caches; a GPU on many simple arithmetic units.

2.1.2 Kernels, threads, blocks and grids

A program hands work to a GPU as a kernel, launched over a grid of threads (figure 2.2).

- A **kernel** is a function that runs on the GPU.
- A **launch** starts a kernel from a program on the CPU. The GPU then runs many copies of the kernel at once.
- Each copy is 1 **thread**. A thread keeps its working values in **registers**, small storage slots inside a processor that each hold 1 value. Each thread also receives its own number, its **index**, and uses it to select its share of the work.
- Threads are grouped into **blocks** of equal size. The threads of 1 block run on the same part of the GPU, so they can share fast memory and wait for each other.
- All blocks of 1 launch form the **grid**.

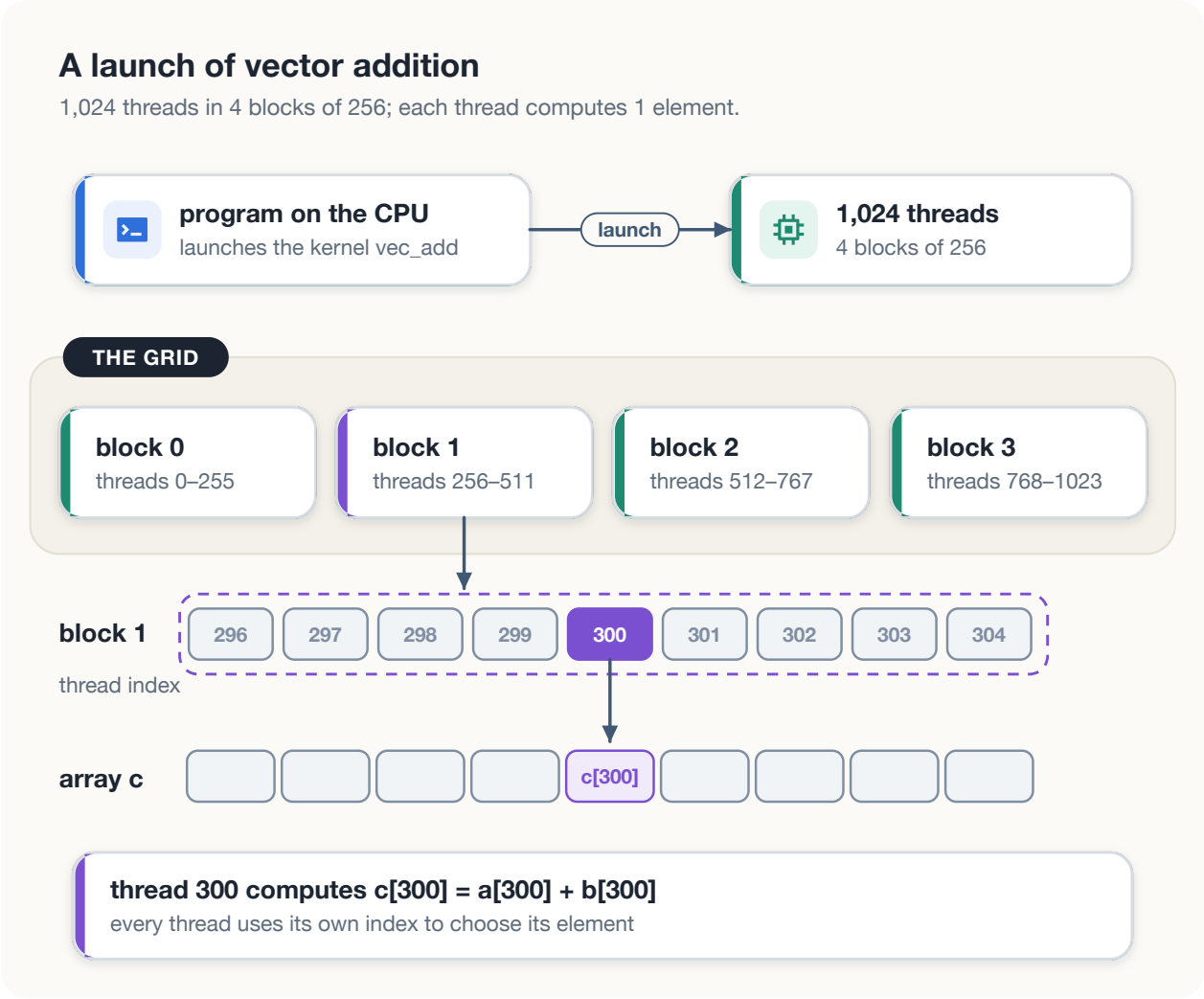


Figure 2.2 A launch of vector addition: 1,024 threads in 4 blocks of 256. Thread 300 lies in block 1 and computes `c[300]`.

For vector addition with N = 1,024, every element needs 1 thread, so the launch needs 1,024 threads. A block holds 256 of them, so the grid needs 1,024 ÷ 256 = 4 blocks.

A thread finds its element through its index. Inside its block, each thread has a **position**, numbered from 0. Block 0 has positions 0 to 255, and so does block 1, so a position alone cannot tell a thread which element is its own. The blocks before a thread's block hold 256 threads each, and its index counts them first:

$$\text{index} = \text{block number} \times \text{block size} + \text{position}$$

Thread 300 is at position 44 of block 1, and $1 \times 256 + 44 = 300$ (figure 2.3). In the other direction, 300 ÷ 256 = 1 with remainder 44 gives back its block and its

position. Each thread computes `c[index]`, so the 1,024 threads cover the 1,024 elements once each, and no 2 threads write the same element.

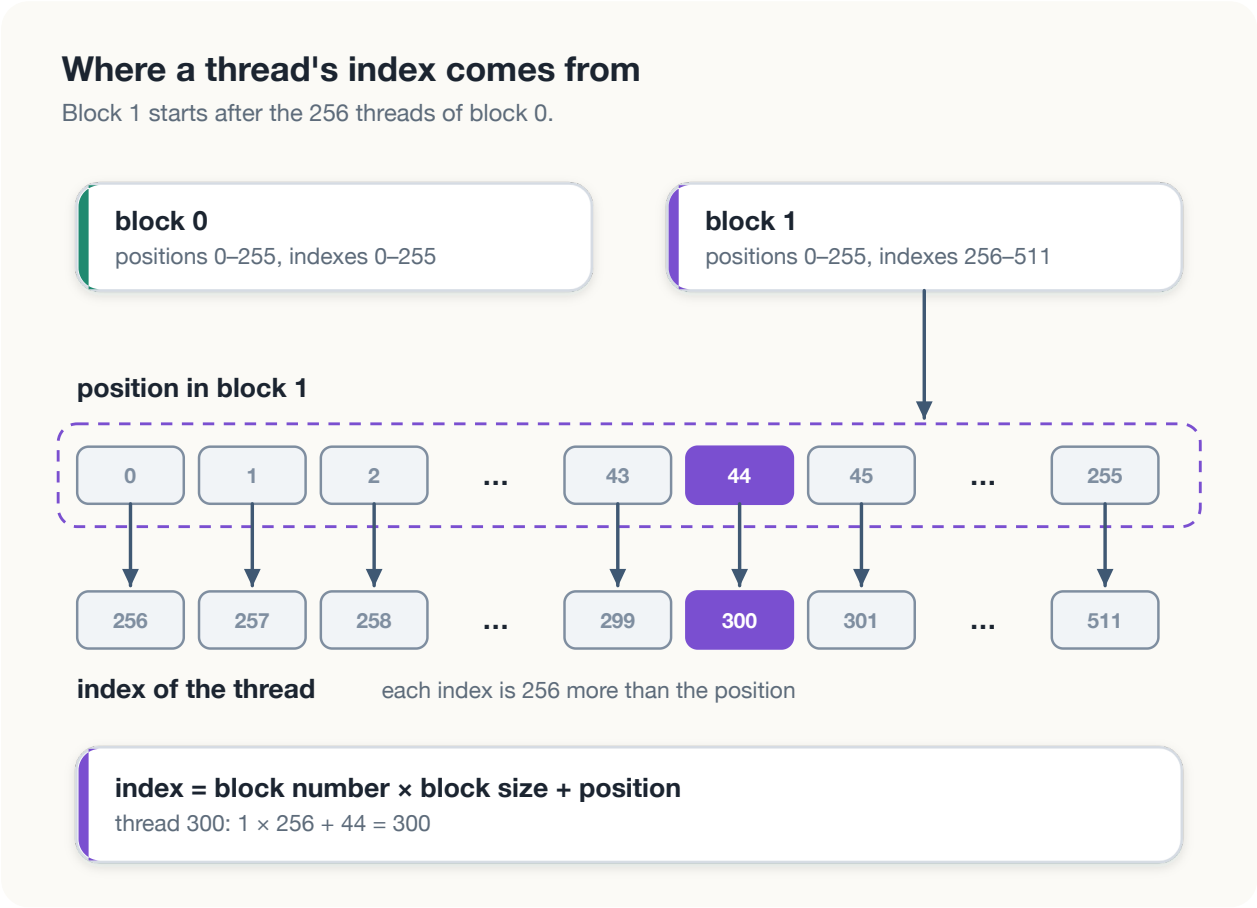


Figure 2.3 A thread's index is its block's starting index plus its position in the block.

In Chapter 3 we show how a kernel reads its block number and position in code, and how blocks and grids can also be laid out in 2 or 3 dimensions, which suits images and volumes.

2.1.3 From the chip to one thread

A GPU chip is organised in levels, each built from copies of the level below it (figure 2.4).

- **Streaming multiprocessor (SM).** The processor of a GPU that executes threads. An SM contains arithmetic units, the circuits that fetch and decode instructions for them, a register file that holds the working values of its threads, and a fast on-chip memory. A GPU chip consists mostly of identical

SMs: 132 on an H100, 8 on a Jetson Orin NX. Each block of a launch runs on 1 SM from start to finish.

- **Arithmetic unit.** A circuit that performs 1 operation on 32-bit floating-point numbers per cycle, for 1 thread. NVIDIA calls these units *CUDA cores*. An H100 SM has 128 of them.
- **Warp.** A group of 32 threads of the same block that execute together. The SM fetches and decodes each instruction of a warp once, and 32 arithmetic units execute it at the same time, 1 for each thread, each on that thread's own values (section 2.3). The position of a thread in its warp, 0 to 31, is its **lane**.
- **Warp scheduler.** The circuit that chooses, in every cycle, which of the warps on its part of the SM executes its next instruction. An H100 SM has 4 warp schedulers, each serving 32 arithmetic units and up to 16 warps.
- **Registers.** The storage locations that hold a thread's working values. Each register belongs to 1 thread. An H100 SM has 65,536 registers of 4 bytes, divided among all its threads.

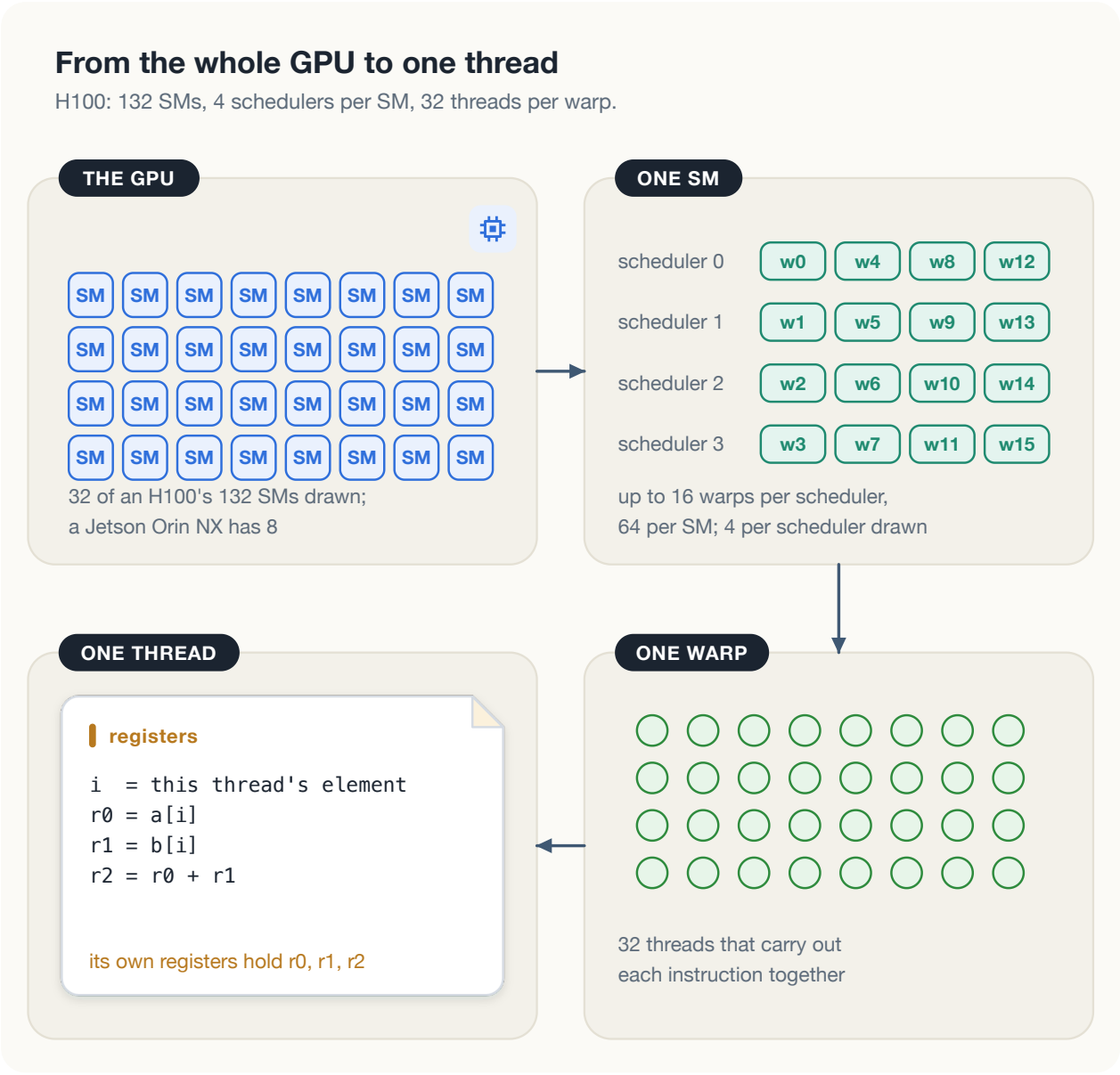


Figure 2.4 From the whole GPU down to 1 thread, for an H100.

2.1.4 GPUs in numbers

GPUs are built in sizes from modules for robots to cards for data centres, with the same design. The table compares 7 of them. The **compute capability**, CC in the table, is NVIDIA's version number for each generation of the design (section 2.11).

GPU	Kind	CC	SMs	Memory	Bandwidth	FP32 peak
Jetson Orin NX 16 GB	embedded	8.7	8	16 GB, shared	102 GB/s	2.4 TFLOP/s
Jetson AGX Thor	embedded	11.0	20	128 GB, shared	273 GB/s	8.1 TFLOP/s
DGX Spark (GB10)	desktop AI	12.1	48	128 GB, shared	273 GB/s	~31 TFLOP/s
GeForce RTX 4090	desktop card	8.9	128	24 GB	1,008 GB/s	82.6 TFLOP/s
GeForce RTX 5090	desktop card	12.0	170	32 GB	1,792 GB/s	104.8 TFLOP/s
A100 SXM 80 GB	data centre	8.0	108	80 GB	2,039 GB/s	19.5 TFLOP/s
H100 SXM	data centre	9.0	132	80 GB	3,350 GB/s	67 TFLOP/s

CC is the compute capability, and “shared” marks memory that the GPU shares with the CPU. The figures are NVIDIA’s published specifications, with peak arithmetic at the highest clock rate. The SM counts of the Thor and the DGX Spark follow from their published numbers of arithmetic units, 2,560 and 6,144, at 128 units per SM. The Orin NX figures were also read from a board with `cudaGetDeviceProperties`, a CUDA function described in section 2.9.4.

The memory column shows 2 kinds of system. The Jetson modules and the DGX Spark are **integrated**: the CPU and the GPU share one set of memory chips. The graphics cards and data-centre GPUs are **discrete**: the GPU has memory chips of its own, beside it on the card, and data moves to it from the CPU’s memory over a link (Chapter 4).

The bandwidth column follows from 2 numbers of each memory system: the width of its **memory bus**, the number of bits that move between the GPU and its memory chips in 1 transfer, and the number of transfers per second. Their product, in bytes per second, is the bandwidth (figure 2.5).

GPU	Memory	Bus width	Transfers per second	Bandwidth
Jetson Orin NX	LPDDR5	128 bits = 16 bytes	6.4 billion	16 × 6.4 = 102 GB/s
Jetson AGX Thor	LPDDR5X	256 bits = 32 bytes	8.53 billion	32 × 8.53 = 273 GB/s
GeForce RTX 4090	GDDR6X	384 bits = 48 bytes	21 billion	48 × 21 = 1,008 GB/s
H100 SXM	HBM3	5,120 bits = 640 bytes	5.23 billion	640 × 5.23 = 3,350 GB/s

The H100's rate is the published bandwidth divided by its bus width, 3,350 GB/s ÷ 640 bytes. Its memory transfers less often than the RTX 4090's, 5.23 billion times per second against 21 billion, yet delivers 3.3 times as much data, because its bus is 13 times as wide. A bus of 5,120 wires cannot run across a circuit board to separate memory chips: the wires would need too much board area and too much power to drive. **High-bandwidth memory**, HBM, stacks DRAM chips on top of each other and places the stacks in the same package as the GPU chip, on a silicon base that carries thousands of short connections. LPDDR5 and GDDR6X chips sit on the circuit board, so their buses are narrower, and they make up part of the difference with a higher rate on each wire.

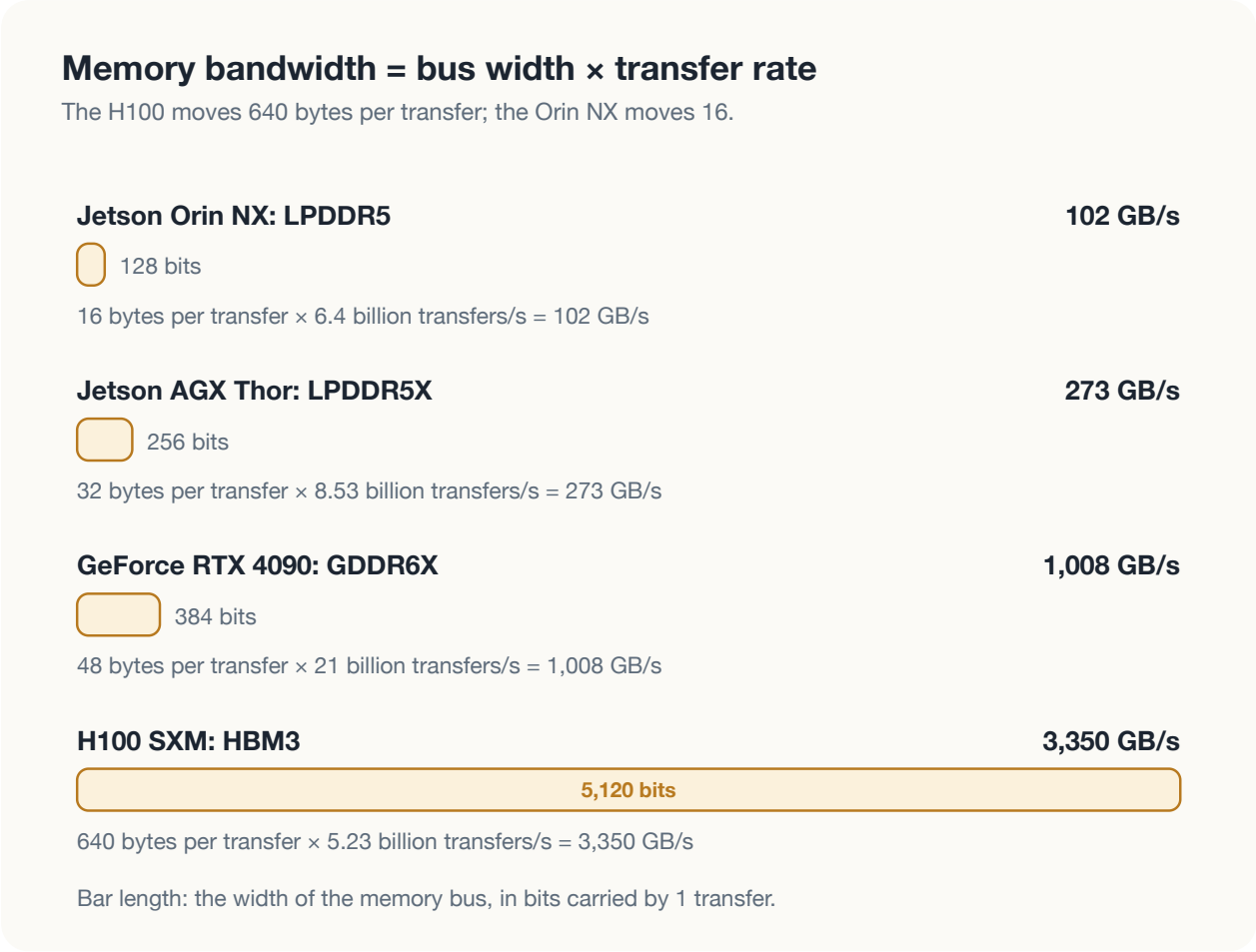


Figure 2.5 Bandwidth is bus width × transfer rate. The H100's HBM3 bus is 5,120 bits wide, 40 times the Orin NX's.

2.1.5 The ridge point of each GPU

In Chapter 1 we defined the **ridge point** of a machine as its peak arithmetic divided by its memory bandwidth: the arithmetic intensity at which a computation stops being limited by memory. For the H100,

$$\frac{67 \times 10^{12}\ \text{FLOP/s}}{3.35 \times 10^{12}\ \text{B/s}} = 20\ \text{FLOP per byte}$$

Dividing FLOPs per second by bytes per second leaves FLOPs per byte. In the time an H100's memory delivers 1 byte, its arithmetic units can perform 20 FLOPs. A kernel that performs fewer than 20 FLOPs on each byte it reads leaves those units waiting for data. For the other GPUs of the table, the same division gives values from 10 to 114.

GPU	Ridge point (FLOP per byte)
A100 SXM 80 GB	19.5 / 2.039 ≈ 10
H100 SXM	67 / 3.35 = 20
Jetson Orin NX 16 GB	2.4 / 0.102 ≈ 24
Jetson AGX Thor	8.1 / 0.273 ≈ 30
GeForce RTX 5090	104.8 / 1.792 ≈ 58
GeForce RTX 4090	82.6 / 1.008 ≈ 82
DGX Spark (GB10)	31 / 0.273 ≈ 114

The data-centre GPUs have the lowest ridge points, because their memory, stacked beside the GPU chip, delivers several terabytes per second. The graphics cards and the DGX Spark have far more arithmetic for each byte of bandwidth, so a kernel needs more reuse to reach their peak. A kernel that is compute-bound on an A100 can therefore be memory-bound on an RTX 4090, with no change to its code.



Vector addition, at 1/12 of a FLOP per byte, is memory-bound on every GPU in the table.

For a memory-bound kernel, the bandwidth also gives a lower bound on the running time, which the measured time can be compared against. The comparison is often called a **speed-of-light** check: the bytes a kernel must move, divided by the bandwidth, is the time it would take if memory never paused.

WORKED EXAMPLE: THE SPEED OF LIGHT FOR VECTOR ADDITION ON A JETSON ORIN NX

1. The arrays hold $N = 2^{26} = 67{,}108{,}864$ floats each. The kernel reads `a` and `b` and writes `c`: 3×4 bytes $\times\ 67{,}108{,}864 = 805{,}306{,}368$ bytes.
2. At 102 GB/s, moving them takes at least $805{,}306{,}368 / (102 \times 10^9)$ s = 7.9 ms.
3. A run measured at 10.0 ms reaches 7.9 / 10.0 = 79% of the speed of light. No change to the code can make it more than 10.0 / 7.9 = 1.27 times faster.

A memory-bound kernel at 90% of its speed of light has little left to gain from tuning, and further speed must come from moving fewer bytes. In Chapter 7 we measure about 68 GB/s for well-ordered reads on the Orin NX, 67% of its rated bandwidth.

2.2 The streaming multiprocessor

Figure 2.6 shows the organisation of 1 SM of an H100. Its 128 arithmetic units form 4 processing blocks of 32. Each processing block has its own warp scheduler and a quarter of the register file, and the 4 processing blocks share 256 KB of on-chip memory.

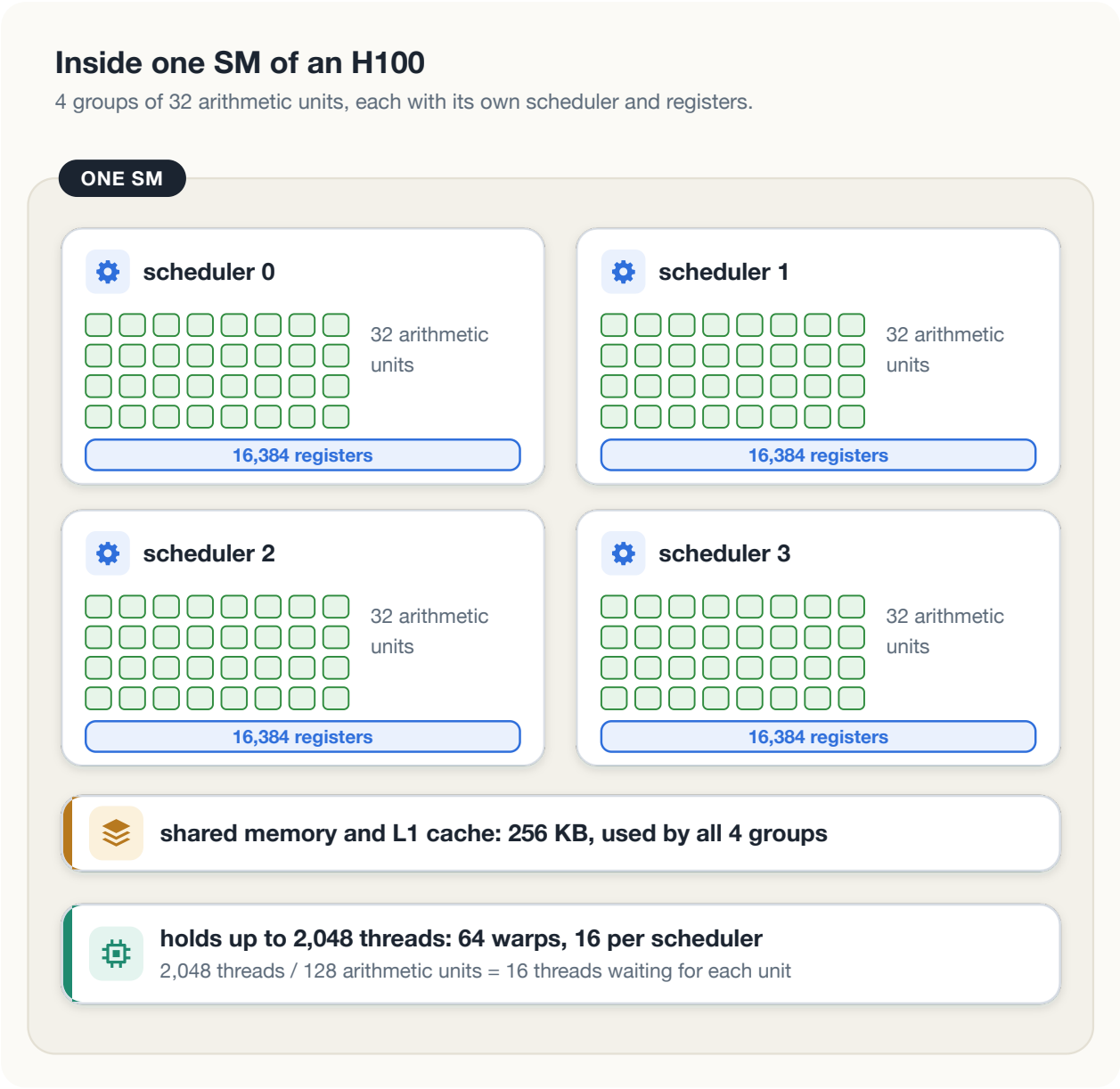


Figure 2.6 Inside 1 SM of an H100: 4 groups of 32 arithmetic units, each with a scheduler and 16,384 registers, above a shared pool of fast memory. The SM holds 16 threads for each arithmetic unit.

- **Arithmetic units.** 4 groups of 32 units for 32-bit floating-point numbers, 128 per SM, often called *CUDA cores*. A group has 32 units because a warp has 32 lanes (section 2.3): 1 instruction keeps a whole group busy. Each unit performs 1 **fused multiply-add** per cycle: it computes $a \times b + c$ in 1 step, which counts as 2 FLOPs.
- **Warp schedulers.** Each group has 1 warp scheduler. In every cycle a scheduler picks 1 warp whose next instruction is ready and sends that instruction to its 32 units.

- **Registers.** 65,536 registers of 4 bytes each, 65,536 × 4 = 262,144 bytes or 256 KB, split into 4 parts of 65,536 ÷ 4 = 16,384, 1 per group. Each thread on the SM gets its own registers from this file. As we will see in section 2.9, the registers a thread needs limit the number of threads that fit.
- **Shared memory and L1 cache.** 1 pool of 256 KB serves 2 purposes. **Shared memory** is memory that all threads of a block can read and write, so threads of a block exchange data through it. The **L1 cache** keeps copies of data the SM has read from main memory. On an H100, up to 228 KB of the pool can serve as shared memory.
- **Other units.** Units for integer arithmetic; **special function units**, which compute fast approximations of `sin`, `cos`, `exp` and square roots; and **tensor cores**, which multiply small matrices in 1 operation (Chapter 11).

Peak arithmetic follows from these parts. Each unit completes 1 fused multiply-add, 2 FLOPs, per cycle, and the clock rate counts cycles per second, so peak FLOPs per second = 2 × number of units × clock rate. For an H100, 2 × 132 SMs × 128 units × 1.98 GHz = 67 TFLOP/s. For an RTX 4090, 2 × 128 × 128 × 2.52 GHz = 82.6 TFLOP/s.

The SM holds far more threads than it has units to run them. An H100 SM can keep 2,048 threads resident, 16 for each of its 128 arithmetic units, and in any one cycle at most 128 of them execute. The other threads supply the independent work that a processor needs while its reads are in flight (section 1.1.5). We show in section 2.10 how a scheduler uses them.

> **NOTE:** SMs of other generations differ in detail. An A100 SM has 64 units for 32-bit floating-point numbers instead of 128. The number of threads and blocks an SM can hold, and its shared memory, depend on the compute capability; they are listed in section 2.9.



2.3 The warp

2.3.1 One instruction, 32 lanes

A warp scheduler issues 1 instruction to a warp (section 2.1.3), and all 32 lanes carry it out together, each on values in its own registers. Lane 0 holds the warp's first thread, lane 1 the next, and so on.

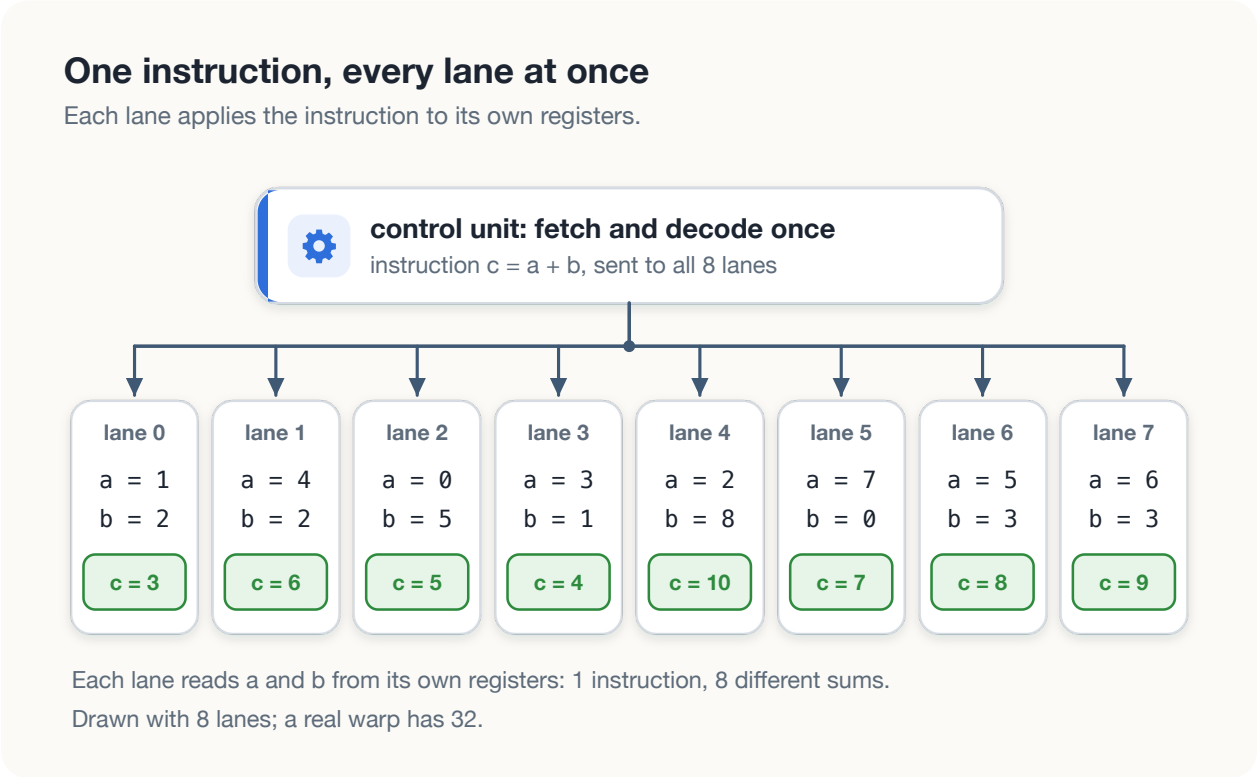


Figure 2.7 One instruction reaches every lane of a warp. Each lane has its own values, so one step gives one answer per lane.

Figure 2.7 draws 8 lanes instead of 32, to keep the values readable. A single instruction, `c = a + b`, gives 8 different results, because each lane holds its own `a` and `b`. In Chapter 1 we called this design SIMT (section 1.7).

The reason for executing threads in groups lies in the way any processor executes an instruction. A register called the **program counter** holds the address of the next instruction. A **control unit** fetches the instruction at that address, decodes it, and sends the resulting control signals to an arithmetic unit: which operation to perform, which registers hold its operands, and which register receives its result. In

a CPU core, 1 control unit serves 1 thread. In an SM, the control signals of 1 control unit go to 32 arithmetic units at once, and each of the 32 has its own registers (figure 2.7). For the instruction `add r1, r2, r3`, all 32 units add, but the values in `r2` and `r3` differ from unit to unit, so the 32 sums differ. The 32 sets of registers belong to the 32 threads of a warp.

A control unit is a complex circuit. It reads instructions from an instruction cache, decodes them, and tracks which operands are ready. Sharing 1 control unit among 32 arithmetic units saves chip area and power, and the SM spends what it saves on more arithmetic units. An H100 SM is divided into 4 such groups, called **processing blocks**, each with 1 control unit, 1 warp scheduler and 32 arithmetic units.

CPUs apply the same idea in their vector instructions, such as AVX on x86 processors (section 1.7). There the programmer, or the compiler, must place the data in vector registers explicitly. On a GPU, a programmer writes kernel code for 1 thread, and the SM forms the warps and executes them on its shared control units.

2.3.2 Why 32

NVIDIA GPUs have used warps of 32 threads since the first CUDA GPUs. The size balances 3 effects:

- A wider warp shares each instruction's cost among more threads.
- 32 threads reading 4 bytes each read 128 bytes, the size of 1 cache line. We explain in section 2.7 why this match helps.
- A wider warp loses more when its threads need to do different things (section 2.3.3).

The warp size is a property of the hardware and is reported by `cudaGetDeviceProperties` (section 2.9.4). A kernel whose correctness depends on the number 32 should read it there.

2.3.3 Divergence

All lanes of a warp carry out the same instruction. An `if` can send some threads of a warp one way and the rest the other way, and a warp cannot run both sides at

once. It runs them in turn (figure 2.8).

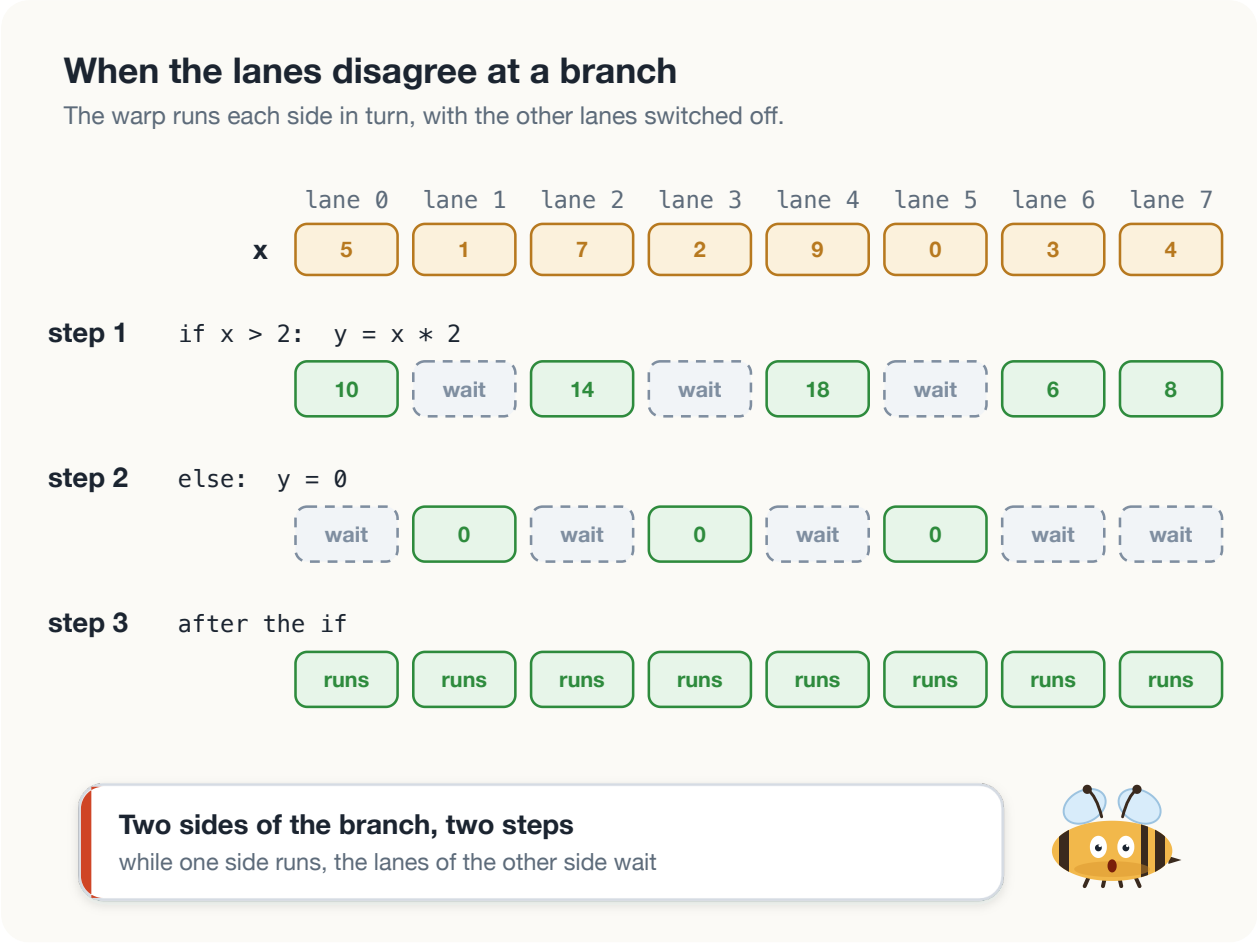


Figure 2.8 A branch on which the lanes of a warp disagree: 5 lanes take the `if` side and 3 the `else` side.

In figure 2.8 the condition `x > 2` holds in 5 lanes and fails in 3.

1. Step 1 runs the `if` side. 5 lanes compute `y = x * 2`; the other 3 are switched off.
2. Step 2 runs the `else` side. 3 lanes compute `y = 0`; the other 5 are switched off.
3. Step 3 comes after the branch, and all 8 lanes run together again.

A branch on which lanes of a warp disagree is called **divergence**. The results remain correct, because a switched-off lane does not write its registers or memory. The cost is time: a diverging warp spends the time of both sides, so when each side is long it takes about twice as long as a warp whose lanes agree.

Loops diverge in the same way. When each lane runs a loop a different number of times, the warp runs it as many times as its busiest lane, and a lane that has finished waits switched off. Suppose 8 lanes run 3, 1, 4, 1, 5, 9, 2 and 6 iterations: the warp runs 9 iterations, which is $9 \times 8 = 72$ lane-iterations of time, and the lanes need $3 + 1 + 4 + 1 + 5 + 9 + 2 + 6 = 31$ of them. The warp's lanes do useful work $31 / 72 = 43\%$ of the time. Such loops are common when each thread processes a row of irregular data, such as a sparse matrix, and the cost depends on the data more than on the code.

Boundary checks cause divergence that costs little. Most kernels check that their index lies inside the array, `if (i < n)`, because the grid is rounded up to whole blocks (Chapter 3). Only warps that straddle the end of the array diverge. With n = 1,000 and blocks of 256 threads, the grid has 4 blocks and 32 warps. Warp 31 holds threads 992 to 1,023, of which 992 to 999 pass the check and 1,000 to 1,023 fail it: 1 diverging warp of 32, about 3% of the warps. With n = 1,000,000 the grid has 31,256 warps. Since $1{,}000{,}000 = 31{,}250 \times 32$ exactly, warps 0 to 31,249 lie wholly inside the array and warps 31,250 to 31,255 wholly outside it, and no warp diverges at all. A boundary check costs almost nothing on a large array.

> **Independent thread scheduling.** Up to the Pascal generation (compute capability 6.x), a warp executed the 2 sides of a branch one after the other and then continued with all lanes together. From Volta (compute capability 7.0) on, each thread has its own program counter, and the scheduler may interleave the 2 sides of a branch. The results of a correct program are the same, but a kernel can no longer assume that the lanes of a warp are back together at any particular point after a branch. Code that relies on lanes exchanging data must say where they meet, with `__syncwarp()` or with the `_sync` forms of the warp functions that we use in Chapter 8.



2.4 Blocks, grids, and warps

When a kernel is launched, the hardware assigns its blocks to SMs and divides each block into warps. The program controls neither step, but the rules of both affect how well a launch uses the GPU.

2.4.1 Assigning blocks to SMs

Each block goes to 1 SM, whole. All threads of a block stay on 1 SM from start to finish, so they can share that SM's shared memory and wait for each other at a barrier (Chapter 5). A block starts only when the SM has room for all of its threads, registers and shared memory at once. If part of a block could start while the rest waited for room, the started threads could wait at a barrier for threads that would never arrive.

Blocks are assigned in any order, as room appears. A grid usually has many more blocks than the GPU can hold at once. The hardware keeps a list of the blocks not yet started, and whenever a block finishes and frees its place on an SM, it starts another one there. The program cannot tell in what order blocks run, and blocks of 1 grid cannot wait for each other. Because of this restriction, the same compiled kernel runs on 8 SMs or on 132: a GPU with more SMs runs more blocks at the same time. The property is called **transparent scalability**.

The blocks that run at the same time form a **wave**, and a grid runs as a sequence of waves. When the blocks take about equal time, the last wave decides how well the grid uses the GPU (figure 2.9). If the number of blocks is not a multiple of the number of places, the last wave is partly empty, and the GPU spends a whole wave's time on a fraction of a wave's work. The effect is called the **tail effect**.



Figure 2.9 20 blocks on 4 SMs and on 8 SMs. The larger GPU finishes in 2 waves instead of 3, but leaves most of its second wave idle.

WORKED EXAMPLE: 1,100 BLOCKS ON AN H100 AND ON A JETSON ORIN NX

A kernel uses blocks of 256 threads and 32 registers per thread, and its blocks take about equal time.

1. **H100.** An SM holds 2,048 threads, so 2,048 / 256 = 8 blocks, and 132 SMs hold 132 × 8 = 1,056 blocks at once. 1,100 blocks are 1,100 / 1,056 = 1.04 waves. The second wave runs 44 blocks, 4% of the places, for as long as a full wave. The GPU spends 2 waves of time on 1.04 waves of work: it is busy 1,100 / 2,112 = 52% of the time.
2. **Orin NX.** An SM holds 1,536 threads, so 6 blocks, and 8 SMs hold 48. 1,100 blocks are 22.9 waves. Only the 23rd wave is partly empty, and the GPU is busy 1,100 / (23 × 48) = 99.6% of the time.

The same grid wastes half of the larger GPU and almost none of the smaller one. On the H100, a grid of 1,056 blocks would finish in about the same time as one of 1,100. Real blocks differ in duration, and a new block starts as soon as any place frees up, so measured tails are softer than this calculation; the effect is still large when a grid has only 1 or 2 waves. A grid of many waves, or a block count that is a multiple of the GPU's places, avoids it. We show in section 2.9.4 how a program reads the number of SMs it runs on.

2.4.2 Dividing a block into warps

Each block is cut into warps of 32 consecutive threads. Threads 0 to 31 form warp 0, threads 32 to 63 form warp 1, and so on (figure 2.10). A program never creates warps itself; warps are how an SM runs a block.

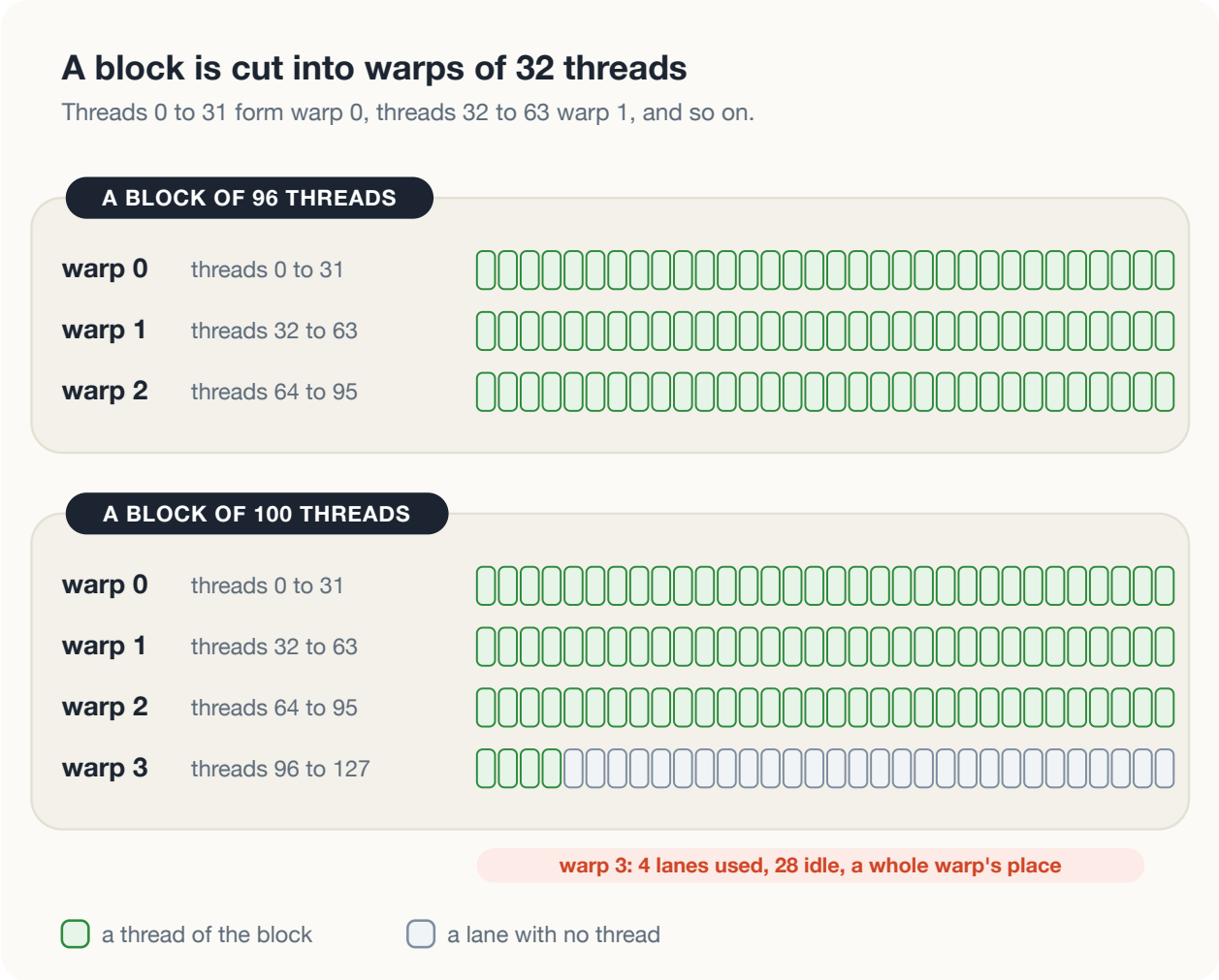


Figure 2.10 A block of 96 threads makes 3 full warps. A block of 100 makes 4, the last one nearly empty.

A block of 100 threads needs 4 warps: 3 full warps hold 96 threads, and the 4 threads left over need a 4th warp. In general a block needs its size ÷ 32 warps, rounded up. The 4th warp's other 28 lanes do no work, yet the warp still takes a full warp's place on the SM. Block sizes are therefore almost always multiples of 32, such as 128 or 256.

A block can also be 2- or 3-dimensional, for example 16 × 16 threads covering a tile of an image. Its threads are first numbered in a single sequence, x fastest, then y, then z, and the sequence is then cut into warps as before. For a block of $D_x \times D_y$ threads, the thread at (x, y, z) has

$$\text{linear number} = x + y \cdot D_x + z \cdot D_x \cdot D_y, \qquad \text{warp} = \left\lfloor \frac{\text{linear number}}{32} \right\rfloor$$

In a 16 × 16 block, the thread at (x = 5, y = 3) has linear number 5 + 3 × 16 = 53 and lies in warp 1. Each warp of this block covers 2 rows of 16: warp 0 holds rows 0 and 1, warp 1 rows 2 and 3. In an 8 × 8 block, the 64 threads form 2 warps, rows 0 to 3 and rows 4 to 7. The 32 lanes of a warp read memory together, so this numbering decides which addresses a warp reads at once (sections 2.7 and 2.8). In Chapter 3 we draw the numbering for 3-dimensional blocks.

	Block	Warp
chosen by	the program, at launch	the hardware
size	up to 1,024 threads	always 32 threads
its threads can	share memory and wait for each other	carry out each instruction together

2.5 The memory hierarchy

A thread can keep data in several kinds of memory. Memory closer to the arithmetic units is faster and smaller. Together, these levels form the **memory hierarchy** (figure 2.10).

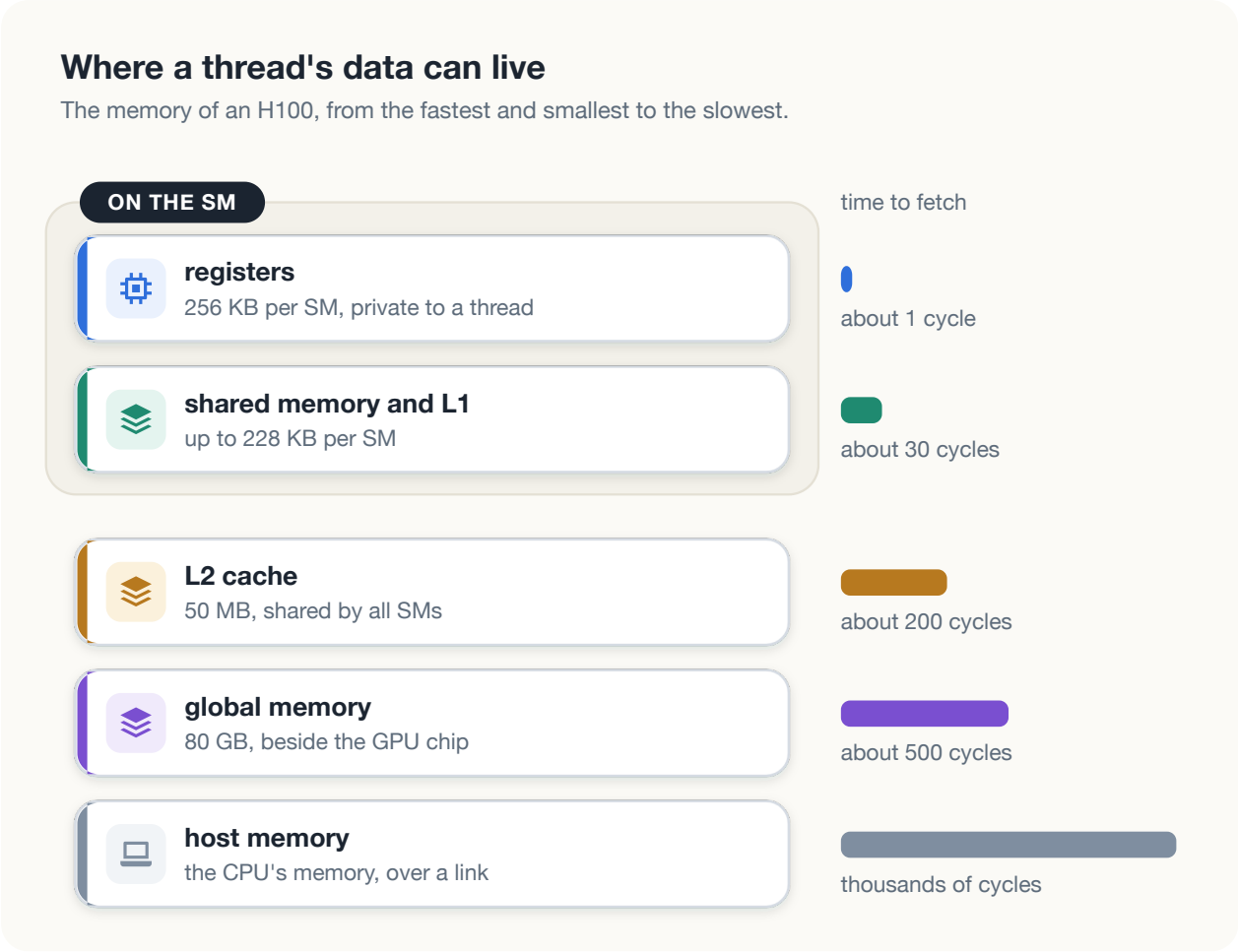


Figure 2.11 The memory hierarchy of an H100, from the fastest and smallest level to the slowest.

Registers belong to 1 thread. A thread's local variables live in registers when they fit. An arithmetic instruction names its registers directly, as in `add r1, r2, r3`, so a value in a register reaches the arithmetic unit with no further instruction. A value in memory needs a load first, `load r2, [address]`, which occupies an issue slot and waits for the memory to answer. An SM's 65,536 registers are shared out among all its threads, so a thread needing many registers leaves room for fewer threads (section 2.9).

Local memory is the place for a thread's private data that does not fit in registers: values the compiler moves out of registers when it runs short, called a **spill**, and arrays that the thread indexes with values known only at run time. Despite its name, local memory lies in global memory, the slowest level on the GPU, and is local only in the sense that each thread sees its own part of it. A spill therefore turns a 1-cycle register access into a read that may take hundreds of cycles. In Chapter 7 we show how to detect one.

Shared memory belongs to 1 block and sits on the SM. Any thread of a block can read what another thread of it wrote, so threads of a block cooperate through shared memory. A read takes 20 to 30 cycles, about 20 times less than a read from main memory. A program decides what goes into shared memory and when; we rely on it in Chapters 7 to 9.

L1 cache uses the same on-chip memory as shared memory, but hardware manages it: data read from main memory stays there in case it is read again soon. Caches store data in fixed-size pieces called **lines**, 128 bytes each on these GPUs. An SM's L1 serves its many resident threads at once, and they compete for its lines, so a kernel cannot count on finding a value in L1 that it read some time ago. For data a block will reuse, shared memory gives a guarantee that the cache does not.

L2 cache is 1 cache shared by all SMs, 50 MB on an H100, with reads of about 200 cycles. Every SM goes through it, so blocks on different SMs see each other's writes there (Chapter 5).

Global memory is the GPU's main memory, built from DRAM chips outside the GPU chip. On an H100 it is 80 GB of HBM3, stacked in the same package as the GPU chip. On an RTX 4090 it is 24 GB of GDDR6X chips on the card. On a Jetson Orin NX it is the 16 GB of LPDDR5 that the CPU also uses. Every array a program allocates for the GPU lives there (Chapter 4). An H100 reads it at 3.35 TB/s, but a single read takes about 500 cycles, and much of the optimisation in Chapters 7 to 9 reduces the number of reads from it.

Host memory is the CPU's memory; CUDA calls the CPU the **host**. A request for host memory leaves the GPU and crosses a link to the CPU, so it is the slowest level. In Chapter 4 we cover moving data across that link efficiently.

2 read-only paths complete the hierarchy. **Constant memory** is fast when every thread of a warp reads the same value, such as a kernel's parameters. **Texture memory** is designed for reading 2-dimensional images. We cover both in Chapter 7.

The place where a variable lives is decided by how it is declared. Each kind of declaration also fixes the variable's **scope**, the threads that can see it, and its **lifetime**, the part of the program's run during which it exists.

Declaration in CUDA C++	Lives in	Scope	Lifetime
scalar variable inside a kernel	register	1 thread	1 launch
array inside a kernel	registers if small and indexed with constants, else local memory	1 thread	1 launch
`__shared__` variable inside a kernel	shared memory	1 block	1 launch
`__device__` variable at file scope	global memory	all threads of all launches	the whole program
`__constant__` variable at file scope	constant memory	all threads of all launches	the whole program

A scalar declared inside a kernel launched with 1 million threads exists 1 million times, once per thread, and disappears when each thread ends. A `__shared__` variable exists once per block and disappears with the block, so a later launch cannot read what an earlier one left there.

> **NOTE:** On integrated systems, such as Jetson modules and DGX Spark, CPU and GPU share 1 set of memory chips, so global memory and host memory are the same hardware. The hierarchy still holds: a read from those chips takes hundreds of cycles, while registers and shared memory sit on the SM.



The energy of a read. The levels of the hierarchy also differ in energy. Mark Horowitz's widely used figures for a 45 nm process put a 32-bit floating-point addition at about 0.9 pJ and a 64-bit read from DRAM at 1,300 to 2,600 pJ, between 1,400 and 2,900 times as much. A read from a small on-chip memory of 8 KB costs about 10 pJ. The numbers are older than any GPU in this chapter, and both have fallen since, but the ratio between arithmetic and off-chip access has stayed large. A kernel that reads each value from DRAM once instead of many times saves energy as well as time.



2.6 The latency table

The table gives each level's read time. Values are typical for a recent GPU and rounded; we measure real ones in Chapter 7.

Where the data is	Time to read it	Location
register	about 1 cycle	inside the SM
shared memory	20 to 30 cycles	inside the SM
L1 cache	about 30 cycles	inside the SM
L2 cache	about 200 cycles	elsewhere on the chip
global memory	400 to 800 cycles	in separate memory chips
host memory	thousands of cycles, plus the transfer	across a link to the CPU

Read time grows with distance. A signal needs time to travel, and data further from the arithmetic units passes through more circuits on its way. Leaving the SM is the largest step: a read from global memory costs about 20 times a read from shared memory. In Chapter 1 we explained the remaining part of the cost of global memory, the time a DRAM chip needs to read its own cells (section 1.1.2).

WORKED EXAMPLE: READING 1 VALUE 20 TIMES

A kernel needs the same value 20 times.

1. Reading it from global memory each time costs $20 \times 500 = 10{,}000$ cycles.
2. Reading it once from global memory into shared memory, then 20 times from shared memory, costs $500 + 20 \times 25 = 1{,}000$ cycles.

The second way is 10 times cheaper. In Chapter 9 we build a matrix multiplication on this trade.

2.7 Coalescing: how a warp reads memory

1 instruction drives all 32 lanes of a warp (section 2.3). When that instruction reads global memory, a warp asks for 32 values at once, 1 per lane. How fast they arrive depends on where those 32 values lie in memory.

2.7.1 Sectors

Global memory does not deliver single values. It delivers 32-byte pieces called **sectors**; 4 sectors make 1 cache line of 128 bytes. A line of 128 bytes holds $128 \div 4 = 32$ floats, 1 for each lane of a warp. Fetching a sector costs the same however many of its bytes a warp uses, so a warp's read costs about 1 fetch per sector it touches.

The reason lies in the DRAM chips. As we saw in section 1.1.2, the slow part of a DRAM read is opening a row of cells and sensing their charges. Once a row is open, the chip can deliver many consecutive bytes from it quickly, one after another, in a **burst**. The cost of opening the row is paid once for the whole burst. A memory system therefore moves data in pieces of consecutive bytes, and a read of 4 bytes costs nearly as much as a read of the whole piece that contains them. The sector is the piece in which a GPU's memory system deals.

2.7.2 Worked by hand

Figure 2.12 shrinks everything by a factor of 4: a warp of 8 lanes, each reading 1 `float` of 4 bytes, and sectors of 8 floats.

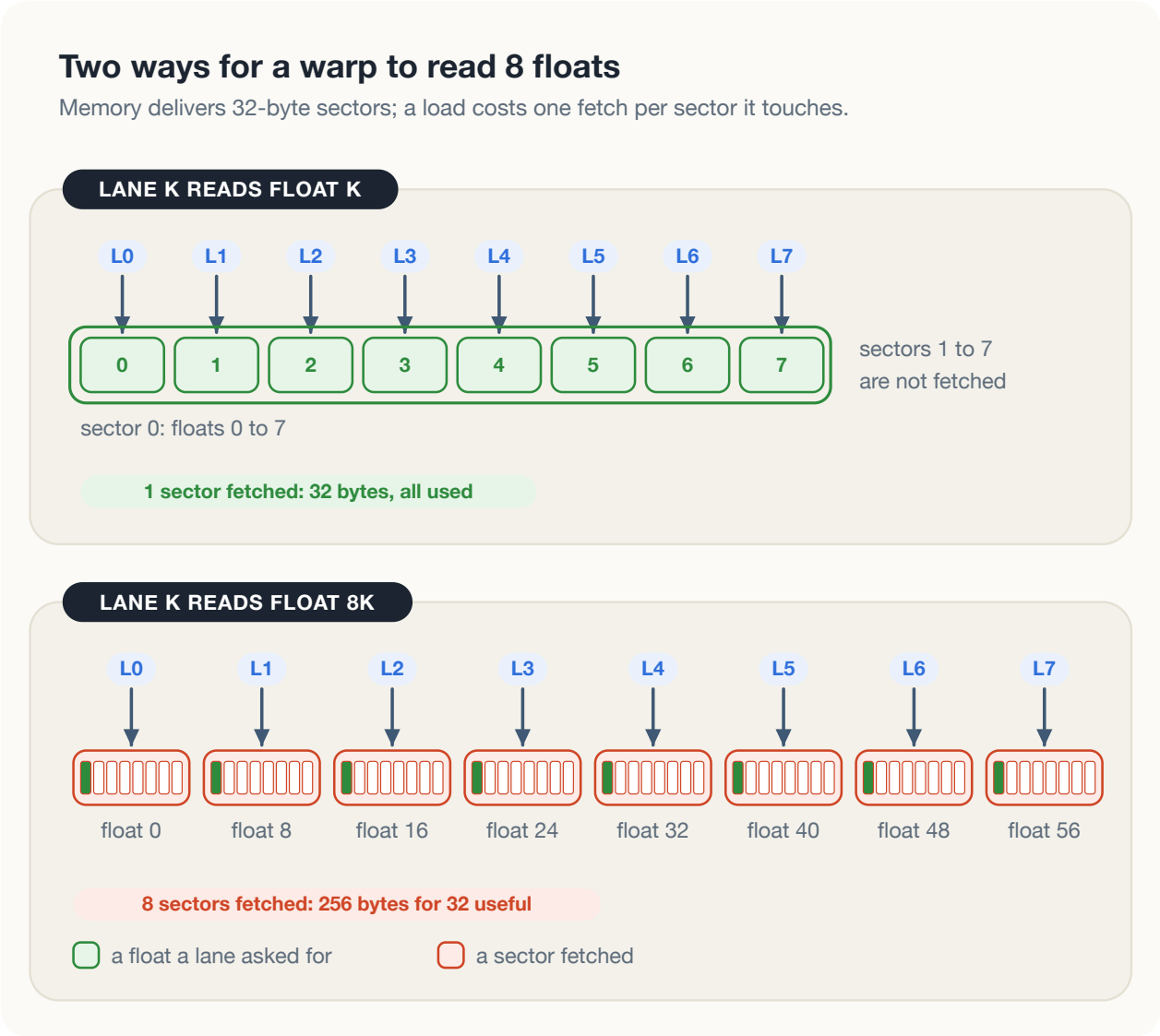


Figure 2.12 8 lanes reading 8 floats: from 1 sector when the floats are consecutive, from 8 sectors when they are 8 floats apart.

- **Lane k reads float k.** Floats 0 to 7 lie side by side in sector 0. Memory fetches 1 sector, 32 bytes, and every byte is used.
- **Lane k reads float 8k.** Floats 0, 8, 16 and so on up to 56 each lie in a different sector. Memory fetches 8 sectors, 256 bytes, to deliver the same 32 useful bytes: 8 times the traffic for the same result.

A real warp behaves the same way with larger numbers. 32 lanes reading 32 consecutive floats need 128 bytes, 4 sectors. Floats 32 apart are $32 \times 4 = 128$ bytes apart, a whole line, so 32 lanes reading every 32nd float touch 32 different sectors and fetch $32 \times 32 = 1{,}024$ bytes for $32 \times 4 = 128$ useful ones.

2.7.3 The rule

A read whose addresses fall into as few sectors as possible is **coalesced**; a read whose addresses spread out is **uncoalesced**. Hardware does not rearrange a warp's requests: it fetches every sector any lane needs.



Neighbouring threads should read neighbouring addresses.

A memory-bound kernel runs at the rate its bytes arrive, so the uncoalesced read above, which moves 8 times the bytes, runs several times slower. In Chapter 7 we measure, on the Orin NX, 68 GB/s of useful bandwidth for reads 4 bytes apart and 21 GB/s for reads 32 bytes apart.

2.7.4 Rows and columns of a matrix

Whether a kernel's reads are coalesced can be read off its index expression. In vector addition, thread i reads `a[i]` with i = block number × block size + position (section 2.1.2). The lanes of a warp have consecutive positions and the same block number, so they read consecutive elements, and the 32 floats of a warp fill 4 sectors exactly.

A 2-dimensional array is stored row after row. In a matrix of W columns, element (row, col) lies at address row × W + col, counted in elements. This order is called **row-major**, and C, C++ and Rust all use it. Figure 2.13 shows a matrix of 4 rows and 8 columns, with sectors of 8 floats as in figure 2.12.

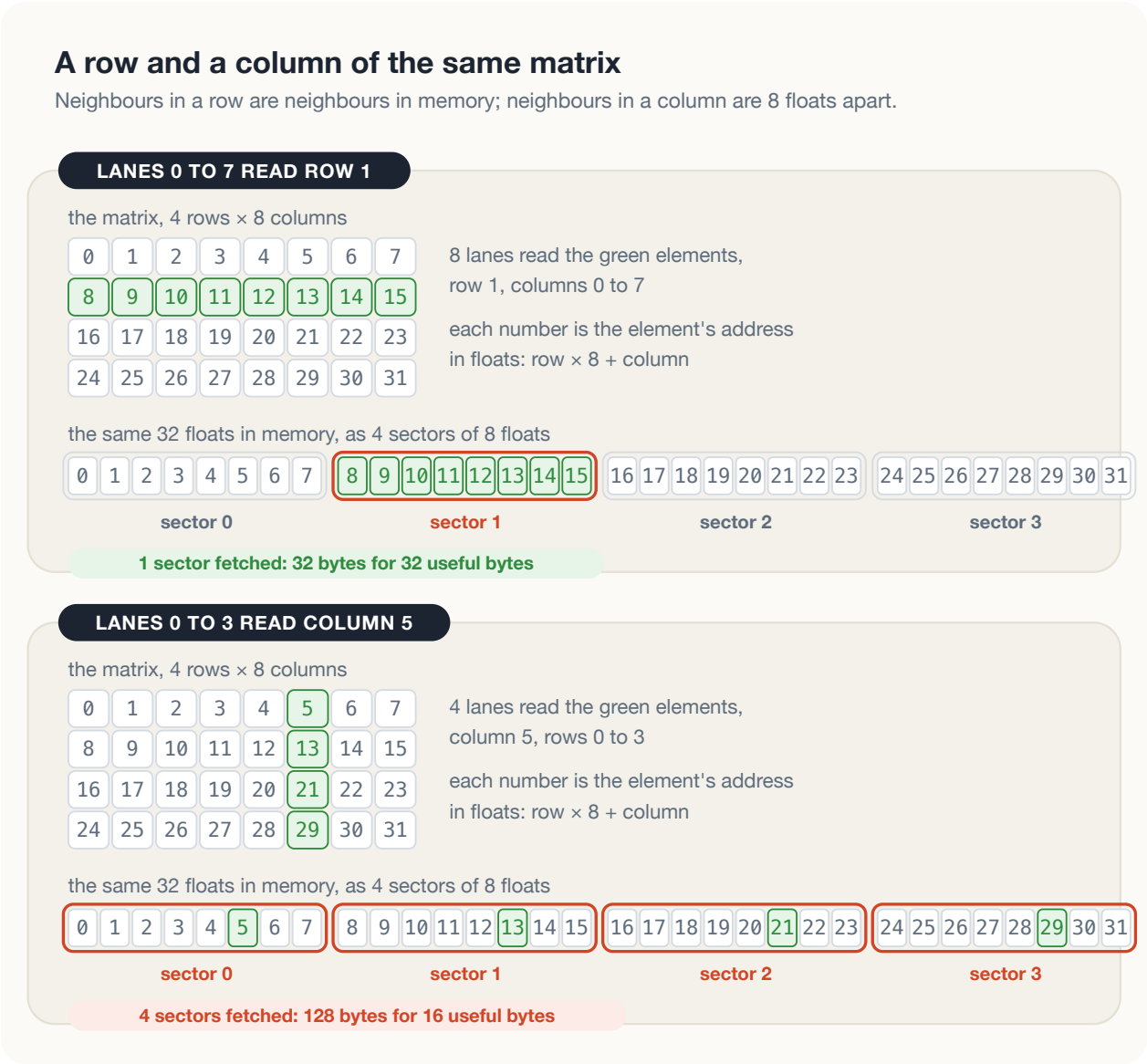


Figure 2.13 A row of a row-major matrix lies in 1 sector; a column of 4 elements lies in 4.

- **Lanes 0 to 7 read row 1.** Lane k reads element (1, k), at address $1 \times 8 + k$. The addresses 8 to 15 are consecutive and fill sector 1: 1 sector, 32 bytes, all used.
- **Lanes 0 to 3 read column 5.** Lane k reads element (k, 5), at address $k \times 8 + 5$. The addresses 5, 13, 21 and 29 are 8 floats apart and lie in 4 different sectors: 128 bytes fetched for 16 useful bytes.

The rule follows from the index expression: the variable that changes from lane to lane must be the one that is multiplied by 1, the column. In a matrix of 4,096 columns, a warp reading down a column touches addresses 4,096 floats, 16 KB, apart: 32 sectors for 32 floats, 8 times the bytes it uses. In Chapter 7 we show how

a kernel that must read columns, such as a matrix transpose, reads a tile by rows into shared memory and then reads the columns from there.

2 further details complete the rule. A warp's read is fully efficient only if it starts at the start of a sector: 32 floats starting at float 4 occupy bytes 16 to 143 and touch 5 sectors instead of 4. Writes follow the same rule as reads, and an uncoalesced write can cost more than an uncoalesced read. On GPUs whose memory uses error-correcting codes, a write that covers only part of a sector can force the memory to read the sector before writing it.

2.8 Shared-memory banks

Shared memory sits on the SM and has to answer all 32 lanes of a warp at once. It is built from SRAM, a faster kind of memory than DRAM that answers in a few cycles and needs no bursts, so it can be organised in narrow parts that each deliver a single word. It consists of 32 separate memory arrays called **banks**, and each bank delivers 1 word of 4 bytes per cycle. Working side by side, 32 banks deliver 32 words in 1 cycle: 1 word for each lane.

Consecutive words lie in consecutive banks. Word 0 goes to bank 0, word 1 to bank 1, and so on to word 31 in bank 31; word 32 starts the next round in bank 0. A word's bank is therefore the remainder when its number is divided by 32:

$$\text{bank} = \text{word number} \bmod 32$$

Word 70, for example, is 2 full rounds of 32 plus 6, so it lies in bank 6.

A warp's read takes 1 cycle when no bank is asked for more than 1 word. When 2 lanes ask 1 bank for different words, the bank delivers them in turn. A read that has to wait for a bank in this way has a **bank conflict**.

Figure 2.14 uses 8 banks and 8 lanes, so a word's bank is its number modulo 8. Words are drawn in rows of 8, so each column is 1 bank.

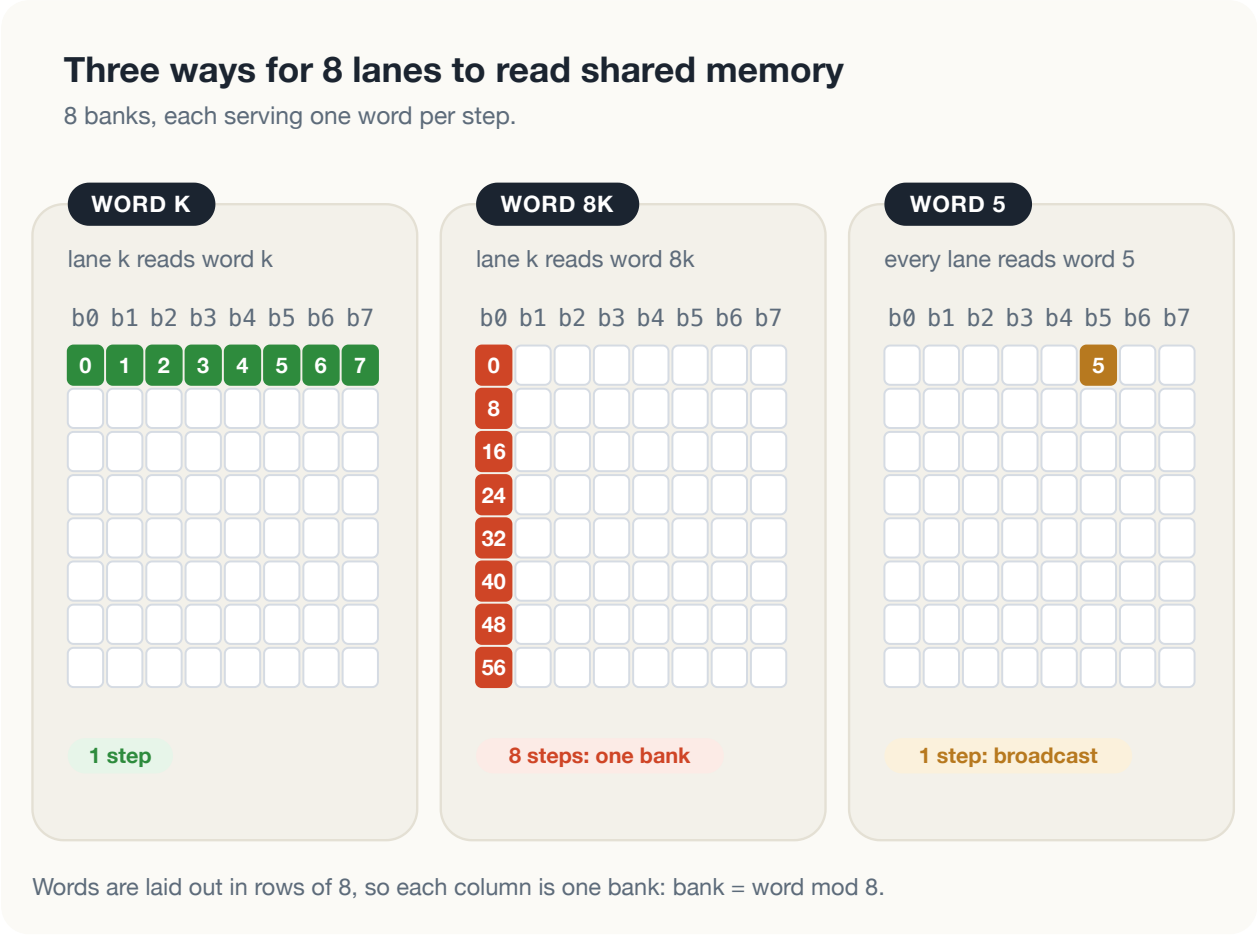


Figure 2.14 8 lanes reading shared memory with 8 banks: 1 step for a row, 8 steps for a column, 1 step when all lanes read the same word.

- **Lane k reads word k.** Words 0 to 7 lie in banks 0 to 7, 1 each: 1 step.
- **Lane k reads word 8k.** Words 0, 8, 16 and so on all lie in bank 0, which serves them 1 at a time: 8 steps for 1 read.
- **Every lane reads word 5.** Many lanes ask 1 bank for the *same* word. The bank reads it once and hands it to every lane, a **broadcast**: 1 step.

With 32 banks, the worst case is 32 lanes asking 1 bank for 32 different words: 32 steps. In general, a warp's read takes as many steps as its busiest bank has different words to deliver.

The middle case appears whenever threads read down a column of a 32×32 table of floats in shared memory. Element (row r, column c) is word $32r + c$, and $(32r + c) \bmod 32 = c$: the row drops out, so every element of column c lies in bank c. We apply the usual fix in Chapter 7. With 1 unused word added to each row, element (r, c) becomes word $33r + c$, whose bank is $(r + c) \bmod 32$. That bank differs from row to row, so the column spreads across all 32 banks.

The same conflict arises in ordinary code. A block of 32 × 32 threads that stores a tile in transposed order,

```
__shared__ float tile[32][32];
tile[threadIdx.x][threadIdx.y] = value;
```

has warps whose 32 lanes share 1 value of `threadIdx.y` and have consecutive values of `threadIdx.x` (section 2.4.2). Lane k writes `tile[k][y]`, at word 32k + y, and every lane writes to bank y: a 32-way conflict, 32 steps instead of 1. Declaring `tile[32][33]` moves lane k's word to 33k + y, in bank (k + y) mod 32, and the 32 lanes write to 32 different banks. Padding has 2 costs. The tile occupies 32 × 33 × 4 = 4,224 bytes instead of 4,096, 3% more shared memory. And the address 33k + y cannot be computed with a shift, as 32k = k << 5 can, so each access needs 1 more addition.

▶ CHECK YOUR UNDERSTANDING: **A warp reads a shared-memory table of floats with 32 columns, each lane reading one element of the same column. How many steps does the read take, and why?**

2.9 Occupancy

2.9.1 Why an SM holds many warps

The threads an SM holds at a given moment are its **resident** threads, and their warps its resident warps. When 1 warp reads global memory and must wait about 500 cycles, the scheduler switches to another resident warp that is ready. More resident warps mean better odds that 1 of them is ready.

An SM's **occupancy** is its number of resident warps divided by the most it can hold. An H100 SM holds up to 2,048 threads, which is 2,048 ÷ 32 = 64 warps, so 32 resident warps give an occupancy of 32 ÷ 64 = 50%.

2.9.2 What limits occupancy

An SM does not give each block a fixed share of its resources. It divides its registers, shared memory, thread places and block places among the blocks that arrive, as each needs them. An SM can therefore hold many small blocks or a few large ones, and 4 limits apply at once: whichever runs out first sets the occupancy. Every compute capability below has 65,536 registers per SM; the other 3 limits vary.

Compute capability	Examples	Resident threads (warps)	Resident blocks	Shared memory
8.0	A100	2,048 (64)	32	up to 164 KB
8.6	GeForce RTX 30 series	1,536 (48)	16	up to 100 KB
8.7	Jetson Orin	1,536 (48)	16	up to 164 KB
8.9	GeForce RTX 40 series	1,536 (48)	24	up to 100 KB
9.0	H100	2,048 (64)	32	up to 228 KB
10.0	B200	2,048 (64)	32	up to 228 KB
12.x	GeForce RTX 50 series, DGX Spark	1,536 (48)	32	up to 100 KB

Values come from NVIDIA's *CUDA C++ Programming Guide* and its tuning guide for each architecture.

The thread and block limits depend on the block size in 2 ways.

- **Small blocks run out of block places.** With blocks of 32 threads, an H100 SM would need 2,048 / 32 = 64 blocks to fill its thread places, but it holds at most 32 blocks: 32 × 32 = 1,024 threads, 50% occupancy. On a Jetson Orin, with 16 block places, the same blocks fill 16 × 32 = 512 of 1,536 thread

places, 33%. Blocks of at least 64 threads on the H100, and of at least 96 on the Orin, avoid this limit.

- **Block sizes that do not divide the thread limit leave places empty.** With blocks of 768 threads, an H100 SM holds 2 blocks, 1,536 of its 2,048 places: 75%. A third block would need 2,304 places. On a Jetson Orin, the same 2 blocks fill all 1,536 places: 100%. The best block size depends on the GPU.

WORKED EXAMPLE: THE REGISTER BUDGET, BLOCKS OF 256 THREADS ON AN H100 SM

A compiler decides how many registers each thread of a kernel needs; in Chapter 9 we show how to see that number and how to cap it. The SM hands out registers to each warp in units of 256, so a thread's count is rounded up to a multiple of 256 / 32 = 8.

Registers per thread	Allocated	Registers per block	Blocks that fit	Resident threads	Occupancy
32	32	256 × 32 = 8,192	65,536 / 8,192 = 8	2,048	100%
33	40	256 × 40 = 10,240	65,536 / 10,240 = 6.4, so 6	1,536	75%
64	64	16,384	4	1,024	50%
128	128	32,768	2	512	25%

Each row divides the SM's 65,536 registers by 1 block's need: the register file has enough for 8, 6, 4 or 2 blocks. The thread limit allows 2,048 / 256 = 8 blocks and the block limit 32, and this kernel uses no shared memory, so registers set the limit in every row.

In the second row, 1 additional register per thread, perhaps from 1 more variable in the kernel, moves the allocation from 32 to 40 and the occupancy from 100% to

75%. Occupancy changes in steps, and a small edit can move a kernel from one step to the next. Such a drop is sometimes called a **performance cliff**.

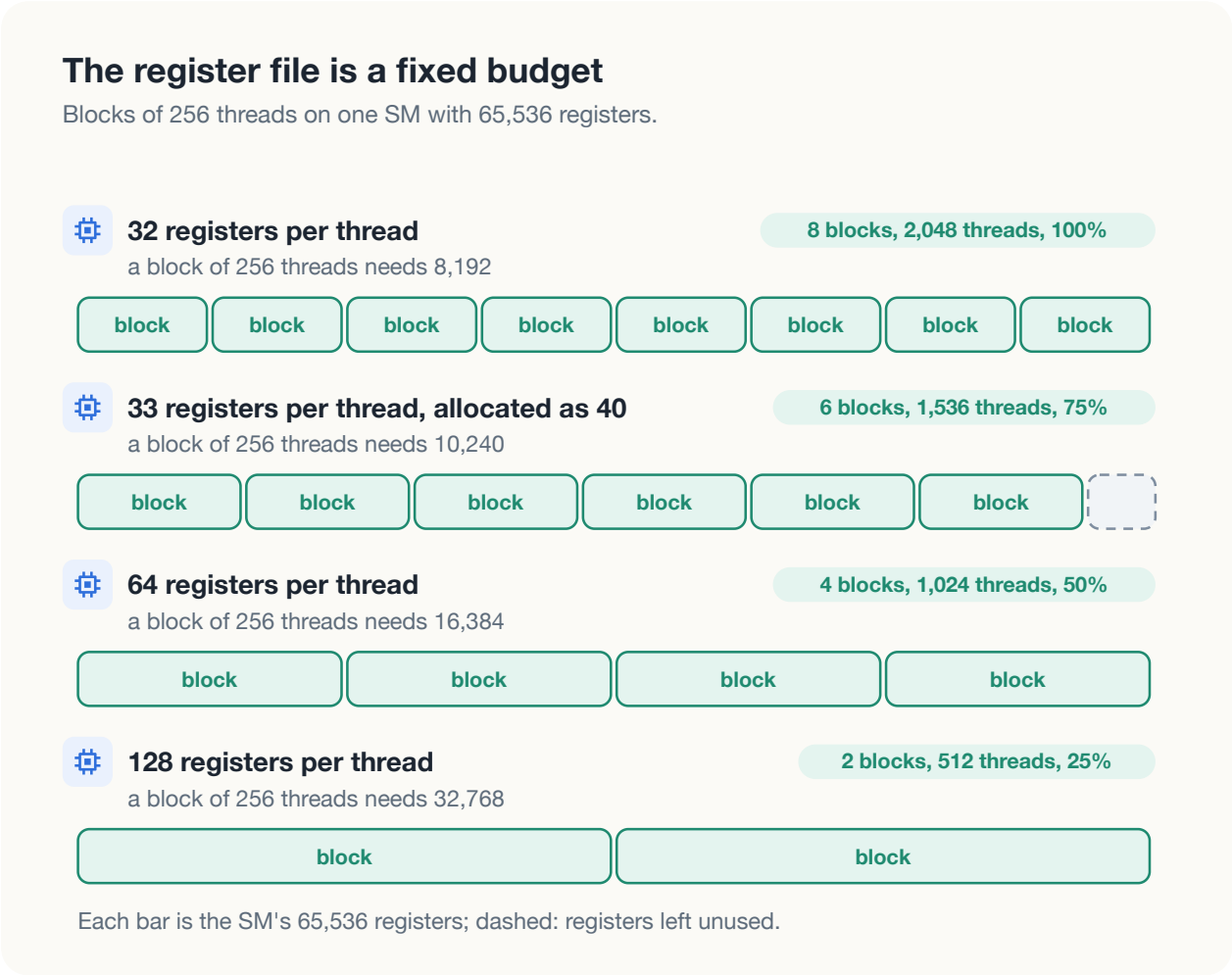


Figure 2.15 The register file is a fixed budget: the more registers each thread needs, the fewer blocks fit. 1 register more than 32 costs 2 of 8 blocks.

As a formula, with T_B threads per block and R_T registers per thread after rounding up to a multiple of 8, the number of blocks that fit on an SM is the smallest of

$$\left\lfloor \frac{65{,}536}{T_B \cdot R_T} \right\rfloor, \quad \left\lfloor \frac{\text{thread limit}}{T_B} \right\rfloor, \quad \text{block limit}, \quad \left\lfloor \frac{\text{shared memory per SM}}{\text{shared memory per block}} \right\rfloor$$

where $\lfloor\ \rfloor$ means rounding down, since part of a block cannot be resident. Occupancy is then that number of blocks times T_B, divided by the thread limit.

2.9.3 More is not always better

High occupancy helps a kernel that often waits for memory, because more warps give the scheduler more chances to find 1 that is ready. High occupancy is not a goal in itself.

- A kernel whose data already sits in registers rarely waits, and can run at full speed with few warps.
- Forcing a kernel to use fewer registers, so that more warps fit, can make the compiler spill values to local memory, adding the very memory traffic the extra warps were meant to hide.
- A kernel keeping a large tile of data in shared memory may run fewer blocks per SM on purpose, because each block then does more work per byte it reads.

A kernel needs enough ready warps to keep the schedulers busy, and no more. In Chapter 9 we measure the trade between registers and occupancy on a matrix multiplication.

2.9.4 Reading the limits of a GPU

The limits of this section differ between GPUs, and a program that runs on more than one should read them at run time instead of assuming them. The CUDA runtime reports them through `cudaGetDeviceProperties`, which fills a `cudaDeviceProp` structure for a given device. For an H100 and a Jetson Orin NX, the fields that hold the limits of this section have the values in the table.

Field of `cudaDeviceProp`	Meaning	H100	Jetson Orin NX
`multiProcessorCount`	number of SMs	132	8
`maxThreadsPerMultiProcessor`	resident threads per SM	2,048	1,536
`maxBlocksPerMultiProcessor`	resident blocks per SM	32	16
`regsPerMultiprocessor`	registers per SM	65,536	65,536
`sharedMemPerMultiprocessor`	shared memory per SM, in bytes	233,472	167,936
`maxThreadsPerBlock`	threads per block	1,024	1,024
`warpSize`	threads per warp	32	32

The shared-memory figures are the 228 KB and 164 KB of the table in section 2.9.2, written in bytes: 228 × 1,024 = 233,472. For a particular kernel, the function `cudaOccupancyMaxActiveBlocksPerMultiprocessor` applies all 4 limits, with the kernel's real register and shared-memory use, and returns the number of blocks that fit on an SM. Multiplied by `multiProcessorCount`, it gives the number of places in a wave (section 2.4.1). We call it in a complete program in Chapter 9 and return to both functions in Chapter 16.

2.10 The SM in action: time slicing

Figure 2.16 follows 1 scheduler with 4 warps. To keep the picture small, a read from memory takes 8 cycles instead of about 500.



Figure 2.16 One scheduler and 4 warps. While warp 0 waits 8 cycles for its data, warps 1 to 3 issue an instruction in every cycle.

1. In cycle 0, warp 0 issues a read. Its data arrives at the end of cycle 8.
2. In cycles 1 to 8, the scheduler issues instructions from warps 1, 2 and 3, whichever is ready. No cycle stays empty.
3. In cycle 9, warp 0 has its data and issues the add that needed it.

No code in a program makes this happen. A scheduler picks among its resident warps in every cycle, and the switch from 1 warp to another costs nothing. A program's part is to provide enough warps (occupancy) and enough independent work in each warp.

Why switching warps costs nothing. A CPU that switches from 1 thread to another, for example when the operating system shares a core between 2 programs, must save the registers of the outgoing thread to memory and load those of the incoming one. The switch costs hundreds of cycles or more, which is why a CPU core does not switch threads to cover a single memory read. An SM avoids the copy entirely: the registers of every resident warp stay in the register file for as long as the warp is resident, and a switch only changes which warp's registers the next instruction uses. The cost is size. An H100 SM's register file holds 256 KB, so the 132 SMs together hold 132 × 256 KB = 33 MB of registers, about 2/3 of the size of the chip's 50 MB L2 cache. The register file is that large because it holds the state of every resident warp.



Little's law (section 1.1.4) gives the number of warps needed. A scheduler issuing 1 instruction per cycle against a latency of 500 cycles needs about 500 independent instructions in flight to stay busy. No SM holds 500 warps per scheduler, and none needs to: a warp usually has several independent instructions between reads, and can have several reads in flight at once. In practice, a few dozen warps per SM cover the wait. The worked example of section 1.1.5 reached the same conclusion from the side of memory: an H100 needs about 106,000 threads in flight for vector addition, 40% of the 270,336 it can hold.

2.11 Architecture generations

The design described in this chapter has been stable across NVIDIA GPUs for more than a decade, while the sizes have changed. Each generation is identified by a **compute capability**, a version number of the form *major.minor*.

Compute capability	Architecture	Examples
7.x	Volta, Turing	V100, GeForce RTX 20 series
8.0, 8.6, 8.7	Ampere	A100 (8.0), GeForce RTX 30 series (8.6), Jetson Orin (8.7)
8.9	Ada Lovelace	GeForce RTX 40 series
9.0	Hopper	H100
10.0, 11.0, 12.0, 12.1	Blackwell	B200 (10.0), Jetson AGX Thor (11.0), GeForce RTX 50 series (12.0), DGX Spark (12.1)

Each generation changes the number of SMs, the size of shared memory, the tensor cores and many of the limits of section 2.9. A performance figure is only meaningful together with the compute capability on which it was measured. The `deviceQuery` sample that ships with CUDA prints the limits of the GPU it runs on; in Chapter 16 we show how to read its output.

Common mistakes

- **Treating higher occupancy as always faster.** Spilled registers or smaller shared-memory tiles can make a kernel at 50% occupancy faster than one at 100%. Measure.
- **Forgetting the warp.** A kernel is written for one thread, but the hardware runs warps. Divergence, coalescing and bank conflicts all depend on what the 32 lanes of a warp do together.
- **Assuming the lanes of a warp stay together.** Since Volta, lanes may separate after a branch. Code that exchanges data between lanes must use `__syncwarp()` or the `_sync` warp functions.
- **Reading a shared-memory table by column.** A column of a 32 × 32 table of floats lies in one bank, so reading it takes 32 times as long as reading a row. Padding each row by one element fixes it (Chapter 7).
- **Launching a grid slightly larger than a whole number of waves.** On an H100 with 1,056 places, 1,100 equal blocks take about as long as 2,112.

- **Quoting one GPU's numbers for another.** Read the limits of the GPU the program actually runs on with `cudaGetDeviceProperties`.

Summary 

- A **kernel** is a function that runs on the GPU. A launch runs it as many **threads**, organised in **blocks**; the blocks of a launch form the **grid**.
- A GPU is a set of **SMs**. Each block runs on one SM, whole, and blocks start in any order as places free up, which gives **transparent scalability**. Blocks that run at the same time form a **wave**; a partly empty last wave wastes the GPU.
- The SM runs threads in **warps** of 32 that carry out each instruction together. When the lanes of a warp disagree at a branch or run a loop different numbers of times, the warp pays for every path: **divergence**.
- A block is divided into warps of 32 consecutive threads, numbered x fastest, then y, then z, so block sizes should be multiples of 32.
- The memory hierarchy runs from registers (about 1 cycle) through shared memory (about 30), the L2 cache (about 200) and global memory (about 500) to host memory. **Local memory** is private to a thread but lies in global memory.
- Global memory delivers 32-byte sectors, because DRAM delivers bursts of consecutive bytes, so neighbouring threads should read neighbouring addresses: **coalescing**.
- Shared memory has 32 **banks**; lanes that ask one bank for different words must wait for each other.
- **Occupancy** is the share of an SM's warp places that are filled. Registers, threads, blocks and shared memory all limit it, and it changes in steps. Enough warps let the scheduler hide memory latency, and switching between them costs nothing because every resident warp keeps its registers on the SM.
- A memory-bound kernel can be judged against its **speed of light**: the bytes it moves divided by the bandwidth.

In Chapter 3 we turn from the hardware to the code: how a kernel is written, how a grid of blocks is described, and how each thread finds its own piece of work.

Exercises

1. A kernel uses 64 registers per thread. How many threads can one SM hold before its 65,536 registers run out?
2. The same kernel uses 128 registers per thread. What share of an H100 SM's 2,048 thread places can be filled?
3. On the Orin NX an SM holds at most 1,536 threads and 16 blocks. A kernel uses 32 registers per thread and blocks of 256 threads. How many blocks fit on one SM, and which limit sets the number?
4. A warp reads 32 consecutive `int`s of 4 bytes each. How many 32-byte sectors does it touch? How many if the lanes read every 32nd `int`, and how many bytes are fetched in each case?
5. Why must all the threads of a block run on the same SM? What 2 features does this make possible?
6. How many steps does a shared-memory read take when all 32 lanes read the same 4-byte word? And when they read words 128 bytes apart?
7. A warp runs `if (i % 2 == 0) { A } else { B }`, where `i` is the thread's index and A and B each take 10 cycles. How long does the branch take, and how long would it take if every lane took the same side?
8. A launch of vector addition with N = 1,000,000 uses blocks of 256 threads. How many blocks does the grid need, and how many threads of the last block have no element to compute?
9. A kernel uses 41 registers per thread and blocks of 128 threads on an H100. How many registers are allocated per thread, how many blocks fit on an SM, and what is the occupancy?
10. A grid of 2,000 equal blocks runs on an RTX 4090 (128 SMs) whose SMs each hold 6 of its blocks. How many waves does it run, and what fraction of the last wave is busy? Suggest a block count close to 2,000 without a partial wave.
11. In a block of 32 × 8 threads, which warp holds the thread at (x = 7, y = 5)? Which rows of the block does each warp cover?
12. A memory-bound kernel reads 2 GB and writes 1 GB on an RTX 5090 in 2.0 ms. What percentage of the speed of light does it reach?
13. 32 lanes of a warp run a loop 1, 2, 3, …, 32 times respectively. What fraction of the warp's lane-iterations does useful work?
14. A GPU's memory bus is 256 bits wide and makes 16 billion transfers per second. What is its bandwidth? How wide would a bus at 5.23 billion

transfers per second need to be to match it?

15. A kernel stores into `__shared__ float t[32][32]` with `t[threadIdx.x][threadIdx.y]`, in blocks of 32 × 32 threads. How many steps does each warp's store take? How many with `t[32][33]`, and how many bytes of shared memory does the padding add?

Sources and further reading

- NVIDIA, *CUDA C++ Programming Guide*, "Hardware Implementation", "Compute Capabilities" and the description of independent thread scheduling: https://docs.nvidia.com/cuda/cuda-c-programming-guide/
- NVIDIA, *H100 Tensor Core GPU Architecture* whitepaper, and the H100 data sheet, for the H100 figures in this chapter.
- NVIDIA, *Volta V100 Architecture* whitepaper, for independent thread scheduling.
- NVIDIA, *Jetson Orin NX Series* and *Jetson Thor Series* module data sheets, the DGX Spark hardware overview (https://docs.nvidia.com/dgx/dgx-spark/hardware.html), and the GeForce RTX 4090 and 5090 specifications, for the other figures of section 2.1.4.
- NVIDIA, *Ampere*, *Ada* and *Blackwell* tuning guides, for the limits per SM in section 2.9: https://docs.nvidia.com/cuda/blackwell-tuning-guide/
- NVIDIA, *CUDA Occupancy Calculator* in Nsight Compute, for the register allocation unit of 256 registers per warp.
- NVIDIA, *H100 Tensor Core GPU Architecture* whitepaper, for the 5,120-bit HBM3 interface; the Jetson Orin NX and AGX Thor data sheets, for their 128-bit LPDDR5 and 256-bit LPDDR5X interfaces; the GeForce RTX 4090 specifications, for its 384-bit GDDR6X interface at 21 Gbps.
- Mark Horowitz, "Computing's Energy Problem (and What We Can Do About It)," IEEE International Solid-State Circuits Conference, 2014, for the energy of arithmetic and of memory accesses.
- David B. Kirk, Wen-mei W. Hwu and Izzat El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4th ed., Morgan Kaufmann, 2022.